C++ 第 1-11 章 学习提纲

——《C++ 语言程序设计（第 5 版）》全书 · 提纲式（含最小示例）

灰底标签 = 与 C 重叠的部分从简；示例仅演示写法，不含完整程序 · 按教材逐知识点核对整理

目录

第 1 章 绪论 1
1.1 计算机程序设计语言的发展 1
1.2 面向对象方法 1
1.3 面向对象的软件开发 1
1.4 信息的表示与存储 2
1.5 程序的开发过程 2
第 2 章 C++ 语言简单程序设计 2
2.1 C++ 语言概述与程序基本成分 2
2.2 基本数据类型和表达式 3
2.3 数据的输入与输出 4
2.4 算法的基本控制结构 4
2.5 类型别名与类型推断 4
2.6 深度探索（了解即可） 4
第 3 章 函数 5
3.1 函数的定义与使用 5
3.2 内联函数 6
3.3 constexpr 函数（C++11） 6
3.4 带默认形参值的函数 6
3.5 函数重载 6
3.6 使用 C++ 系统函数 6
3.7 深度探索（了解即可） 6
第 4 章 类与对象 7
4.1 面向对象程序设计的基本特点 7
4.2 类和对象 7
4.3 构造函数和析构函数 7
4.4 类的组合 8
4.5 UML 图形标识 9
4.6 结构体和联合体 9
第 5 章 数据的共享与保护 10
5.1 标识符的作用域与可见性 10
5.2 对象的生存期 10
5.3 类的静态成员 10
5.4 类的友元 11
5.5 共享数据的保护（const） 11
5.6 多文件结构和编译预处理 11
第 6 章 数组、指针与字符串 12
6.1 数组 12
6.2 指针 13
6.3 动态内存分配 15
6.4 用 vector 创建数组对象 15
6.5 深层复制与浅层复制 15
6.6 字符串 16
第 7 章 类的继承 16
7.1 基类与派生类 16
7.2 访问控制 17
7.3 类型兼容规则 17
7.4 派生类的构造函数和析构函数 17
7.5 派生类成员的标识与访问 18
7.8 深度探索（了解即可） 19
第 8 章 多态性 19
8.1 多态性概述 19
8.2 运算符重载 19
8.3 虚函数 20
8.4 纯虚函数与抽象类 21
8.7 深度探索（了解即可） 21
第 9 章 模板与群体数据 22
9.1 函数模板与类模板 22
9.2 线性群体 23
9.3 群体数据的组织（排序与查找） 24
第 10 章 泛型程序设计与 C++ 标准模板库（STL） 24
10.1 泛型程序设计及 STL 的结构 24
10.2 迭代器 24
10.3 容器的基本功能与分类 25
10.4 顺序容器 25
10.5 关联容器 26
10.6 函数对象 26
10.7 算法 27
第 11 章 流类库与输入输出 28
11.1 I/O 流的概念 28
11.2 输出流 28
11.3 输入流 29
11.4 输入输出流 30

第 1 章 绪论

1.1 计算机程序设计语言的发展

- **机器语言：**硬件可直接识别的二进制指令；晦涩难懂、难记忆
- **汇编语言：**机器指令的助记符（ADD、SUB）；抽象层次太低
- **高级语言：**屏蔽机器细节；20 世纪 60 年代末结构化程序设计语言（结构化数据/语句、数据与过程抽象）——但数据与操作分离
- **面向对象语言：**直接描述客观事物及其关系
- Smalltalk 是第一个真正的面向对象语言
- C++ 在 C 基础上扩充、兼容 C，是应用最广的面向对象语言

1.2 面向对象方法

- **由来：**早期面向过程（设计求解问题的过程）→ 结构化程序设计（自顶向下、逐步求精，顺序/选择/循环三种基本结构）→ 数据与操作分离、可重用性差、软件危机 → 面向对象方法
- **对象：**描述客观事物的实体，由一组属性 + 一组行为（服务）构成
- **类：**具有相同属性和服务的一组对象的集合；类与对象如模具与铸件，对象是类的实例
- **封装：**把属性和服务结合成不可分割的独立单位，并尽可能隐蔽内部细节，只保留有限对外接口（信息隐蔽）
- **继承：**特殊类拥有其一般类的全部属性与服务；简化认识与描述，对软件复用有重要意义
- **多态性：**一般类中定义的功能被特殊类继承后，可以有不同的具体实现（"绘图"在椭圆、多边形中各不同）
- 计算机系统 = 硬件系统 + 软件系统；程序是指令的集合，一台硬件能识别的全部指令 = 指令系统；1946 年 ENIAC 第一台数字电子计算机
- 汇编语言把机器指令映射为助记符，但抽象层次太低，程序员仍需考虑大量机器细节
- 高级语言（20 世纪 60 年代末结构化程序设计）：结构化数据、结构化语句、数据抽象、过程抽象；模块化实现手段是子程序；主要问题——数据和对数据的操作分离为独立实体，数据结构改变时所有相关处理过程都要修改，可重用性差；GUI 应用很难用过程描述
- **面向对象语言的根本不同点：**设计出发点就是直接描述客观世界的事物（对象）及它们之间的关系；问题域 = 软件要解决的业务范围
- **面向对象方法的基本特征：**类中大多数数据只能用本类的方法处理；类通过简单外部接口与外界发生关系，对象之间通过消息通信；模块独立性和数据安全性有保障
- **软件复用：**特殊类继承一般类本身就是软件复用；类可作为构件放入构件库

1.3 面向对象的软件开发

- **OOSE 五阶段：**OO 分析 → OO 设计 → OO 编程 → OO 测试 → OO 软件维护
- **分析：**建立说明系统"必须做什么"的模型，直接用问题域中的事物建模
- **设计：**把分析模型搬到设计阶段，并补充人机界面、数据存储、任务管理等实现相关部分
- **测试：**以类为基本测试单位，更准确发现错误、提高效率

- **维护**：封装使一个类的修改对其他类影响小
- 20 世纪 60 年代初期"软件危机"促进软件工程学形成；面向对象软件工程 = OO 分析/设计/编程/测试/软件维护五阶段
- **分析阶段**：精确抽象系统"必须做什么"而不是"如何实现"；直接用问题域中客观存在的事物建模，保留原貌、不做转换、不打破界限重新组合
- **设计**：①把分析模型直接搬到设计阶段作为系统设计一部分 ②针对实现补充人机界面、数据存储、任务管理等部分
- **编程**：用面向对象编程语言把设计模型的每个成分书写出来；学习 OOP 仍要注重基本思考过程，不能只学实现技巧

1.4 信息的表示与存储

- **数字系统**：只采用 R 个基本符号的数制称基 R 数制；每个固定位置对应的单位值称"权"；外界信息须转换为二进制
- **R 进制→十进制**：按权求和
- **十进制→R 进制**：整数"除 R 取余"、小数"乘 R 取整"
- **二↔八/十六进制**：按 3 位/4 位一组互转（小数点向两侧划分）
- 位 bit（最小单位）；字节 byte = 8 bit（KB=1024B、MB、GB）
- 字 word；机器字长 = 寄存器二进制位数，代表机器精度（32/64 位）
- **二进制的 4 个优点**：物理实现容易、运算规则简单、便于逻辑判断、抗干扰能力强（可靠性高）
- **原码/反码/补码**：补码 = 反码末位 +1（求反加 1）；正数三者相同；零的补码唯一
- **补码运算**：符号位可作数值参加运算；减法转换为加法（进位自然丢失）；运算结果仍为补码，为负时需再求补还原
- **补码的模数概念**：n 位整数的模 = 2^n；负数 −X 的补码 = $2^n - X$（"减一个数 = 加它的补数"，进位自然丢弃）；钟表是模 12 的直观例子
- 原码/反码的 0 有两种表示（+0/−0），n 位范围 −(2^(n−1)−1) ~ 2^(n−1)−1；补码 0 唯一，故范围多一个 −2^(n−1)
- **定点数与浮点数**：N = M×R^E；阶码 E 位数反映数的范围，尾数 M 位数反映精度；扩大范围加阶码位数、提高精度加尾数位数
- **m 位补码整数范围**：−2^(m−1) ~ 2^(m−1)−1（8 位：−128~+127）
- **溢出**：结果超出可表示范围（两个正数相加得负）
- ASCII 码用 7 位表示 128 个字符（95 可打印 + 33 控制字符）；机内占 1 字节，最高位作校验位
- 汉字 GB2312 双字节（两字节最高位置 1 与 ASCII 区分）
- **信息分两大类**：控制信息（指令、控制字）和数据信息（数值信息：定点数/浮点数；非数值信息：字符数据/逻辑数据）
- **表 1-1 进位记数制**：二进制基数 2 逢 2 进 1、八进制 8、十进制 10、十六进制 16（A~F 对应 10~15）
- **十进制→R 进制拼接规则**：整数部分余数从低位到高位、小数部分整数从高位到低位；十进制小数常常不能准确换算为二进制小数，存在换算误差
- **二→八/十六**：从小数点向左右按 3/4 位划分，中间（有效位之间）的 0 不能省略，两头不够补 0；八/十六→二：每位独立转 3/4 位，仅左边最高位不足时补 0
- 字长常用固定值 8、16、32、64 位；存储容量递进：1KB=1024B、1MB=1024KB、1GB=1024MB
- **模数系统**：8−2 = 8+10 (mod 12)，因为 2 与 10 对模数 12 互为补数（2+10=12）；模数系统中减法 = 加补数
- **求补码例**：X=−1010101 → [X]原=11010101，[X]反=10101010，[X]补=10101011（反码末位 +1）
- **补码运算例**：67−10 → [+67]补=01000011，[−10]补=11110110，相加 1 00111001，最高位进位自然丢失得 57；10−67 得负数补码，需再求补还原成真值
- **浮点数细节**：阶码和尾数都是带符号的数——尾数常用原码或补码，阶码多用补码；浮点格式随机器而异（例 16 位机：阶码 4 位 + 尾数 12 位）；阶码每加一位，数的范围扩大一倍
- **n 位定点小数范围**：原码/反码 −(1−2^{-n}) ~ (1−2^{-n})；补码 −1 ~ (1−2^{-n})
- 西文还有 EBCDIC 码（8 位，256 个编码状态）；汉字编码方案有 2/3/4 字节，GB2312 用两个 7 位二进制数表示一个汉字，两字节最高位置 1 与 ASCII（最高位 0）区分，由软件或硬件按字节最高位判断

1.5 程序的开发过程

- **基本术语**：源程序（待翻译）→ 目标程序（机器语言）
- **翻译程序三类**：汇编程序、编译程序、解释程序（边翻译边执行、效率较低）
- **完整过程**：编辑 → 编译（查语法错）→ 连接（查连接错）→ 运行调试
- **目标程序**：源程序经翻译加工后的程序（机器语言，或汇编/中间语言——编译目标是汇编语言时还需再汇编）
- **解释程序**：边翻译边执行、不产生整个目标程序、重复语句需重复解释，效率较低
- **小结补充**：C 语言 1972 年贝尔实验室 Dennis Ritchie 在 B 语言基础上开发；C++ 1980 年 Bjarne Stroustrup 创建；主要特点：全面兼容 C + 支持面向对象

第 2 章 C++ 语言简单程序设计

2.1 C++ 语言概述与程序基本成分

- **简史**：1980 年 Stroustrup 于 AT&T 贝尔实验室创建（"带类的 C"），1983 年正式命名；1998 ISO 国际标准，2011 C++11、2014 C++14
- **特点①**：全面兼容 C（更好的 C：类型系统扩充、编译期检查更多类型错误）
- **特点②**：支持面向对象
- **兼容的后果**：既支持面向过程也支持面向对象
- **最简单的程序**：#include <iostream> + using namespace std;
- 有且只有一个 main 函数（程序执行的开始，return 0 表示正常结束）
- **语句种类**：声明语句、表达式语句、选择语句、循环语句、跳转语句、复合语句
- **main 可带命令行参数**：int main(int argc, char *argv[])（argc = 参数个数，argv = 参数字符串数组）

```
#include <iostream>
using namespace std;
int main() {
    cout << "Hello!" << endl;
    return 0;   // 0 = 正常结束
}
```

C 基础 · 从简（C 语法大多相同，重点看 C++ 新增：bool、引用、范围 for、auto/decltype、constexpr、string）

- **字符集**：字母、数字、特殊字符
- **词法记号五类**：关键字 / 标识符 / 文字 / 操作符 / 分隔符
- **标识符规则**：字母或下划线开头，由字母、下划线、数字组成；大小写不同；不能是关键字
- **空白**：空格、制表符、换行、注释的总称，仅分隔记号
- **注释两种**：/* … */（不可嵌套）、// 到行尾
- **C 语言优点**：简洁灵活、运算符和数据结构丰富、结构化控制语句、执行效率高、兼有高级与汇编语言优点、可直接访问物理地址、可移植性好
- **标准化历程**：1994 ANSI 草案 → 1998 ISO 国际标准 → 2003 第二版 → 2011 C++11（核心新机能 + 扩展标准库）→ 2014 C++14（小扩展）
- **"更好的 C"**：类型系统改革扩充，比 C 更安全、编译能检查更多类型错误；C 代码与库函数大都不经修改即可用于 C++
- 有过程式经验者应着重学习面向对象特征
- **简单程序逐行解析**：main 前的 int = main 的返回值类型；cout 是系统预定义的输出流对象；<< 是输出操作符，把紧随其后的字符串输出到标准输出设备；endl = 换行符；return 0 = 以 0 退出（0 正常、非 0 异常）
- **#include <iostream>**：预处理指令，预处理时把 iostream 的代码嵌入指令所在处；iostream 声明了输入输出操作的有关信息；因被嵌入程序开始处故称头文件；用了系统提供的功能就必须嵌入相关头文件
- 程序文本存为扩展名 .cpp 的源文件，经编译、连接后产生可执行文件

- **字符集：**除字符型数据外，其他所有成分都只能由字符集中的字符构成（英文字母、数字字符、特殊字符）
- 词法记号 = 最小词法单元；关键字是语法预先规定的词汇；标识符例 Rectangle、Draw_line、_No1 合法，No.1、1st 不合法；注意 override、final 在特定上下文有特殊含义（语法标志而非普通标识符）；一些标识符保留给标准库应避免使用
- **分隔符：**( ) { } , ; : ' ——不表示任何实际操作，仅用于构造程序
- 空白 = 空格、制表符、垂直制表符、换行符、回车符和注释的总称；词法分析时把正文分解为词法记号和空白；空白只指示记号的开始和结束位置，其余被忽略；C++ 程序可不必严格按行书写；程序还要给人阅读，书写应清晰易读
- 注释不增加最终可执行程序的大小

2.2 基本数据类型和表达式

- **基本类型（IA-32，字节数）：**bool 1、char 1、short 2、int 4、long 4（64 位编译器 8）、long long 8
- float 4（≥7 位有效数字）、double 8（≥16 位有效数字）
- char 本质是 1 字节整数（存 ASCII）
- **整型常量：**十进制、0 开头八进制、0x 开头十六进制；后缀 L/LL/U
- 实型常量默认 double、后缀 F 变 float
- 字符常量单引号 + 转义序列（\\n \\t \\\\ 等）
- 字符串常量末尾隐含 '\\0'
- **字符/字符串前缀（表 2-3）：**u'…'=char16_t、U'…'=char32_t、L'…'=wchar_t、u8"…"=UTF-8 字符串（C++11）
- **通用转义序列：**\\nnn（1~3 位八进制）、\\xnnn（十六进制），如 '\\x41' 就是 'A'
- **变量先声明后使用；4 种初始化：**int a=0; / int a={0}; / int a(0); / int a{0}
- 列表初始化 { } 不允许信息丢失（double→int 拒绝编译）
- **默认初始化：**函数体外的基本类型变量 → 0；函数体内 → 值不确定（必须自己初始化）
- **存储类型：**register / extern / static / thread_local / mutable（仅类数据成员，使 const 属性无效）
- **符号常量：**const 类型 名=值; 声明时必须初始化、之后不可改
- **constexpr：**编译器验证的常量表达式，必须由常量表达式初始化
- % 只用于整型
- ++/-- 前置后置表达式值不同（cout<<i++ 输出旧值）
- 复合赋值 10 种
- 逗号表达式值 = 最右
- 关系/逻辑结果 bool
- **逻辑运算短路：**&& 前假不求后、|| 前真不求后
- 条件运算符 ? : 是唯一三元运算符，右结合，优先级高于赋值
- sizeof(表达式) 不对表达式求值
- **位运算（整数按位）：**& 置 0/取位、| 置 1、^ 异或翻转、~ 取反、<< >> 移位（移位不改变量本身；有符号右移补符号位）
- **优先级：**括号 > 单目 > 算术 > 移位 > 关系 > 位 & ^ | > && > || > ?: > 赋值（右结合）> 逗号；不确定就加括号
- **隐含转换：**低类型→高类型（char→short→int→long→long long→float→double，安全）
- **赋值一律转为左值类型：**浮点赋整只留整数部分、无符号超出取模、有符号超出未定义
- **赋值时的布尔转换：**算术/指针值 → bool（0 为 false、非 0 为 true）；bool → 整型（true 变 1）
- **显式转换：**类型(表达式)（C++ 风格）或 (类型)表达式
- **四种新式转换：**static_cast / dynamic_cast / const_cast / reinterpret_cast

```
int a = static_cast<int>(3.99);   // 3
bool b = 42;                      // true（非 0 即 true）
```

- **表 2-1 取值范围（IA-32）：**char −128~127；unsigned char 0~255；short −32768~32767；unsigned short 0~65535；int -2^{31}~$2^{31}-1$；unsigned int 0~$2^{32}-1$；long long -2^{63}~$2^{63}-1$；float 3.4×10^{-38}~3.4×10^{38}；double 1.7×10^{-308}~1.7×10^{308}
- ISO 标准只规定大小顺序 signed char ≤ short int ≤ int ≤ long int ≤ long long int，未规定字节数；IA-32 编译器 int 和 long 皆 4 字节，一些 64 位编译器 long 占 8 字节；无特殊要求用 int（处理效率高）
- **修饰符：**signed/unsigned、short/long；short int 的 int 可省略；long 可修饰 int 和 double
- char、signed char、unsigned char 是三种不同类型；char 默认是否有符号因编译环境而异
- **整型常量类型规则：**十进制默认带符号，类型取 int/long/long long 中能容纳的最小者；八/十六进制可能带符号也可能无符号，类型取 int/unsigned/long/unsigned long/long long/unsigned long long 中最小者；后缀 L=long、LL=long long、U=unsigned
- **实型常量指数形式：**0.345E+2 = 0.345×10^2（E 可大小写）；整数和小数部分可省略其一、不可都省
- **转义序列表 2-2（补全）：**\a 响铃、\n 换行、\t 水平制表、\v 垂直制表、\b 退格、\r 回车、\f 换页、\\ 反斜杠、\" 双引号、\' 单引号、\? 问号
- **字符串与字符不同：**"a" ≠ 'a'；字符串按字符顺序存放，每字符 1 字节，末尾加 '\0'
- 布尔常量只有 false 和 true
- **变量声明语句形式：**数据类型 变量名 1, 变量名 2, …, 变量名 n;（同语句可声明多个同类型变量）
- **声明 vs 定义：**声明只把标识符信息告诉编译器，不引起内存分配；定义分配内存空间；C++ 中大多数情况下声明也就是定义
- **初始化 ≠ 赋值：**初始化是创建变量时赋予初值；赋值是把变量当前值擦除、用新值替代；初始化值可为任意表达式，同一条定义语句可用先定义的变量初始化后定义的变量
- **存储类型 5 种细则：**register 存通用寄存器；extern 所有函数和程序段都可引用；static 固定地址存放、整个程序运行期有效；thread_local 只能修饰命名空间及块作用域中变量或已 static 的变量；mutable 只能用于类数据成员、不能与 const/static 同时使用、使其 const 属性无效允许更新；const 或 static 修饰的变量必然具有 static 存储类型性质
- **符号常量两种声明形式：**const 数据类型 常量名=值; 或 数据类型 const 常量名=值;（const float PI=3.1415926;）；声明时必须初始化、程序中不能再改；多处用同一文字常量时改符号常量即可，避免不一致
- **常量表达式：**值在编译阶段确定的表达式；判定例——const int max_size=100 是；const int limit=max_size+1 是；int student_size=30 不是（无 const）；const int size=get_size() 不是（需运行时确定）
- constexpr 变量隐含 const 属性，必须由常量表达式初始化；constexpr int size=get_size(); 仅当 get_size 是 constexpr 函数时才正确
- **表达式定义：**由运算符、操作数（常量、变量等）和括号组成；一个常量或标识符是最简单的表达式；表达式的值可参与其他运算形成更复杂的表达式
- **算术运算：**/ 用于两个整型相除结果取整，符号由两操作数符号决定；% 只用于整型，优先级与 / 相同
- **赋值表达式：**类型为左边对象类型、值为左边对象赋值后的值、右结合（a=b=c=5 从右向左）；10 种复合赋值前 5 个算术复合、后 5 个位复合，都是二元、优先级与 = 相同、右结合
- 关系运算注意 == 不要误写为 =
- **逻辑运算真值表（表 2-4）：**!a、&&、||；短路细节：第二操作数有副作用（如 ++c==1）时尤其要注意
- **条件运算符细则：**表达式 1 必须 bool 类型；表达式 2、3 可为任何类型且可不同（最终类型为两者中较高的类型）；优先级高于赋值、低于逻辑，右结合
- **sizeof 两种形式：**sizeof(类型名) 或 sizeof(表达式)
- **位运算细则：**& 可将若干位置 0（a=a&0xfe 最低位置 0）或取指定位（c=a&0xff 取低字节）；^ 与 0 异或保持原值、与 1 异或翻转该位（可翻转指定位）；~ 单目取反；<< 低位补 0、移出高位舍弃；>> 无符号高位补 0，有符号高位补符号位或补 0（VC++、gcc 采用补符号位）；移位只改变表达式值，不改变量本身
- **运算符优先级表 2-5 全部 17 级：**1 [] () . -> 后置++/--（左→右）；2 前置++/-- sizeof & * +(正) -(负) ~ !（右→左）；3 强制转换；4 .* ->*；5 * / %；6 + -；7 << >>；8 < > <= >=；9 == !=；10 &；11 ^；12 |；13

&&；14 ||；15 ?:（右→左）；16 赋值类（右→左）；17 逗号（左→右）

- **表 2-6 混合运算隐式转换：**有 long double→转 long double；有 double→double；有 float→float；整数类按 unsigned long long > (long long+unsigned long→unsigned long long) > unsigned long > (long+unsigned int→unsigned long) > unsigned int > long > int 逐级；位运算操作数必须是整数，二元位运算类型不同按表 2-6 转换
- 逻辑运算符要求操作数必须是 bool 型，其他类型编译系统自动转换
- **赋值转换 6 条规则（补全）：**①非布尔算术值赋给布尔：0→false，否则 true ②布尔赋给非布尔：false→0，true→1 ③浮点赋整数：只留整数部分 ④整数赋浮点：小数部分记 0，超出容量则精度损失 ⑤无符号赋超出范围的值：对该类型能表示的数值总数取模（8 位 unsigned char：−1→255）⑥带符号赋超出范围的值：结果未定义
- **显式转换注意：**高类型转低类型可能不安全（精度损失）；转换是暂时的、一次性的——int(z) 只临时转换，z 内存中的值并未真正改变

2.3 数据的输入与输出

- **I/O 流：**数据的流动抽象为"流"；cin 标准输入、cout 标准输出
- 从流取数 = 提取 >>，向流送数 = 插入 <<；<< 后可串联任意复杂表达式
- 格式控制需 #include <iomanip>
- dec/hex/oct 进制、endl 换行并刷新
- setprecision(n) 有效数字位数、setw(n) 域宽（只影响下一项）

```
int a, b;
cin >> a >> b;                          // 空格分隔
cout << setw(5) << setprecision(3) << 3.1415 << endl;
```

- **语句种类：**空语句、声明语句、表达式语句、复合语句、标号语句等；一个独立分号构成空语句（不产生任何操作）；表达式末尾加分号 = 表达式语句；一对花括号包围 = 复合语句
- C++ 没有赋值语句也没有函数调用语句——赋值与函数调用功能通过表达式实现；表达式可作为更复杂表达式的一部分，语句不能
- **I/O 流：**流在使用前要被建立、使用后要被删除；cin 处理标准输入（通常键盘）、cout 处理标准输出（通常屏幕）
- 提取符 >> 每个后跟用于存放输入值的变量；键盘输入两个 int 以空格分隔（输入 5 6 → a=5 b=6）
- 格式控制须先 #include <iomanip>；表 2-7 操纵符：dec 十进制、hex 十六进制、oct 八进制、ws 提取空白符、endl 插入换行符并刷新流、ends 插入空字符、setprecision(int) 设置浮点数的小数位数（表 2-7 注；默认方式为有效数字位数）、setw(int) 设置域宽

```
int year;  cin >> year;
bool leap = (year%4==0 && year%100!=0) || (year%400==0);
// 闰年
cout << setw(5) << setprecision(3) << 3.1415 << endl;   //
_3.14
```

2.4 算法的基本控制结构

C 基础 · 从简（if/switch/while/do…while/for 与 C 相同；范围 for 是 C++11 新增）

- **三种基本结构：**顺序、选择、循环；流程图符号：处理（矩形）、判断（菱形）、起止（圆角矩形）
- **if…else / else if：**各分支依次判断；else 与最近的未配对 if 配对，不确定就用 { }
- **switch：**表达式限整型/字符型/枚举型；case 常量不能重复；每个分支后加 break，否则顺序贯穿；无匹配走 default
- while（先判断）、do…while（先执行至少一次）、for(初始; 条件; 增量)——三者可互相替代，for(;;) 是死循环
- **范围 for（C++11）：**for (auto &r : 序列) 遍历每个元素；要修改元素必须声明为引用类型
- **break：**跳出循环体或 switch；continue：结束本次循环
- **goto：**破坏结构应少用，仅适合多重循环整体跳出
- **循环与选择可互相嵌套构成多重循环（例：**输出菱形图案、穷举法求因子）

```
string s = "abcde";
for (auto &r : s) r += 1;   // 每个字符+1，必须带 &
```

- **流程图符号（补全）：**处理（矩形）、判断（菱形）、数据输入输出（平行四边形）、预定义过程、开始和结束（圆角矩形）、联系（圆）、准备（六边形）、流线
- **if 细节：**语句 2 可为空，else 可省；语句 1/语句 2 可为单条语句或花括号复合语句
- **嵌套 if 注意：**每层 if 要与 else 配对，省略 else 时要用 { } 括起该层 if 以确定层次
- **switch 五条注意：**①switch 后表达式类型可为整型、字符型、枚举型 ②各 case 常量表达式的值不能相同，次序不影响结果 ③每个 case 分支可有多条语句但不必用 { } ④每个 case 分支后应加 break 结束整个 switch，否则从入口点一直执行到 switch 结束 ⑤若干分支执行相同操作时可共用一组语句
- **while 注意：**循环体中应包含改变循环条件表达式值的语句，否则造成死循环
- **do…while 与 while 区别：**一开始条件就为假时，do…while 至少执行一次循环体，while 一次都不执行；其余情况两者相同
- **for 三点说明：**①初始语句、表达式 1、表达式 2 都可省略，分号不能省；全省略 for(;;) 相当于 while(true)（死循环）②表达式 1 是循环控制条件，其中应包含不断变化的循环控制变量 ③初始语句可是表达式语句（可以是逗号表达式）或声明语句——声明的循环变量只在循环内有效，推荐 for (int i=1; i<=100; i++)
- 表达式 2（增量）省略时，须在循环体内另有语句改变循环控制条件；只剩表达式 2 的 for 完全等同于 while
- **范围 for 细则：**for (声明 : 表达式) 的表达式必须是序列（数组、vector、string 等拥有 begin/end 成员的类型）；序列每个元素必须能转换成循环变量类型，最简单用 auto；要修改元素必须声明为引用
- **break：**出现在 switch 或循环体中，跳出循环体和 switch，不宜用在别处；continue：结束本次循环，接着判断是否继续下一次
- **goto 语法：**goto 语句标号; ——语句标号是标识符，放在语句最前面、用冒号与语句分开；破坏程序结构应少用或不用；适宜场合：多重循环中整体跳出（break 难以直接做到时）

```
for (i=0, j=100; i<=j; i++, j--) k=i+j;   // 初始语句是逗号表达式
for (int i=1; i<=100; i++) sum += i;      // 声明式：i 只在循环内有效
string s = "abcde";
for (auto &r : s) r += 1;                 // 范围 for（等价传统：for (auto beg=s.begin(), end=s.end(); beg!=end; ++beg) { auto &r=*beg; r+=1; }）
```

2.5 类型别名与类型推断

- typedef 旧名 新名; 一句可声明多个别名
- using 新名 = 旧名;（C++11）一句一个别名
- **auto：**由初始值推断类型，必须有初始值；一条声明中只能有一种类型（auto i=0, pi=3.14 错误）
- **decltype(表达式)：**取表达式结果的类型，只分析不计算

```
using Area = double;          // = typedef double Area;
auto x = 1 + 2.5;             // double
decltype(x) y = 2;            // y 也是 double
```

2.6 深度探索（了解即可）

- **变量实现机制：**运行时变量值存内存，目标代码靠地址区分变量（反汇编：incl 0x80495f8 对地址中的值 +1）
- **声明 vs 定义：**声明只把名字告诉编译器，定义才分配内存；一个变量有且只能定义一次
- **表达式执行原理：**CPU 读写寄存器（IA-32 有 eax 等 8 个 32 位寄存器）；mov 传送、add 加法、imul 乘法；再复杂的表达式也是每步只执行一次运算
- **typedef 与 using 区别：**typedef 一句可声明多个别名（typedef double Area, Volume;）；using 一句只能声明一个别名（用 using 需写成两句）
- **decltype 使用需外加圆括号：**decltype(i) j=2; ——编译器分析表达式得到类型但不实际计算其值
- **变量实现机制：**不同名称的变量运行时占据不同内存单元、地址互不相同，目标代码靠地址区别变量；反汇编 = 把机器语言代码转换为对应的汇编语言代码（汇编与机器指令一一对应）

- **IA-32 通用寄存器 8 个：**eax、ebx、ecx、edx、esp、ebp、esi、edi，都是 32 位；mov 在寄存器和内存间交换数据；add 加法；imul 有符号整数乘法（64 位结果：低 32 位→eax，高 32 位→edx）
- **表达式执行原理：**再复杂的表达式也是一步一步执行、每步只执行一次运算；寄存器既存储变量值又保存中间结果
- **符号扩展与零扩展：**短类型→长类型，signed 执行符号扩展（符号位填充高位），unsigned 执行零扩展（0 填充高位）
- **一串二进制数只有与数据类型关联才有含义：**同样的 32 位串可以是 int −1082130432、unsigned 3212836864 或 float −1.0；同类操作（如 ++）对 int 和 unsigned 结果可能不同（溢出语义不同）
- 指令由操作符和操作数构成（如 incl = 加 1 操作）

```
int a=1, b=2;  a++;  b++;
// 反汇编（gcc 4.2 / IA-32）：
//   incl 0x80495f8   ← 对地址中的 a 加 1
//   incl 0x80495fc   ← 对地址中的 b 加 1
```

第 3 章 函数

3.1 函数的定义与使用

C 基础 · 从简（定义、调用、递归与 C 相同；重点看引用传递、默认形参、重载、内联、constexpr）

- **定义形式：**类型说明符 函数名(形式参数表) { 语句序列 }
- return 表达式; 返回值并结束函数；void 函数可不带 return
- **调用前先声明：**函数原型 = 类型说明符 函数名(形参表); 声明只告诉编译器信息，定义才产生代码
- **嵌套调用：**函数可调用其他函数；递归调用：函数直接或间接调用自身
- **值传递：**形参是实参的副本，形参改变不影响实参（单向传递）
- **引用：**变量的别名（int &r = i;），声明时必须初始化、之后不可改指
- **引用作形参：**对形参的操作直接作用于实参
- 引用作形参既可"改实参"又可避免大对象复制的开销；只读的大对象用 const 引用

```
void swap(int &a, int &b) {   // 引用传递
    int t = a; a = b; b = t;
}
void swap2(int a, int b) {    // 值传递：改的是副本
    int t = a; a = b; b = t;  // 实参不变
}
```

- **递归两阶段：**递推（把原问题分解为子问题，直到结束条件）→ 回归（从已知解逐一求值返回）
- **递归例：**n!、汉诺塔、组合数 comm(n,k)=comm(n−1,k)+comm(n−1,k−1)
- **系统函数：**#include <cmath> 中 fabs/sqrt/sin 等
- rand() 产生伪随机数，srand(seed) 设置种子
- **可变数量形参（3.1.3）：**initializer_list<double> lst ——元素同类型、个数任意；调用传 { 值列表 }，函数体内按范围 for 遍历

```
void print(initializer_list<double> lst) {
    for (auto d : lst) cout << d << " ";
}
print({1.0, 2.5, 3.0});   // 花括号列表
```

- **函数的作用：**面向过程程序设计中函数是模块划分的基本单位、对处理问题过程的抽象；面向对象程序设计中函数是功能抽象的基本单位；写好可重复使用，只关心功能和使用方法而不必关心实现——利于代码重用、提高开发效率、增强可靠性、便于分工合作和修改维护
- **主调函数与被调函数：**调用其他函数的是主调函数，被调用的是被调函数；一个函数可能在此关系中充当主调、在彼关系中充当被调
- **形参形式：**type1 name1, type2 name2, …, typen namen；形参表为空表示无任何形参；形参作用：实现主调函数与被调函数之间的联系（传递处理的数据、影响功能的因素或处理结果）
- **返回值细则：**main 的返回值最终传递给操作系统；void 函数可不写 return，也可写不带表达式的 return;
- **函数原型声明细节：**形参名可以省略，形参表只含完整类型信息即可（type1, type2, …, typen），但形参名可向编程者提示每个参数的含义；在所有函数之前声明的原型在本文件任何地方有效；在某个主调函数内部声明的原型只在该函数内部有效
- **调用形式：**函数名(实参列表)；实参是与形参个数相同、类型相符的表达式；调用可作一条语句（函数可以没有返回值），也可出现在表达式中（此时必须有明确返回值）
- **调用机制：**先计算实参列表各表达式的值 → 主调函数暂停执行 → 形参初值 = 实参表达式求值结果 → 被调函数执行到 return 或函数末尾 → 继续执行主调函数
- 可用 decltype 简化返回值类型定义：decltype(a) myMax(decltype(a) lhs, decltype(a) rhs) { return lhs>rhs? lhs:rhs; }
- **嵌套调用：**函数 1 调函数 2、函数 2 再调函数 3 形成嵌套调用

```
double power(double x, int n) {   // 例 3-1 求 x 的 n 次方
double val = 1.0;
while (n--) val *= x;
return val;
}
cout << power(5, 2);   // 25（调用出现在表达式中）

int fun2(int m) { return m*m; }              // 例 3-7
int fun1(int x, int y) { return fun2(x)+fun2(y); }  // 嵌套调用
```

- **递归细则：**函数直接或间接调用自身；直接调用自身=函数体中出现对自身的调用表达式；间接=fun1 调 fun2、fun2 又调 fun1；只有有限的递归调用才有意义，无限递归永远得不到解
- **递归两阶段（例 5!）：**递推 5!=5×4!→4!=4×3!→…→0!=1（从未知向已知推进，达到递归结束条件）；回归 0!=1→1!=1×0!=1→…→5!=120（按递推逆过程逐一求值）
- **递归注意：**同一函数的多次不同调用中，编译器为形参和局部变量分配不同空间、互不影响（fac(2) 调用 fac(1) 用 1 初始化被调用的 n，不会改变主调中的 n）
- **参数传递：**函数未被调用时形参不占有实际内存空间也没有实际值；被调用时才为形参分配存储单元并与实参结合（形实结合）；每个实参都是表达式，类型必须与形参相符
- **引用定义例：**int i, &ri=i; 后 ri=j 相当于 i=j——通过引用名与被引用变量名访问效果一样
- **引用形参的初始化时机：**不在类型说明时进行，而是在执行主调函数调用表达式时才为形参分配内存空间并用实参初始化；引用形参成为实参的别名，对形参的任何操作直接作用于实参
- 引用传递与值传递的区别只是形参写法不同，主调函数中的调用表达式完全一样

```
void fiddle(int in1, int &in2) {   // 例 3-13 值传递 vs 引用传递
in1 += 100;  in2 += 100;         // in1 改副本，in2 改实参
}
int v1=7, v2=12;  fiddle(v1, v2);    // 结果 v1=7 不变，v2=112 变
```

- **汉诺塔（例 3-10）：**n 个盘子从 A 移到 C（借助 B），每次只许移动一个盘且始终保持大盘在下；分解为三步：① A 上 n−1 个盘子移到 B（借助 C）② A 剩下一个盘子移到 C ③ B 上 n−1 个盘子移到 C（借助 A）；步骤①③是递归过程（hanoi），步骤②是基本操作（move）

```
void move(char src, char dest) { cout << src << " --> " << dest; }
void hanoi(int n, char src, char medium, char dest) {
if (n == 1) move(src, dest);
else {
hanoi(n-1, src, dest, medium);   // 借助 dest 移到 medium
move(src, dest);                 // 移动最底下的盘
hanoi(n-1, medium, src, dest);   // 借助 src 移到 dest
}
}
hanoi(3, 'A', 'B', 'C');   // 3 个盘子需 7 步
```

- initializer_list 是标准库类型（同名头文件），表示某种特定类型的值的数组；实参类型全相同时用它，类型不同时用可变参数模板（9.5 节）

- **操作：**① initializer_list<T> lst; 默认初始化（T 类型空列表）② initializer_list<T> lst{a,b,c…}; 元素数量和初始值一样多、元素是初始值的副本、列表中的元素是 const ③ lst2(lst)、lst2=lst 复制/赋值但不复制元素（原始列表和副本共享元素）④ lst.size() 元素数量 ⑤ lst.begin() 指向首元素的指针 ⑥ lst.end() 指向尾元素下一位置
- initializer_list 是类模板，使用时模板名后跟尖括号给出类型参数：initializer_list<string> ls; initializer_list<int> li;
- **特殊点：**其对象中的元素永远是常量值，无法修改
- 想向 initializer_list 形参传递值的序列必须把序列放在一对花括号中；可与其他类型形参同时使用：void print_err(initializer_list<string> lst, int error_code);

```
void print_err(initializer_list<string> lst) {
for (auto beg=lst.begin(); beg!=lst.end(); ++beg) cout <<
*beg << ' ';
}
print_err({"return", actual, "is", "expected"});  // 花括号序列
```

3.2 内联函数

- **背景：**函数调用有参数传递、控制转移的开销
- **内联函数：**编译时将函数体嵌入每个调用处
- **语法：**inline 类型 函数名(形参表) { … }
- inline 只是请求，编译器不承诺
- 适合简单、小、调用频繁的函数；直接递归的函数无法内联

```
inline int add(int a, int b) { return a + b; }  // 请求内联
```

3.3 constexpr 函数（C++11）

- **能用于常量表达式的函数：**返回类型和所有形参类型必须是字面值类型，函数体中有且仅有一条 return 语句
- 编译时验证并替换成结果值；被隐式指定为 inline

```
constexpr int get_size() { return 20; }
```

3.4 带默认形参值的函数

- **定义时预先给形参默认值：**调用给实参用实参，否则用默认值
- **规则：**有默认值的形参必须在形参表最右（右侧不能再有无默认值的形参）
- 默认值在函数原型中给出，同一作用域不可重复给出

```
int add(int x, int y = 5, int z = 6);  // 对
// int add(int x = 1, int y, int z);   // 错：默认值不在最右
```

- **内联注意：**现代编译器中没用 inline 修饰的函数也可能被编译为内联函数；复杂函数内联会造成代码膨胀、增大开销，多数编译器自动转换为普通函数（什么算复杂各编译器不同）
- **constexpr 函数约定：**返回类型及所有形参类型必须是字面值类型，函数体中有且仅有一条 return 语句；函数体也可包含空语句、类型别名、using 声明等（只要运行时不执行任何操作）
- constexpr 函数并不一定返回常量表达式：constexpr int len(int arg){return get_size()*arg;} ——仅当实参是常量表达式时返回值才是常量表达式
- **数组长度必须用常量表达式：**int arr[len(2)]; 正确（len(2) 是常量表达式）；int i=2; int a2[len(i)]; 错误
- **默认形参细则：**实参与形参按从左向右顺序建立对应关系，所以有默认值的形参必须在最右；同一作用域内不允许在同一函数的多个声明中重复定义同一参数的默认值（即使值相同也不行）；函数定义也算声明——原型给了默认值，定义处不能再给，用注释说明是好习惯：int add(int x/*=5*/, int y/*=6*/){ … }
- **默认形参+重载的二义性陷阱：**void fun(int length, int width=2, int height=3); 与 void fun(int length); 当调用 fun(1) 时编译器无法确定执行哪个，指出语法错误

```
int add(int x=5, int y=6);   // 原型中给默认值
add(10, 20);   // 30
add(10);       // 16（y 用默认值 6）
add();         // 11（x=5, y=6）

int getVolume(int length, int width=2, int height=3);  // length 无默认，必须给
```

3.5 函数重载

- 同一作用域内函数名相同、参数表不同（个数或类型）即重载；编译器按实参选择最佳匹配
- 仅返回值类型不同不构成重载
- 避免歧义（如只靠隐式转换区分的重载）

```
int    add(int a, int b);
double add(double a, double b);    // 重载：参数表不同
// int add(int a, int b, int c);   // 也是重载（个数不同）
```

- **重载的引入：**自然语言一词多义（"擦桌子、擦皮鞋、擦车"都用"擦"字）；C++ 允许对不同功能赋予相同函数名，编译时根据上下文（实参类型和个数）确定使用哪一具体功能；没有重载机制时，不同类型数据的加法需定义 int add / float fadd 等不同名函数，调用不便
- **重载规则细则：**两个以上函数同名、形参个数或类型不同即重载；编译程序对实参和形参类型及个数进行最佳匹配；函数名相同、形参类型也相同（无论返回值是否相同）→ 语法错误（函数重复定义）
- **错误示例：**int add(int x,int y); int add(int a,int b); 错误（编译器不以形参名区分）；int add(int x,int y); void add(int x,int y); 错误（编译器不以返回值区分）
- **习惯：**不要将不同功能的函数定义为重载函数，以免误解混淆——如 int add(int x,int y){return x+y;} 与 float add(float x,float y){return x-y;}

```
int    sumOfSquare(int a, int b)    { return a*a + b*b; }
// 例 3-16
double sumOfSquare(double a, double b) { return a*a + b*b; }  // 重载
cout << sumOfSquare(m, n);   // 编译器按实参类型选版本
```

3.6 使用 C++ 系统函数

- **标准库函数按功能分头文件：**cmath（数学）、cstring（字符串）、cstdlib 等
- 查编译系统文档了解原型与用法

3.7 深度探索（了解即可）

- **运行栈：**调用前实参压栈、call 跳转、返回时 leave 恢复；形参和局部变量每次调用各占一份空间
- 函数原型 = 主调函数与被调函数之间的协议（参数个数/类型/返回类型）；编译器据此检查调用
- **C++ 比 C 安全：**C 允许不完整声明（只写函数名和返回类型），参数类型不匹配也不报错 → 隐蔽错误；C++ 有原型就检查
- **系统函数：**系统库提供几百个函数（sqrt、abs 等）；系统函数的原型声明已由系统提供、分类存于不同头文件，程序员只需用 include 嵌入相应头文件即可使用
- **编译环境的系统函数分两类：**标准 C++ 函数（各编译环境普遍支持，优先使用、可移植性好）和非标准 C++ 函数（特定操作系统或编译环境特有）
- **头文件 c 前缀表示继承自标准 C：**cmath/cstdlib/cstdio/ctime ↔ C 中的 math.h/stdlib.h/stdio.h/time.h；C++ 允许继续用 .h 头文件仅为兼容，编写 C++ 程序应尽量使用不带 .h 的头文件；常用函数可查 cppreference.com
- **运行栈原理：**局部变量不能像全局变量那样用固定地址——① 局部变量生存期远小于程序运行周期，各占固定地址降低空间利用率 ② 递归时同一函数的多次调用同名局部变量有不同值，必须同时保存、互不影响，故地址必须不同；形参与局部变量存储在栈（运行栈）中
- **栈：**只能从一端（栈顶）存入（压入）和取出（弹出），另一端是栈底；后进先出 LIFO（餐厅盘子：只能从最上面拿/放）；嵌套调用越早开始越晚结束，与 LIFO 吻合
- **栈帧：**运行栈中数据分为一个个栈帧，每个栈帧对应一次函数调用，包含形参值、控制信息、局部变量值和临时数据（表达式中间值、某些返回值）；调用时压入栈帧、返回时弹出；执行中的函数栈帧总在栈顶，可直接随机访问
- **参数传递实现：**主调函数调用前把实参值压入运行栈，这部分空间主调与被调都可直接访问，形实结合就是通过访问这部分公共空间完成；形参和局部变量地址相对于栈顶/帧指针是确定的
- **IA-32 调用执行细节：**esp = 栈指针（记录栈顶地址）；ebp = 帧指针（保存刚被调用时的栈指针位置，因有些函数栈帧大小不固定）；形参地址用帧指针计算：add(int a, int b) 中 a=ebp+8、b=ebp+12、局部变量

c=ebp−4；call 指令把返回地址（call 下一条指令地址）压栈；被调函数第一条 push %ebp 保存原帧指针（"控制信息"=ebp 原值+返回地址）；整型返回值经 eax 寄存器传递；leave 指令恢复 esp 和 ebp 到调用之初状态

- 其他微处理器对函数调用处理方式有差异，但通过运行栈存储形参和局部变量这一点是共同的

```
// int add(int a, int b){ int c=a+b; return c; } 的主调汇编（gcc -O0）：
movl $0x7, 0x4(%esp)   // 参数 7 写入栈
movl $0x5, (%esp)      // 参数 5 写入栈
call 8048434           // 调用并把返回地址压栈
mov %eax, -0x8(%ebp)   // 返回值从 eax 取出存 x
```

- **类型安全细则：**运行时数据只以二进制序列表示、不保存类型信息，类型特性全部蕴含在所执行的操作中；add 例：若调用前未把实参放入栈顶或放入 float，add 仍被调用，读 ebp+8/ebp+12 得到的是不可预料的整数值——多数情况函数仍能执行完毕、不主动报错（任何 32 位二进制序列都可解释为 int），只是结果无意义
- **void 函数当整型调用：**返回后读取 eax 得到的不是返回值而是不可预料的值，计算机不会主动提示错误；调用前声明函数原型即可避免这两类错误
- 函数原型 = 主调函数和被调函数之间的协议：主调依据它把数量和类型正确的参数压入运行栈；被调依据它存储类型正确的返回值
- **C 比 C++ 不安全的细节：**C 允许不完整声明（只声明函数名和返回类型，不声明参数类型），甚至允许调用前根本不声明；C 中声明函数时括号内为空并不表示无参数，而表示参数未知——声明无参函数应写入 void；C 反例 double add(); s=add(1,2); 编译器不对 1、2 做类型转换（作为整型压栈），结果不正确
- **设计原则：**一个错误与其被淹没在运行中，不如暴露在编译时——设计良好的语言应尽量多地在编译时检测错误（C++ 比 C 做得好）

第 4 章 类与对象

4.1 面向对象程序设计的基本特点

- **抽象：**对具体问题分类，只关注与目标有关的本质特征（数据抽象 + 过程抽象）
- **封装：**将数据和操作结合成类，通过访问控制隐蔽内部细节
- **继承与多态：**第 7、8 章详述

4.2 类和对象

- **类定义：**class 类名 { public: 公有成员; private: 私有成员; };（分号不可省）
- **访问控制：**public 外部可访问；private 只能本类成员函数访问；protected 本类及派生类；默认 private
- **类内初始值（C++11）：**声明数据成员同时给初值
- **对象定义：**类名 对象名; 访问成员用 对象.成员 / 指针->成员
- 成员函数可在类内定义（隐式内联）或类外定义（类名::函数名）
- 成员函数调用时 this 指向目的对象
- 成员函数可访问本类任何对象的私有成员（不限于 this）：void Point::copy(const Point &p) { x = p.x; }

```
class Clock {
public:
    void setTime(int h=0, int m=0, int s=0);
    void showTime() const;
private:
    int hour, minute, second;
};
void Clock::setTime(int h, int m, int s) { hour=h; minute=m; second=s; }
```

- **多态（第 4 章视角）：**广义多态性 = 一段程序能够处理多种类型对象的能力；C++ 中 4 种形式——强制多态（数据类型转换，隐式或显式）、重载多态（函数重载、运算符重载，给同一名字赋予不同含义）、类型参数化多态（模板）、包含多态（虚函数）；强制多态和重载多态属于特殊多态性（只是表面的多态性）；包含多态和类型参数化多态属于一般多态性（真正的多态性）；虚函数是多态性的精华
- **类的地位：**面向过程程序设计中程序模块由函数构成，面向对象程序设计中程序模块由类构成；类是对逻辑上相关的函数与数据的封装，集成程度更高，更适合大型复杂程序开发
- **类定义语法形式：**class 类名称 { public: 外部接口 protected: 保护型成员 private: 私有成员 };; 类中可以只声明函数原型，函数体在类外定义
- **类内初始值：**创建对象时用于初始化数据成员，没有初始值的成员被默认初始化；必须以等号或花括号表示（int hour=0 或 int hour{0}）
- **变量声明的双重意义：**不只分配存储，还限定对变量的操作——对 int 可做算术运算、关系运算，对 bool 只能做逻辑运算、关系运算
- **访问控制细则：**私有成员紧接着类名称则 private 可省略；习惯上一个类的数据成员都应声明为私有（内部结构不对外造成影响，模块间相互作用降到最小）；保护成员性质和私有成员相似，差别在继承过程中对产生的新类影响不同；图 4-1 比喻：私有成员=黑盒子，公有成员=透明（对外接口），保护成员=笼子（给派生类的特殊访问属性）
- **成员声明次序：**不同访问属性的成员可按任意顺序出现，关键字可多次出现，但一个成员只能有一种访问属性；书写习惯把公有类型放在最前面便于阅读
- **对象的内存：**对象只占据存放数据成员的空间，函数成员不在每个对象中存副本——每个函数的代码在内存中只占一份；访问：对象名.数据成员名 / 对象名.函数成员名(参数表)；类外只能访问公有成员，类的成员函数中可以访问类的全部成员
- **成员函数实现形式：**返回值类型 类名::函数成员名(参数表) { 函数体 }——成员函数名需用类名限制（Clock::showTime）；类外实现时须指明类名
- **目的对象：**调用成员函数时用 . 操作符指出调用所针对的对象（myClock.showTime() 中 myClock 是目的对象）；成员函数体内可直接引用目的对象的数据成员（不用 . ）；引用其他对象的属性或调用其他对象的方法必须用 . 操作符
- **带默认形参值的成员函数：**规则同普通函数，但默认值一定要写在类定义中，不能写在类外的函数实现中——void setTime(int newH=0, int newM=0, int newS=0);
- 内联成员函数两种声明方式（效果完全相同）：隐式声明=函数体直接放在类体内；显式声明=类外实现时返回值类型前加 inline（inline void Clock::showTime(){…}）；权衡：减少调用开销但增加编译后代码长度，只有相当简单的成员函数才声明为内联

```
class Clock {
public:
void setTime(int newH=0, int newM=0, int newS=0);  // 默认值写在类内
void showTime();              // 原型在类内，实现在类外
private:
int hour=0, minute=0, second=0;   // 类内初始值（= 或 {}）
};
void Clock::setTime(int h, int m, int s) { hour=h; minute=m; second=s; }  // 类名::限制
```

4.3 构造函数和析构函数

- **构造函数①：**函数名与类名相同
- **构造函数②：**无返回值
- **构造函数③：**通常 public；对象创建时自动调用，完成初始化
- **默认构造函数：**调用时无须提供参数；类中一旦声明了任何构造函数，编译器就不再生成隐含的
- **初始化列表：**构造函数 : 成员 1(初值), 成员 2(初值) { }
- 初始化列表是唯一能为常数据成员、引用成员、无默认构造的成员对象初始化的方式

```
class Point {
public:
    Point(int x = 0, int y = 0) : x(x), y(y) {}
private:
    int x, y;
};
Point a;        // 调 Point(0,0)
Point b(3, 4);  // 调 Point(3,4)
```

- **委托构造函数（C++11）：**构造函数把自身的一些（或全部）初始化职责委托给本类另一个构造函数

- **委托构造执行次序：**受委托的构造函数的初始值列表和函数体依次执行完，控制权才交还委托者函数

```
Clock(int h, int m, int s) { … }  // 普通构造函数
Clock() : Clock(0, 0, 0) {}       // 委托构造：初始化列表里调另一个
```

- **复制构造函数一般形式：**类名(类名 &对象名); ——类外实现写 类名::类名(类名 &对象名) { 函数体 }
- **复制构造函数的作用：**用一个已存在的同类对象（由形参指定）去初始化同类的新对象
- 未定义复制构造 → 系统生成隐含复制构造函数：把初始值对象的每个数据成员值复制到新对象（克隆 clone）
- **复制构造三种调用场合①：**用已有对象初始化新对象——Point b(a); 与 Point c = a; 两种写法等价
- **复制构造三种调用场合②：**对象作函数的值传递形参——传引用不调复制构造；大对象宜传 const 引用避免复制开销
- **复制构造三种调用场合③：**函数返回值为对象——返回时调用复制构造生成无名临时对象
- **深复制与浅复制：**默认复制构造逐成员复制——含指针成员时只复制地址（浅层），两个对象共享同一资源；需要时自定义复制构造函数逐成员深复制
- **析构函数：**~类名()，无返回值无参数；对象离开作用域时自动调用做清理；不显式写则由编译器生成
- **移动构造函数（C++11）：**Point(Point &&p) 接管右值的资源而非复制
- **左值/右值：**左值 = 有名字、可取地址的对象；右值 = 临时值/字面量（不可取地址）
- **右值引用：**类型 &&r = 右值; 只能绑定右值；std::move(x)（<utility>）把左值"转交"为右值引用——"我要搬走它，别再复制"
- 移动构造/移动赋值一般声明为 noexcept（否则某些场景会退回报复制）
- **析构细节：**无参数故不能重载；对象数组消亡时每个元素分别调用析构函数
- **= default：**显式要求编译器生成；= delete：禁止使用（如禁止复制构造）

```
class Point {
public:
    Point(int xx=0, int yy=0) { x=xx; y=yy; }  // 构造函数
    Point(Point &p);            // 复制构造函数（声明）
    int getX() { return x; }
private:
    int x, y;
};
Point::Point(Point &p) {        // 类外实现
    x = p.x;  y = p.y;
}
int n = 5;
float &&rr = std::move(n);   // 右值引用
```

- **构造函数的由来：**对象建立时向操作系统申请内存，但类对象太复杂、编译器不知道如何产生初始化代码——需要对象初始化时程序员要编写初始化程序；不编写初始化程序却在声明对象时贸然指定初值，不仅不能实现初始化，还会引起编译时语法错误；C++ 严格规定了初始化程序的接口形式（构造函数）并有自动调用机制
- **构造函数的特征：**类的成员函数；函数名与类名相同；没有返回值；通常声明为公有函数；只要类中有构造函数，编译器就在建立新对象的地方自动插入调用代码
- **构造函数也是成员函数：**可以是内联函数、可以带参数表、可以带默认形参值、可以重载
- **初始化列表效率：**使用初始化列表对成员初始化比在函数体中用赋值语句效率更高——直接用初始值初始化、不需要复杂计算时首选初始化列表
- **隐含默认构造函数：**类中没写构造函数时编译器自动生成（参数列表和函数体皆空）；生成这个"不做任何事"的函数是因为"建立对象时自动调用构造函数"是 C++ 的例行公事；但函数体空的构造函数并非不做任何事——还要负责基类构造和成员对象构造（组合和继承章节）
- **自定义带参构造后：**Clock c(0,0,0); 调用有参构造；Clock c2; 编译指出语法错（没有给出必要的实参）——所以常把有参构造与无参构造两种重载形式同时给出
- **默认构造函数三原因（4.3.2，重要）：**① 编译器只在类不包含任何构造函数时才生成默认构造函数——一旦定义了其他构造函数，除非自己再定义一个，否则类没有默认构造函数 ② 合成的默认构造函数可能执行错误操作——块中基本类型默认初始化后值未定义，含基本类型成员的类应在类内初始化这些成员或自定义默认构造函数 ③ 有时编译器无法合成——类包含一个无默认构造的类类型成员时必须自定义
- **合成默认构造函数的初始化规则：**存在类内初始值就用类内初始值初始化成员；否则以默认方式初始化成员（基本类型成员默认是未定义的值）
- **深复制注意（p132）：**共享数据成员时修改可能影响复制来源对象；应避免不必要的复制构造函数调用；默认复制构造函数是浅复制（只复制对应数据成员），必要时编写复制构造函数实现深复制（深复制可能带来数据安全问题，需权衡）
- **析构函数细则：**对象消失前的扫尾工作——构造时若分配了资源（如动态申请内存），对象消失时就要释放；局部对象在函数运行结束返回调用者时消失；析构与构造作用几乎正好相反，在对象生存期即将结束时被自动调用，调用完成后对象消失、内存空间被释放；名称是类名前加 ~；没有返回值；不接受任何参数，但可以是虚函数；不显式说明时系统生成函数体为空的隐含析构函数；函数体为空的析构函数未必不做任何事
- **移动构造函数细则（4.3.6）：**被复制对象仅作复制之用后销毁时，移动而非复制可大大提高性能；左值 = 赋值语句左侧的对象变量，右值 = 赋值语句右侧的值（不依附于对象）；对短暂存在可被移动的右值的引用称右值引用；float &&rr_n = n*n; 正确，float &rr_n = n*n; 错误（左值引用不能绑定右值）；左值对象不能绑定右值引用
- std::move（<utility>）：把左值对象移动成右值——move 告诉编译器将变量当作右值使用，承诺除对其重新赋值或销毁外不再通过其他方式使用它
- **移动构造函数执行：**参数为该类对象的右值引用；构造中移动源对象的资源（不分配新资源新内存，理论上不会失败）；构造后源对象不再指向被移动的资源，可重新赋值或被销毁；为配合编译器异常捕获机制须声明 noexcept
- **=default 与 =delete：**=default 可要求编译器合成默认/复制/移动构造和析构（如 MyStr()=default;）；其他带参构造函数编译器不知构造逻辑，需自定义；=delete 将复制构造删除可禁止复制（MyStr(MyStr &str)=delete;）；delete 不限于无参和复制构造函数——除析构函数外都可指定 delete

```
class MyStr {
public:
MyStr() = default;                  // 合成默认构造
MyStr(string _s) : s(std::move(_s)) {}  // 有参构造须自定义
MyStr(MyStr &str) = delete;         // 禁止复制
~MyStr() = default;                 // 合成析构
private:
string s;
};
```

4.4 类的组合

- **组合：**一个类的成员是另一个类的对象（has-a）；成员对象（内嵌对象）用初始化列表初始化
- **内嵌对象初始化写法：**类名::类名(总形参表) : 内嵌对象 1(形参), 内嵌对象 2(形参), … { 本类函数体 }
- **构造顺序：**先调内嵌对象构造函数（按类定义中的声明次序，与初始化列表书写顺序无关）→ 再执行本类构造函数函数体；析构顺序相反
- 内嵌对象未出现在初始化列表 → 调它的默认构造函数（无形参）
- **必须在初始化列表初始化的两类成员：**① 无默认构造函数的内嵌对象 ② 引用类型数据成员；含这两类的类编译器不提供隐含默认构造，必须写显式构造且初始化列表至少初始化它们
- **前向引用声明：**class B; 只声明类名；此时只能使用该符号，不能涉及类的任何细节（不能定义对象成员，内联函数不能使用）

```
class Line {                    // 组合：成员是 Point 对象
    Point p1, p2;
public:
    Line(Point a, Point b) : p1(a), p2(b) {}  // 先初始化内嵌对象
};
```

- **组合的正式定义：**类的组合描述一个类内嵌其他类的对象作为成员的情况，它们之间是包含与被包含的关系；创建类的对象时如果类具有内嵌对象成员，各个内嵌对象将被自动创建（部件对象是复杂对象的一部分）
- **组合类复制构造函数：**编译系统必要时自动生成隐含复制构造函数，自动调用内嵌对象的复制构造函数为各个内嵌对象初始化；自定义时写法：Line::Line(Line &l) : p1(l.p1), p2(l.p2) { len=l.len; }
- **例 4-4 运行分析：**Point 类复制构造函数在整个程序中被调用 6 次——Line 构造函数形参形实结合时 2 次、初始化内嵌对象 p1/p2 时 2 次、复制构造 line2 时 2 次
- **前向引用声明细节：**两个类相互引用（循环依赖）时，无论哪个类定义在前都会引起编译错误；解决办法：class B; 前向引用声明——把类名告诉编译器，完整定义可放在程序其他地方
- **前向引用声明的限制：**只能使用被声明的符号、不能涉及类的任何细节——不能定义该类的对象作为数据成员（可声明该类对象的指针或引用）；类 Barney 的内联函数 method() 中使用了 Fred 类对象而 Fred 尚未完整定义 → 编译错误；解决：更改类的定义次序，或把 method() 改为非内联形式并在 Fred 完整定义之后给出函数定义

```
class Line {
Point p1, p2;     // 内嵌对象成员
double len;
public:
Line(Point xp1, Point xp2) : p1(xp1), p2(xp2) {
double x = p1.getX()-p2.getX(), y = p1.getY()-p2.getY();
len = sqrt(x*x + y*y);
}
Line(Line &l) : p1(l.p1), p2(l.p2) { len = l.len; }  // 调内嵌对象复制构造
};
```

4.5 UML 图形标识

- **UML：**面向对象建模语言（不是编程语言）；类图是静态结构图
- **类用三段矩形表示：**类名 / 属性 / 操作
- **访问控制符号：**+ 公、− 私、# 保护；对象名加下划线
- **类间关系图形（耦合由弱到强）：**依赖（虚线箭头）→ 关联（实线）→ 聚集/组合（菱形端）→ 泛化/继承（空心三角箭头指向基类）
- **UML 简史：**1994.10 Booch 和 Rumbaugh 将 Booch 方法与 OMT 方法统一为 UM 0.8；1995 年底 Jacobson 加盟；1996.6 发布 UML 0.9；1997.11.17 OMG 采纳 UML 1.1；2003.6 OMG 通过 UML 2.0；UML 是标准化建模语言，不是一种编程语言
- UML 图形描述软件模型的静态结构、动态行为和模块组织管理；类图（Class Diagram）属于静态结构图；对象图（object diagram）是类图的特例（只含对象不含类），使用相当有限，UML 1.5 以后工具软件可以不实现
- **类三段矩形：**顶部类名、中间属性（数据成员）、底部操作（函数成员）；除名称外其他两部分可选——只写类名的矩形就可以代表一个类；两种表示场合：详细描述成员及访问控制属性时用完整表示，着眼点在类之间的关系时用简洁表示
- **数据成员表示语法：**[访问控制属性] 名称[重数][:类型][=默认值][{约束特征}]——例：- hour : int（私有，名称 hour，类型 int）；+size: area=(100,100)（公有，类型 area，默认值 (100,100)）；重数=多重性；约束特征如 {只读}
- **函数成员表示语法：**[访问控制属性] 名称[(参数表)][:返回类型][{约束特征}]；参数表按"[方向]名称: 类型=默认值"给出，方向指明 in（输入）/ out（输出）/ inout（输入输出），与 .cpp 文件格式不同；例：+setTime(newH: int=0, newM: int=0, newS: int=0) : void
- **对象表示：**一个矩形，对象名加下划线；上部区域"对象名: 类名"冒号隔开，可省略对象名或类名
- **依赖关系：**一个事物变化可能影响使用它的另一个事物，反之不成立；一个类把另一个类作为函数成员参数时使用依赖关系，常用 <<use>> 构造型；UML 绘成指向被依赖事物的虚线（A 使用 B / A 依赖 B）
- **关联（作用关系）：**一个类的对象和另一个类的对象之间相互作用的连接；用实线表示，线段两端通常包含多重性（重数是关联最重要的特性）；表 4-1 重数标记：*（任何数目含 0）、1（恰好一个）、n（恰好 n 个）、0..1（0 或 1 个，关联可选）、n..m（最少 n 最多 m）、2..4（离散结合）
- **包含关系（聚集和组合）：**整体与部分关系；聚集分共享聚集（部分可参加多个整体，如课题组成员）和组合（整体拥有各部分、部分与整体共存——整体不存在部分随之消失，如视窗口由标题、外框、显示区组成）；UML 中聚集=空心菱形，组合=实心菱形
- **例 4-5（Line 与 Point）：**Line 数据成员含两个 Point 对象（重数 2）→ 用聚集关系；Line 构造函数使用 Point 对象的公有函数 → 用依赖关系描述
- **继承关系（泛化）：**UML 中称泛化；带空心三角形的直线段——三角尖指向父类，对边上的线指向子类；单继承与多继承
- **注释：**UML 中最重要的能独立存在的修饰符号；表示为带褶角的矩形，用虚线连接到 UML 元素上

```
┌──────────────────────────┐
│         Clock            │   类 = 三段矩形
├──────────────────────────┤   数据成员语法：
│ - hour : int             │   [访问控制] 名称[:类型][=默认值]
│ - minute : int           │
│ - second : int           │   函数成员语法：
├──────────────────────────┤   [访问控制] 名称(参数表):返回类型
│ + setTime(newH:int=0,    │
│   newM:int=0,            │   + 公  - 私  # 保护
│   newS:int=0) : void     │   对象名加下划线
│ + showTime() : void      │
└──────────────────────────┘
```

4.6 结构体和联合体

C 基础 · 从简（struct 与 C 类似但可含函数、默认 public；union 是特殊形态的类）

- **结构体 struct：**默认 public，可整体赋初值 Student stu={97001,"Lin Lin",'F',19};（类成员私有故一般不行）
- **联合体 union：**全部数据成员共享同一组内存单元，至多一个成员有效
- **union 限制：**不能有自定义构造/析构/重载赋值，不能继承、不支持多态
- **无名联合体：**union { int i; char c; }; 成员地址相同，可由成员名直接访问

```
Student stu = {97001, "Lin", 'F', 19};   // struct 整体初始化
union { int i; char c; };   // 无名联合体：i 和 c 占同一地址
```

- **结构体与类的唯一区别：**默认访问控制属性不同——类中未指定访问控制的成员默认 private，结构体中默认 public；结构体把公有成员放最前面时可省去 public:
- **保留 struct 的历史原因：**C 语言只有结构体没有类；C 结构体只允许数据成员、全部成员公有、没有访问控制概念；C++ 为结构体引入成员函数、访问权限、继承、多态等面向对象特性；因 structure 表示这种抽象数据类型不再贴切，C++ 引入 class 作为首选关键字；为保持与 C 程序兼容保留 struct 并规定其默认访问权限为公有
- **风格建议：**与类不同，结构体习惯将数据成员设为公共；只需把不同类型数据组合成整体方便保存（没什么操作）的类型适合定义为结构体——若用类定义，为遵循"数据私有"的习惯要为每个数据成员编写读写函数，反而麻烦
- **聚集体初始化条件：**一个结构体全部数据成员都是公共成员、没有用户定义的构造函数、没有基类和虚函数，其变量可用 类型名 变量名={成员 1 初值, 成员 2 初值, …}; 初始化（类对象语言规则上也可，但类一般不满足条件）
- **联合体的引入场景：**一组数据中有的数据只在一些情况下有效、另一些数据在其他情况下有效（不会同时有效），如成绩——等级制用字符、只记通过与否用布尔、百分制用整数
- **联合体语法：**union 联合体名称 { 公有成员; protected: 保护型成员; private: 私有成员; }; 与结构体一样默认公有；全部数据成员共享同一组内存单元 → 一个联合体变量的成员同时至多只有一个是有效的
- **联合体限制：**① 各个对象成员不能有自定义的构造函数、自定义的析构函数和重载的赋值运算符（对象成员的对象成员也不能有，依此类推）② 不能继承，也不支持包含多态

- **联合体使用惯例：**一般只用联合体存储一些公有的数据，不为它定义函数成员
- **无名联合体：**不声明名称（没有标记名），只是声明一个成员项的集合——这些成员项具有相同的内存地址，可由成员项的名字直接访问

```
union Mark {              // 成绩三选一，共享内存
char  grade;          // 等级制
bool  pass;           // 通过/不通过
int   percent;        // 百分制
};                        // 同一时刻至多一个成员有效

union { int i; char c; };  // 无名联合体：i、c 同地址，直接写 i / c
```

第 5 章 数据的共享与保护

5.1 标识符的作用域与可见性

- **作用域：**标识符在程序正文中有效的区域；可见性：从引用角度看的有效范围

C 基础 · 从简（C 有块作用域/文件作用域的概念，类作用域和命名空间是 C++ 新增）

- **函数原型作用域：**最小；只在原型形参括号之间 double area(double radius);
- **局部作用域（块作用域）：**声明点 → 所在块的花括号结束
- **文件作用域：**声明点 → 文件尾（全局变量）
- **类作用域：**成员函数内直接访问 / x.m / X::m（静态）/ ptr->m
- **命名空间作用域：**namespace 名 { … }，用 名::实体 或 using 访问
- 限定作用域的 enum（枚举类，C++11）
- **枚举类元素隐藏：**元素不泄漏到外层作用域，访问必须写 枚举类名::元素（Color::red），也不隐式转 int

```
namespace Math { const double pi = 3.14; }
cout << Math::pi;   // 或 using namespace Math;
```

- **可见性四条：**①声明在前引用在后 ②同一作用域不可同名 ③不同作用域同名互不影响 ④包含关系同名 → 外层被内层隐藏

```
int i = 5;                    // 文件作用域
void f() { int i = 7; … }  // 块内隐藏外层的 i
```

- 作用域 = 有效域；文件作用域的"文件"指程序文件
- 函数原型作用域是程序中最小的作用域，形参名只出现在函数原型声明中，编译后去掉（仅起提示作用），与函数体中形参名无关
- **局部作用域即块作用域：**从声明点开始，到所在块结束为止；函数形参的作用域一直延伸到整个函数体结束，而不是到形参所在行结束
- **类作用域内的成员有 3 种访问方式：**①同类成员函数中可直接访问其他成员；②对象外通过 x.m 或 X::m（静态成员）访问；③对象指针通过 ptr->m 访问
- **枚举类的两个常错写法：**`color c = red;`（正确，省略作用域名）与 `color2 c = red;`（错误，C++11 强类型枚举必须写 color2::red），正确写法 `color2 c3 = color2::red;`
- 可见性（能引用标识符的区间）遵循四条教材规则：①同一作用域内不能定义同名标识符；②若未定义，外层标识符内层可见（就近原则）；③内层定义同名标识符则外层被隐藏（隐藏≠不存在）；④使用 ::m 访问全局，m 访问局部
- **作用域与可见性的讨论适用于所有标识符：**常量名、类型名、函数名、枚举取值名，不限于变量名

```
enum class color2 { red, yellow, blue };
color2 c3 = color2::red;   // 对：强类型枚举必须带作用域名
// color2 c = red;          // 错：这里找不到 red
double x;
void f() { int x; x = 1.5; ::x = 1.5; } // 内层隐藏外层，:: 取全局
```

5.2 对象的生存期

- **静态生存期：**与程序运行期相同
- **动态生存期：**诞生于声明点，结束于所在块执行完毕
- **未初始化：**静态 → 0；动态 → 不确定
- **局部静态变量：**不随调用产生副本、保持上次的值；递归调用也共享同一变量
- static int i=5; 是初始化（仅一次），不是每次进函数赋值

```
void f() { static int count = 0; count++; }
// 每次调用在上次基础上 +1，不重新归零
```

- **静态/动态生存期判别：**生存期是否与程序运行期相同
- **注意：**静态成员 ≠ 静态变量；动态生存期 ≠ 动态内存分配
- **静态生存期：**该生存期与程序运行期相同，只要程序开始运行，这种生存期的对象就分配了存储空间；具有文件作用域的对象都具有静态生存期
- `static int i = 5;` 中的 =5 只起初始化作用，只在第一次进入时被赋一次值，不是每次进入函数都重新赋值
- **未指定初值时：**具有静态生存期的对象初值为 0；具有动态生存期的（局部自动）对象初值不确定（随机数）
- **动态生存期：**诞生于声明点，结束于该声明所在的块执行完毕
- 类的成员对象的生存期与其所属对象的生存期一致（静态成员除外）
- 例 5-2 的运行结果（含 OTHER 子函数两次调用）：MAIN i:1 a:0 b:-10 c:0 → OTHER i:33 a:4 b:0 c:15 → MAIN i:2 a:10 b:0 c:20 → OTHER i:33 a:4 b:0 c:15，体现静态局部变量只在第一次调用时初始化、自动变量每次重新分配

```
void f() {
static int i = 5;  // 只初始化一次
int a = 0;         // 每次进入函数都重新分配
a += 2;  i += 33;
}
int x;          // 静态生存期，隐含初值 0
void g() { int y; }  // y 初值不确定
```

5.3 类的静态成员

- **实例属性：**非静态数据成员，每个对象一份，值各不相同
- **类属性：**静态数据成员，整个类一份，所有对象共同维护（比喻：静态成员属于工厂，不属于每个产品）
- **静态数据成员：**类内引用性声明 + 类外定义性声明（分配内存）

```
class Point {
    static int count;           // 类内声明
public:
    static void showCount() { cout << count; }
};
int Point::count = 0;           // 类外定义并初始化
Point::showCount();             // 类名直接调用
```

- **例外：**整数/枚举类型的静态常量可类内初始化；被取地址时仍需类外定义，但不可二次初始化
- **静态成员函数：**可用类名或对象名调用（非静态只能对象名调用）
- 静态成员函数无目的对象 → 不能直接访问非静态成员，必须指明对象
- 静态数据成员有静态生存期；类内声明、类外定义初始化
- 静态成员函数一般只访问静态成员；访问非静态成员必须借助引用或指针
- **结构化程序设计中共享的途径只有两种：**参数传递和全局变量——代价是难以保证数据的逻辑一致性与安全性，安全性得不到保证，这就是引入面向对象封装共享（静态成员）的动机
- **实例属性：**类的每一个对象都有自己独立的该属性副本，即属性是以实例为单位存放的（例：每个产品有各自的产品号）；类属性：类的所有对象共同维护一个副本，任何一个对象对它的修改都会影响其他对象（例：所有产品共享同一个工厂名）
- 静态数据成员采用 static 关键字在类内作引用性声明；还必须在类外（通常是类定义后）用 `类型 类名::静态成员名 = 初值;` 做定义性声明——因为引用性声明不分配内存，类外定义时才为它分配内存并初始化
- **C++11 新标准允许：**在类内对静态常量（整型或枚举类型）直接给常量值初始化，即 constexpr 类内初始化
- 被取地址的静态常量仍需在类外给出定义性声明，但不可二次初始化
- UML 图中静态数据成员名字带下划线；静态成员函数在 UML 中标注 <<static>>
- **静态成员函数的由来：**有时需要在没有建立任何对象之前访问静态数据（如读取 Point 的对象计数），若写成

普通成员函数则编译报错（没有对象，无法 this 调用），故把不依赖于对象状态的成员函数声明为静态
- 静态成员函数可直接访问本类的静态成员，因为属于类而非对象；不能直接访问本类的非静态成员，因为不知道属于哪个对象——访问非静态成员时必须指明对象，通过参数取得对象名再通过对象访问
- 静态成员属于类、不属于任何对象，为所有对象共享，不计入对象的大小

```
class Point {
public:
Point(int x = 0, int y = 0) : x(x), y(y) { count++; }
~Point() { count--; }
static int getCount() { return count; }
constexpr static int origin = 0;   // C++11 类内初始化
private:
int x, y;
static int count;                  // 类内引用性声明
};
int Point::count = 0;                // 类外定义性声明（分配内存）
constexpr int Point::origin;         // 被取地址时仍需类外定义，不可二次初始化
Point a, b;
cout << Point::getCount();         // 无对象时直接以类名调用
```

5.4 类的友元

- **友元**：一个类主动声明"朋友"，给对方访问私有/保护成员的特许；是对封装的有限破坏，为的是共享与效率
- **友元函数**：friend 修饰的非成员函数（普通函数或其他类的成员函数）
- **友元类**：friend class A; → A 的全部成员函数都是友元
- **三条性质**：不传递 / 单向 / 不被继承
- **习惯**：不修改的参数用常引用传（复制构造参数也宜常引用）

```
class Point {
    friend float dist(Point &p1, Point &p2);  // 声明
    …
};
float dist(Point &p1, Point &p2) {  // 类外定义，直接用 p1.x
    return sqrt(p1.x*p1.x + p2.y*p2.y);
}
```

- 友元 = 授权，可访问 private
- **友元函数**：friend int A::…; 友元类：friend class A;
- **引入友元的动机**：需要类外的普通函数或另一个类的成员函数访问本类私有成员时，放在类外则无法访问私有数据；把该函数做成成员函数在语义上又不恰当（如计算两点距离的函数并不属于任何点的行为）；用组合把另一类型声明为本类的数据成员则每次调用都要构造对象，影响可读性
- 友元的关系是被访问的类主动声明「这是我的一个朋友」，授予其访问特许——友元不是该类的成员函数，而是类外的普通函数或其他类的成员函数
- 友元是对封装的「必要但有限的破坏」，关键在于把握好度：只在确实需要直接访问私有数据时才声明，不可滥用
- **友元函数声明**：在类定义体内加 friend 关键字；实现时写普通函数（或其他类成员函数）的定义，不加 friend、也不加类名限定
- **友元类语法**：`friend class 友元类名;` 可将整个类声明为友元，其所有成员函数都是本类的友元
- **友元三条性质**：①不具有传递性——A 是 B 的友元、B 是 C 的友元，不能推出 A 是 C 的友元；②不具有交换性——A 是 B 的友元，B 不一定是 A 的友元，要互友须各自声明；③不被继承——B 是 A 的友元，B 的派生类不是 A 的友元（「别人信任你，不见得信任你的孩子」）
- UML 中友元标注 <<friend>>

```
class Point {
friend float dist(Point &p1, Point &p2); // 声明（类内）
public: Point(int x, int y) : x(x), y(y) {}
private: int x, y;
};
float dist(Point &p1, Point &p2) {       // 实现（类外，不加 friend）
double dx = p1.x - p2.x;  // 可直接访问私有成员 x, y
return sqrt(dx*dx);
}
```

5.5 共享数据的保护（const）

- **常对象**：const 类型 对象名; ——必须初始化、不能更新；只能调用常成员函数
- **常成员函数四条**：①定义处也要带 const ②常对象唯一的对外接口 ③调用期间目的对象视同常对象 ④const 可区分重载（非 const 对象优先匹配非 const 版）

```
class R {
public:
    void print();        // 普通版本
    void print() const;  // 常成员函数（重载）
};
const R b(20, 52);
b.print();             // 只能调 print() const
```

- **常数据成员**：任何函数都不能赋值；只能经初始化列表初始化

```
class A { const int a; … };
A::A(int i) : a(i) {}   // 唯一的初始化方式
```

- **常引用**：const 类型 &引用名; ——可绑常对象；通过它只能把对象当常对象用
- 常对象 const A a;（必须初始化、不可改）
- 常成员函数 void print() const;（不能修改成员）
- **常引用**：不能改值的左值引用；可作形参 float dist(const A &a);
- **常对象声明语法**：`const 类型 对象名;`（const 写在类型名后也可）；常对象在初始化后不能被修改
- **语法保护机制的两条途径**：①直接对常对象禁止赋值类操作，编译器报错；②通过常成员函数，保证成员函数不会修改数据成员
- **初始化 ≠ 赋值**：常对象只能初始化不能赋值；常数据成员同样只能在初始化列表中初始化，之后无法修改
- **常成员函数声明**：`类型 函数名(参数) const;`（const 写在参数表后、函数体前）；注意 const 是类型的一部分，函数声明与实现都要带 const，位置不能错
- **常成员函数 4 条注意**：① const 是函数类型的一部分，声明与实现都要带；②常成员函数不更新对象数据成员，不能调用该类中没有 const 修饰的成员函数（因为后者可能修改数据）；③常对象只能调用它的常成员函数，不能调用非常成员函数；④ const 关键字可用于重载：同名同参数表的常/非常函数构成重载，非 const 对象优先匹配非常版本，const 对象只能匹配常版本
- **编程习惯**：无须改变对象状态的成员函数都应该声明为 const
- **常数据成员**：只能通过构造函数的初始化列表进行初始化，之后无法修改
- **静态常量成员**：C++11 允许类内初始化，编译器在用到该常量处用数值直接替换（因此不一定分配内存）；被取地址则必须在类外再给出定义性声明，但不可二次初始化
- **常引用**：非 const 引用不能绑定到常对象；常引用绑定到一个对象后，无论原对象是否常对象，都一律作为常对象使用，不能通过该引用修改
- **例 5-9**：dist 函数参数用常引用 Point &p1，说明无须改变值的参数都可用常引用提高安全性
- **编程习惯**：无须改变值的参数尽量声明为常引用（大对象可避免复制的开销，复制构造函数的实参也习惯用常引用 const X&）

```
class A {
public:
void print() const { }         // 常成员函数
void print() { }                // 与上构成重载，优先被普通对象选中
A(int v) : r(v) { }             // 常数据成员只能在初始化列表初始化
private: const int r;
};
const A a(5);
a.print();                      // 只能调用 const 版
// A b;  b = a;                // 错：常对象不能赋值
void f(const Point &p) { }   // 参数不改值 → 用常引用
```

5.6 多文件结构和编译预处理

- **三文件组织**：类定义 .h / 类实现 .cpp / 主函数 .cpp；每个源文件 = 一个编译单元
- **原则**：声明放头文件，分配空间的定义放源文件；内联函数定义要放头文件
- **外部变量**：extern 声明 → 引用性声明（可多处）；定义性声明只能一处
- **外部函数**：原型声明后可在其他编译单元调用
- **限制在本编译单元**：static 修饰（旧，已不鼓励）或匿名命名空间（推荐）

```
// 文件 1：int i = 3;              // 定义性声明（唯一）
// 文件 2：extern int i;           // 引用性声明（可多处）
namespace { int n; }              // 匿名命名空间：别的文件看不见
```

- **标准 C++ 库 6 类**：输入输出 / 容器与 ADT / 存储管理 / 算法 / 错误处理 / 运行环境支持
- **习惯**：using namespace 不宜放在头文件中
- **工程组织**：Windows 用 IDE 的“工程/项目”统一管理多文件（自动编译连接）；UNIX 环境用 make 按依赖关系自动编译
- **#include <文件名>**：搜系统目录（标准头文件）；#include "文件名"：先当前目录再系统（自定义头文件）
- **#define**：符号常量（C++ 中 const 更好）/ 空符号（只表示"已定义"状态）；#undef 删除宏
- **条件编译五种**：#if…#endif / #if…#else…#endif / #if…#elif…#endif / #ifdef / #ifndef
- **defined(标识符)**：预处理操作符（不带#）；已定义 → 非 0
- **防重复包含三件套（每个头文件都写）**：

```
#ifndef MYHEAD_H
#define MYHEAD_H
  …头文件内容…
#endif
```

- **典型 C++ 程序组织为三文件结构**：头文件（.h，类的声明与内联函数定义）、类实现文件（.cpp，类成员函数的实现）、主程序文件（.main.cpp，main 函数与使用）
- **声明与定义分离的原则**：能放头文件的只有声明；成员函数与全局变量的定义（分配内存的）必须放 .cpp 文件
- **例外**：内联函数（inline）的定义必须放在头文件里，因为内联是在编译时展开的——若误将内联函数定义放在类实现文件中，会出现连接错误（多重定义/未定义），因为每个使用它的编译单元都需看到定义
- **#include 两种写法及搜索规则**：#include <文件名> 按编译器约定的系统目录搜索；#include "文件名" 先在当前目录（源文件所在目录）搜索，找不到再按系统目录处理
- **#include 指令可以嵌套**：一个头文件中还可以有 #include（如头文件里包含所需其他头文件）；预处理程序自动处理嵌套
- **多文件结构优点**：只修改某个类的实现时，只需重新编译该类所在的编译单元，再连接即可（Windows 下由工程文件管理，UNIX/Linux 下由 make 工具管理）
- **外部变量（全局变量）**：默认外部链接属性（其他编译单元可用）；其他编译单元要用 extern 作引用性声明（不分配内存）；定义性声明（分配内存）在整个程序中只允许出现一次；若 extern 声明同时给初值则该声明升级为定义性声明（只应出现一次）
- **外部函数**：定义在类之外的函数默认具有文件作用域，在此编译单元中可直接使用；原型声明后其他编译单元也可调用（需在调用处写函数原型，通常放头文件）；加 extern 修饰与不加效果相同；将函数声明为 static 或放入匿名命名空间可限制在本编译单元内（static 方式 C++ 2003 标准已不鼓励，推荐匿名命名空间）
- **static 的三种用法共同点**：都使对象或函数具有静态生存期/静态存储——文件作用域静态变量、块内静态局部变量、类内静态成员
- **标准 C++ 库 6 大组成部分**：输入输出流、容器与迭代器、算法、数值计算、诊断、通用工具（如 string）；使用名字空间 std
- using namespace 语句不宜放在头文件中（会把名字污染传给所有包含该头文件的源文件）
- 预处理指令以 # 引导、单独占一行、不要以分号结束（#define 结尾写分号是常见错误）
- #define 可定义符号常量（现代 C++ 更推荐 const 与内联函数）、带参数的宏（多行用反斜线续行，注意括号，容易出错，已被内联函数取代）、空符号（用于条件编译）；#undef 取消已定义符号
- **条件编译五种形式**：#if 表达式 … #endif；#if 表达式 … #else … #endif；#if 表达式 … #elif 表达式 … #endif；#ifdef 符号 … #endif；#ifndef 符号 … #endif；defined 是操作符不是指令（不带 #），常用于 #if defined(X) && !defined(Y) 的组合判断
- **防止头文件重复包含的标准三件套**：#ifndef _HEADER_H、#define _HEADER_H、… #endif

```
// point.h
#ifndef _POINT_H              // 三件套防重复包含
#define _POINT_H
class Point { /* ... */ };
#endif
// ext.cpp（多文件共享全局变量）
int sharedCount = 0;          // 定义性声明（只此一次）
// main.cpp
extern int sharedCount;       // 引用性声明（不分配内存）
namespace {                   // 匿名命名空间：仅限本编译单元
void helper() { }
}
#if defined(DEBUG) && !defined(NDEBUG)
cerr << "debug\n";
#endif
```

第 6 章 数组、指针与字符串

6.1 数组

6.1.1 声明与使用

C 基础 · 从简

- **声明**：数据类型 标识符[常量表达式 1][常量表达式 2]…; ——界必须是编译期常量、正整数
- 下标从 0 开始；元素个数 = 各维之积；只能逐个元素操作
- **越界**：编译不查、运行未必报错 → 自己守住上下界
- **范围 for（C++11）**：for (const auto &e : a) cout<<e;

```
int a[10];          // 10 个元素 a[0]~a[9]
int m[2][3];        // 2 行 3 列
for (const auto &e : a) cout << e << " ";
```

6.1.2 存储与初始化

C 基础 · 从简

- **存储**：连续存放；二维按行优先；二维 = 一维数组的数组
- **初始化**：全给初值可省个数；部分给 → 其余补 0；不能间隔赋初值
- **未初始化**：静态数组 → 全 0；动态数组 → 不确定
- **常量数组**：const float fa[5]; ——必须给初值、元素不可改

```
int a[] = {1, 2, 3};            // 全给可省个数
int b[5] = {1, 2};              // 其余补 0
int m[2][3] = {{1,0,0},{0,1,0}};
```

6.1.3 数组作为函数参数

- 数组名作实参 = 传地址；形参与实参共享内存
- 被调函数改形参数组 → 实参数组跟着变
- 第一维大小一般不写，写了也被忽略

```
void rowSum(int a[][4], int n);  // 第一维可省/被忽略
int arr[3][4];  rowSum(arr, 3);   // 传的是首地址
```

6.1.4 对象数组

- **声明**：类名 数组名[常量表达式]; 访问：数组名[下标].成员名
- 初始化 = 对每个元素调用构造函数
- 未给初值的元素 → 调默认构造函数
- 数组消亡 → 每个元素分别调用析构函数

```
Location a[2] = {Location(1,2), Location(3,4)};
a[0].show();   // 访问元素成员
```

- **声明数组的三步骤**：①确定数组元素的类型；②确定数组元素的名称；③确定数组的结构（维数、每一维的大小）
- **数组类型声明一般形式**：数据类型 标识符[常量表达式 1][常量表达式 2]…; 各「常量表达式」称为数组的界，

必须是在编译时就可求出的常量表达式，其值必须为正整数；有 n 个下标的数组称为 n 维数组，元素个数是各下标表达式的乘积

- **使用数组时的两点注意：**①数组元素的下标表达式可以是任意合法的算术表达式，其结果必须为整数；②数组元素的下标值不得超过声明时确定的上下界，否则运行时出现数组越界错误——而且越界有时有提示、有时毫无提示，错误悄悄发生
- 数组元素在内存中顺序、连续存储，逻辑上相邻的元素物理地址也相邻；二维数组按行优先存放（先第 1 行再第 2 行，每行内按列下标由小到大）；C++ 中二维数组被当作一维数组的数组（int m[2][3] 即大小为 2、每个元素是大小为 3 的 int 数组），所以自然按行优先
- **多维数组的存储次序可看作计数器：**右边是低位，某一位超过上界就向左进位，本位及右边各位回到下界；最左一维变化最慢，最右边一维变化最快；下界是 0，上界是下标表达式值减 1；三维以上很少用，最常用一维数组
- **初始化规则：**①列出全部元素初值可不写元素个数（int a[]={1,1,1};）；②只初始化部分元素时元素个数必须明确指出；③初始化只能针对所有元素或从起始地址开始的前若干元素，不能间隔赋初值；④初值个数小于数组大小时，剩余元素以 0 赋值；⑤静态生存期数组未初始化时各元素为 0，动态生存期数组未初始化时初值不确定；⑥多维数组给出全部初值时第一维可省略（int a[][3]={…};），也可按第一维分组用花括号括起每一组（int a[2][3]={{1,0,0},{0,1,0}};），分组写法更易读
- 常量数组（const float fa[5]）：每个元素被当作常量，值初始化后不可改变；声明为常量的数组必须给定初值
- **数组作为函数参数：**用数组名做参数时传递的是地址，形参数组与实参数组首地址结合、后续元素按内存存储顺序对应，被调函数改变形参数组元素值会改变实参数组相应元素（共享）；数组作形参时一般不指定第一维大小，即使指定也会被忽略
- **对象数组：**初始化就是调用构造函数对每个元素对象初始化；未指定初值的元素调用默认（缺省）构造函数；设计类时应考虑对象数组的需要——各元素初值相同时提供默认构造函数，初值不同时提供带形参构造函数；数组元素对象被删除时系统自动调用析构函数完成扫尾

```
int b[10];         // 10 个元素 b[0]~b[9]
int a[5][3];       // 5 行 3 列，元素 15 个，行优先存放
int x[5]={1,2,3}; // x[3]、x[4] 自动补 0
int m[][3]={{1,0,0},{0,1,0}};  // 全初始化时第一维可省略
const float fa[5]={1.0,2.0,3.0,4.0,5.0}; // 常量数组必须给初值
void rowSum(int a[][4], int n);  // 第一维可省略/被忽略
// for(int i=1; i<=10; i++) 改写成这样：编译没错但越界！
Location a[2]={Location(1,2), Location(3,4)}; // a[1] 不写给初值则调默认构造
```

6.2 指针

6.2.1–6.2.3 基础概念

C 基础 · 从简

- **声明：**数据类型 *标识符; ——类型指"所指对象"的类型
- "*" 和 "&" 在声明语句 vs 执行语句中含义不同：声明中 * 表指针、& 表引用；执行中 * 表解引用、& 表取地址
- **指针可指向：**基本类型、数组元素、函数、对象、指针

6.2.4 指针的赋值

- **铁律：**定义后必须先赋值、再使用（未赋值的指针里是随机地址）
- 对象必须先声明、类型要一致；多个指针可指向同一变量
- 数组名 = 常量指针（不能被赋值）
- **指向常量的指针：**不能经它改所指对象，但它可改指别处

```
int i;  int *p = &i;   // 定义即初始化
int a[10];  int *ptr = a;    // 数组名 = 首地址
const int *p1 = &a;  p1 = &b;   // ✔  p1 改指
                     *p1 = 1;  // ✘ 不能改所指
```

6.2.5 指针运算

- **算术：**p+n = 后方第 n 个元素的地址；*(p+n) 等价 p[n]；只适用于连续空间
- **关系：**同类型才能比较；可与 0 比较（0 = 空指针 = NULL）
- **赋值：**只能赋地址或 0；没目标时先置空
- **指针运算前提：**结果地址必须仍落在所指数组（含尾后一格）之内，越出后再运算/比较均属未定义
- **void 指针：**可存任何类型地址，用时要显式转换

```
int a[5];  int *p = a;
*(p+2) 就是 a[2]
int *q = 0;   // 没目标先置空
```

6.2.6 用指针处理数组元素

- **数组名 = 首地址：***(a+i) 就是 a[i]
- **形参 3 等价写法：**void f(int p[]); / void f(int p[3]); / void f(int *p);
- begin(a) / end(a)（C++11，<iterator>）：安全取首尾指针，end 指向尾后位置

```
for (int *p=begin(a); p!=end(a); ++p) cout << *p;
```

6.2.7 指针数组

- **声明：**数据类型 *数组名[下标]; ——每个元素是指针，先赋值再引用
- **与二维数组的区别：**指针数组真的存了指针；二维数组的行首是算出来的

```
int line1[]={1,0,0}, line2[]={0,1,0};
int *pLine[2] = {line1, line2};
cout << pLine[0][1];   // 输出 0
```

6.2.8 指针作函数参数

- **效果：**改 *形参 = 改实参（双向传递）
- 不改内容的参数 → 声明为指向常量的指针，常对象地址也能传

```
void splitFloat(float x, int *ip, float *fp) {
    *ip = static_cast<int>(x);  *fp = x - *ip;
}
splitFloat(3.14, &n, &f);   // 传地址
```

6.2.9 指针型函数 · 返回数组指针

- 函数不能返回数组，但可以返回数组的指针
- **声明：**类型 (*函数名(参数表))[维度]
- **简化：**using arr=int[10]; arr* foo(int i);
- **尾置返回（C++11）：**auto foo(int i) -> int(*)[10];

6.2.10 指向函数的指针

- 函数名 = 函数代码首地址
- **声明：**数据类型 (*函数指针名)(形参表)
- **赋值：**函数指针名=函数名;（类型、形参表必须一致）

```
int (*fp)(double);       // 声明
fp = myFunc;             // 赋值
int r = fp(3.14);        // 调用
```

6.2.11 对象指针

- **声明：**类名 *对象指针名; 访问：ptr->成员（等价 (*ptr).成员）
- 使用前必须初始化；对象的内存只存数据成员
- **前向引用声明后：**不能定义该类对象成员，但可定义该类指针/引用成员

```
Point a(4,5);  Point *pp = &a;
cout << pp->getX();   // 等价 (*pp).getX()
```

this 指针

- 隐含于每个非静态成员函数（含构造/析构），指向目的对象
- **本质：**成员函数的隐含参数，调用时自动传入目的对象地址
- 常成员函数中 this 是指向常量的指针
- *this = 正在调用该函数的对象；局部变量与成员同名时用 this->成员 区分

```
int Point::getX() const { return this->x; }  // 实际效果
```

指向类成员的指针

- **声明：**类型 类名::*指针名;（数据） / 类型 (类名::*指针名)(参数表);（函数）
- **赋值：**指针名=&类名::成员名; ——类外不能对私有成员取地址
- **使用：**对象名 .*指针名 或 对象指针名 ->*指针名
- **静态成员：**用普通指针即可

```
int Point::*pm = &Point::x;   // 指向数据成员
cout << a.*pm;                 // 结合对象使用
int *ps = &Point::count;      // 静态成员用普通指针
```

- **访问内存数据的两种方式：**通过变量名（直接访问）和通过地址（间接访问）；需要用地址访问的场合：不同函数间传送大量数据时传地址可减小系统开销；动态分配的内存单元没有名称，只能通过地址访问
- **声明指针要指出所指对象类型的原因：**声明变量不仅声明所需内存空间，还限定了变量可进行的运算及其规则，指针的运算规则与所指对象类型密切相关
- 指针可以指向基本类型、数组（元素）、函数、对象，也可以指向指针
- * 和 & 在不同语境的含义（三种情况）：①* 出现在声明语句中、在被声明的变量名之前 → 声明的是指针；②* 出现在执行语句中或声明语句的初值表达式中 → 访问指针所指对象的内容（解析）；③& 出现在声明语句中位于被声明的变量左边 → 声明的是引用；④& 给变量赋初值时在等号右边或执行语句中作一元运算符 → 取对象的地址
- **指针赋值：**定义了一个指针只是得到一个用于存储地址的变量，其中地址值是不确定的——指针所指内存单元可能存放着重要数据或程序代码，盲目访问可能破坏数据或造成系统故障，因此定义指针之后必须先赋值，然后才可以引用
- **赋初值两方法：**①定义指针的同时初始化赋值：存储类型 数据类型 *指针名=初始地址; ②定义之后单独用赋值语句：指针名=地址;
- 用对象地址作初值或赋给指针时，该对象必须在赋值之前已声明过，且对象类型应与指针类型一致；可用一个已赋值的指针初始化另一个指针（多个指针指向同一个变量）
- **数组名是常量指针：**数组名表示数组首地址，是一个不能被赋值的指针（指针常量），可用数组名初始化指针 int *ptr=a;
- **指向常量的指针（注意类型 1）：**const int *p1=&a; 不能通过指针改变所指对象的值，但指针本身可以改变指向（p1=&b 正确；*p1=1 编译出错）；可确保指针所指的常量不被意外更改
- **void 类型指针：**可存储任何类型对象的地址，经显式类型转换可访问任何类型数据；不能声明 void 类型的变量；一般只在所指数据类型不确定时使用

```
int i=10;
int *ptr=&i;       // 声明指针并初始化（* 此处声明指针）
cout<<*ptr;        // 输出 10（* 此处解析，间接访问）
int &rf=i;         // & 此处声明引用
int *pa=&i, *pb=&i;  // & 此处取地址
ptr=&i;            // 指针必须先赋值再引用
const int *p1=&a; // 指向常量的指针：p1 可改指向，*p1 不可改值
void *vp; vp=&i;  // void 指针可存任何地址
```

- **指针算术运算：**p1+n1 表示 p1 所指位置后方第 n1 个数的地址，p1-n1 前方；*(p1+n1) 等价 p1[n1]；指针++/--指向下一个/前一个数据地址
- 算术运算不慎会导致指向无法预期的地址，要慎用；须确保运算结果所指地址是程序分配使用的地址；算术运算一般与数组（连续内存）相联系，对独立变量地址做算术运算可能破坏数据/代码
- **指针关系运算：**只能在指向相同类型数据的指针之间进行；两同类型指针相等表示指向同一地址；不同类型指针之间或与非 0 整数比较没有意义；指针可与整数 0 比较，0 表示空指针
- **指针赋值：**赋给指针的值必须是地址常量（如数组名）或地址变量，不能是非 0 整数；可赋值 0 设为空指针；空指针也可用 NULL 表示（NULL 是多数头文件中定义为 0 的宏）；声明指针时若无确定地址可赋值，先设为空（用 0 作初值的习惯），避免指向不确定地址造成不可预见错误
- **用指针处理数组元素：**数组名 array 就是数组首地址（&array[0]）；下标为 i 的元素就是 *(数组名+i)；*array 就是 array[0]
- 数组作函数形参等价于指向元素类型的指针，三种形参写法等价：void f(int p[]); void f(int p[3]); void f(int *p);
- C++11 标准库函数 begin 和 end（定义在 iterator 头文件）：begin(a) 返回指向数组首元素的指针，end(a) 返回指向尾元素下一位置的指针；用 pbeg!=pend 控制循环可安全解引用，比手动计算尾指针更不易出错

```
int a[10];
*(a+3) == a[3];              // 等价
for(int *p=a; p!=end(a); ++p) cout<<*p; // begin/end 遍历
int *pbeg=begin(arr), *pend=end(arr);
while(pbeg!=pend && *pbeg>=0) ++pbeg;   // 找第一个负数
int *p=0;                    // 无确定地址时先设空指针
```

- **指针数组：**每个元素都是指针变量的数组，元素必须是同一类型的指针；声明：数据类型 *数组名[下标表达式]; 每个元素都是指针，必须先赋值后引用，声明后为指针元素赋初值必不可少
- pLine[i][j] 与 *(pLine[i]+j) 等价：先读出指针数组第 i 个指针，再取其指向地址后方的第 j 个数
- **指针数组 vs 二维数组（本质差异）：**pLine[i] 的值需读取指针数组的第 i 个元素才能得到（内存中确有一个指针数组存这些指针，各行可独立存储）；array2[i] 是通过二维数组首地址计算得到的，内存中并没有一个指针数组来存储 array2[i] 的值（整块连续内存、行优先）；二者访问形式相同，可把二维数组当作指针数组来访问
- 二维数组元素可用 *(*(array2+i)+j) 访问（即 array2[i][j]）；array2[0] 在表达式中表示指向该行首地址的整型指针，*(array2[0]+1) 即 array2[0][1]

```
int line1[]={1,0,0}, line2[]={0,1,0}, line3[]={0,0,1};
int *pLine[3]={line1, line2, line3};  // 指针数组：每行独立数组
cout<<pLine[i][j];         // 等价 *(pLine[i]+j)
int array2[3][3]={{11,12,13},{21,22,23},{31,32,33}};
cout<<*(*(array2+i)+j);    // 即 array2[i][j]，整块连续内存
```

- **指针作为函数参数：**实参把地址值传给形参，使实参和形参指针指向同一内存地址，子函数通过形参指针对数据的改变同样影响实参所指数据（双向传递）
- **C 语言中指针形参的三个作用：**①使实参与形参指向共同内存空间，达到双向传递——这个作用在 C++ 中已由引用实现；②减少函数调用时数据传递的开销——C++ 中有时可用引用，有时仍需指针；③通过指向函数的指针传递函数代码首地址
- **习惯：**如果函数体中不需要通过指针改变指针所指对象的内容，应在参数表中将其声明为指向常量的指针（const T*），这样常对象取地址后也可作为该函数的实参；当指针和引用都能达到同样目的时，使用引用会使程序的可读性更好
- **指针型函数：**函数不能返回数组，但可以返回数组的指针；不用类型别名时形式为：类型说明符 (*函数名(参数表))[数组维度]，例 int (*foo(int i))[10]；也可用 C++11 尾置返回类型：auto foo(int i) -> int(*)[10]
- **指向函数的指针：**函数名表示函数代码在内存中的起始地址；函数指针声明：数据类型 (*函数指针名)(形参表)；赋值：函数指针名=函数名（返回值类型和形参表必须相同）；之后可用函数指针名直接调用该函数；可用 typedef 简化声明 typedef int (*DoubleIntFunction)(double);

```
void splitFloat(float x, int *intPart, float *fracPart) {
*intPart=static_cast<int>(x);  *fracPart=x-*intPart;
}
float average(const double *arr, int n); // 不改内容→指向常量的指针
using arr=int[10];  arr* foo(int i);      // 返回数组指针
auto foo2(int i) -> int(*)[10];           // 尾置返回类型
int (*fp)(double)=&func;  fp(3.5);        // 函数指针调用
```

- **对象指针：**对象所占内存空间只用于存放数据成员，函数成员不在每个对象中存储副本；声明：类名 *对象指针名; 使用前必须初始化，指向已声明的对象；访问成员：对象指针名->成员名，与 (*对象指针名).成员名 等价
- **前向引用声明的应用：**class Fred; class Barney { Fred *x; }; class Fred { Barney y; }; ——Barney 中只能声明 Fred 的指针（此时 Fred 定义尚不完善，不能声明对象），待 Fred 完整定义后方可声明其对象作数据成员
- **this 指针（细则）：**this 是隐含于每一个类的非静态成员函数（含构造、析构）中的特殊指针，指向正在被成员函数操作的对象；this 实际上是成员函数的一个隐含形参，调用时目的对象的地址自动传给该形参；对常成员函数，这个隐含参数是常指针类型；this 是一个指针常量，常成员函数的 this 同时是指向常量的指针；可用 *this 标识正在调用该函数的对象；当局部作用域声明了与类成员同名的标识符时，直接引用代表局部标识符，可用 this->成员 访问类成员
- **指向类的非静态成员的指针：**声明形式：类型说明符 类名::*指针名;（数据成员） / 类型说明符 (类名::*指针名)(参数表);（函数成员）；赋值：指针名=&类名::成

员名；对类成员取地址同样受访问权限约束（类外不能对私有成员取地址）；成员指针中存放的是成员在类中的相对位置（偏移量），必须与具体对象的起始地址结合才能访问：对象名.*类成员指针名 或 对象指针名->*类成员指针名

- **成员函数指针的调用必须通过对象：**(对象名.*函数成员指针)(参数表) 或 (对象指针名->*函数成员指针)(参数表)；因为非静态成员函数需要确定 this 指针；常成员函数与普通成员函数类型不同，指向常成员函数的指针声明时必须明确写出 const
- **指向类的静态成员的指针：**静态成员的访问不依赖于对象，可用普通指针指向和访问：int *ptr=&Point::count; void (*funcPtr)()=Point::showCount;

```
Point a(4,5);  Point *p1=&a;
p1->getX();  (*p1).getX();  a.getX();
return this->x;   // 系统区分数据属于哪个对象就靠 this
int Point::*px=&Point::x;   // 指向数据成员的指针（偏移量）
Point a(4,5);  cout<<a.*px;
int (Point::*f)() const=&Point::getX;
cout<<(a.*f)();             // 必须通过对象调用
int *ptr=&Point::count;     // 静态成员：普通指针即可
```

6.3 动态内存分配

- new 类型(初值); → 返回该类型指针；失败抛异常
- delete 指针; ——对象自动调析构；同一内存只能 delete 一次
- **new T vs new T()：**内置类型加 () 值初始化（置 0），不加 () 值不确定；类类型两种写法都调构造函数
- **内存泄露：**new 后不用 delete 释放 → 内存只增不减

```
int *p = new int(2);    // 初值 2
int *z = new int();     // 置 0
delete p;  delete z;
```

- **动态数组：**new 类型名[长度]; ——长度可以是运行期表达式
- **删除数组：**delete []指针; ——漏写 [] 是错误
- **多维数组：**第 1 维任意正整数表达式，其余维必须常量表达式

```
int n;  cin >> n;
int *a = new int[n]();      // () 全部置 0
…
delete []a;
float (*cp)[25][10] = new float[10][25][10];  // 多维
```

- **对象数组：**new Point[2] 逐个调构造函数；delete []ptr 逐个调析构
- assert(条件)（<cassert>）：假则中止报错；只在调试模式生效

```
Point *ptr = new Point[2];
ptr[0].move(5,10);
delete []ptr;
```

- **动态内存分配的动机：**程序运行前无法确切知道需要多少元素，数组声明太大浪费、太小不够；运行中申请和释放的存储单元称为堆对象，申请和释放过程称为建立和删除
- **new 语法：**new 数据类型(初始化参数列表); 申请成功返回指向新分配内存的指针，失败抛出异常
- **基本类型细节：**new int(2) 分配并赋初值 2；不希望设初值可省去括号 new int；保留括号但不写数值表示用 0 初始化 new int()
- 建立类对象时按参数类型和个数调用构造函数；细节：类有用户定义的默认构造函数时 new T 与 new T() 效果相同；无用户定义默认构造函数时 new T 调用系统生成的隐含默认构造函数，new T() 还会为基本数据类型和指针类型成员用 0 赋初值，且这一过程是递归的（成员对象的基本数据类型成员同样被 0 初始化）
- **delete 语法：**delete 指针名; 删除对象时析构函数被调用；对同一内存空间多次 delete 会导致运行错误，只能删除一次；用 new 分配的内存必须用 delete 释放，否则动态分配的内存无法回收、程序占据内存越来越大，称为内存泄露
- **new 创建一维数组：**new 类型名[数组长度]; 长度可以是任何能得到正整数值的表达式；方括号后可加 () 但括号内不能带任何参数：不加 () 与 new T 相同，加 () 则与 new T() 相同（值初始化为 0），如 int *p=new int[10](); 将全部元素初始化为 0
- 删除数组必须 delete []指针名; 少了 [] 是常见错误
- **new 创建多维数组：**第一维长度可以是任何结果为正整数的表达式，其他各维必须是结果为正整数的常量表达式；new 返回的不是 T* 而是指向 T 类型数组的指针（元素个数为除最左维外各维的乘积）：float (*cp)[25][10]; cp=new float[10][25][10];
- **动态数组封装思路：**把 new/delete 过程封装成类，通过成员函数访问元素并每次检查下标越界；检查可用 assert（cassert 头文件中的宏），条件不为 true 时程序中止并报告错误；assert 只在调试(debug)模式生效，发行(release)模式下不执行任何操作；assert 只检查程序本身的逻辑错误，用户不当输入造成的错误应用其他方式处理

```
int *point=new int(2);   // 基本类型：赋初值 2
int *p2=new int;         // 无初值
int *p3=new int();       // 用 0 初始化
Point *ptr1=new Point;       // 调默认构造
Point *ptr2=new Point(1,2); // 调带参构造
int *pa=new int[10]();       // 动态数组，全 0
delete ptr1;                 // 删对象：自动调析构
delete []pa;                 // 删数组：必须带 []
float (*cp)[9][8]=new float[8][9][8]; // 多维：其他维须常量
delete []cp;                // 用 new[] 建立就必须 delete[] 释放
assert(index>=0 && index<size);  // 越界检查，debug 模式生效
```

6.4 用 vector 创建数组对象

- vector<元素类型> 数组对象名(数组长度); ——类模板；长度可含变量
- **所有元素都被初始化：**基本类型 → 0；类类型 → 默认构造函数
- **访问同数组：**arr[i]；成员函数 arr.size() 返回大小
- 名字是对象不是首地址

```
vector<int> arr(10);       // 10 个 0
vector<int> v(10, 1);      // 10 个 1
for (unsigned i=0; i<arr.size(); i++) cin >> arr[i];
```

- vector 是标准库提供的被封装动态数组，不是一个类而是一个类模板（模板第 9 章介绍，本章只需记住使用形式）
- **定义形式：**vector<元素类型> 数组对象名(数组长度); 长度是表达式，可以包含变量
- **与普通数组的关键区别：**vector 定义的所有元素都会被初始化——基本类型元素以 0 初始化，类类型元素调用默认构造函数；因此元素类必须具有默认构造函数
- **也可为所有元素指定相同初值：**vector<元素类型> 数组对象名(数组长度, 元素初值);
- **元素访问方式与普通数组相同：**数组对象名[下标表达式]；重要成员函数 size() 返回数组大小
- vector 数组对象的名字表示的是一个数组对象而非数组首地址——数组对象不是数组，而是封装了数组的对象；vector 的大小可以扩展（第 10 章详细介绍）

```
#include <vector>
vector<int> arr(x);          // 长度可为变量，元素全初始化为 0
vector<double> d(n, 1.5);    // 全部元素初值 1.5
for(unsigned i=0; i<arr.size(); i++) sum+=arr[i];
double average(const vector<double>&arr);  // 传参用常引用
```

6.5 深层复制与浅层复制

- **浅层复制（默认）：**逐成员复制 → 只复制指针值，两个对象共享内存
- **弊病①：**改一个对象影响另一个
- **弊病②：**两对象析构 → 同一内存 delete 两次 → 运行错误
- **深层复制：**自写复制构造函数，new 新内存 + 复制所指内容

```
ArrayOfPoints::ArrayOfPoints(const ArrayOfPoints& v) {
    size = v.size;
    points = new Point[size];   // 重新分配
    for (int i=0; i<size; i++) points[i] = v.points[i];
}
```

- 隐含（默认）复制构造函数完成的只是浅层复制：将两个对象的对应数据项直接复制（逐成员复制）；当成员含指针时，只复制指针值，两个指针指向同一块内存，表面上完成了复制，并没有形成真正的副本

- **浅层复制的两大弊病：**①通过一个对象修改所指数据会影响另一个对象（例 6-21 中移动 pointsArray1 的点，pointsArray2 的点也被移动）；②程序结束前析构函数自动释放动态内存，由于两个对象共用同一块内存，该空间被两次释放，导致运行错误
- **深层复制的解法：**自己编写复制构造函数，为指针成员新分配内存并把原对象的数据逐个复制过去（例 6-22：ArrayOfPoints::ArrayOfPoints(const ArrayOfPoints& v){ size=v.size; points=new Point[size]; for(int i=0;i<size;i++) points[i]=v.points[i]; }）
- **深层复制后：**移动一个对象的点不再影响另一个对象；程序结束前分别释放各自的内存空间，不再引起错误

```
class ArrayOfPoints {
public:
ArrayOfPoints(const ArrayOfPoints& v) {      // 深层复制
size=v.size; points=new Point[size];
for(int i=0; i<size; i++) points[i]=v.points[i];
}
private: Point *points; int size;
};
// 若用默认复制构造：两个对象的 points 指向同一块内存，
// 析构时同一内存被 delete 两次 → 运行错误
```

6.6 字符串

6.6.1 字符数组存字符串

C 基础 · 从简

- 字符串常量末尾隐含 '\0'；表示 char 数组首地址
- 数组大小 ≥ 字符数 + 1
- **cstring 函数：**strcpy / strcmp / strcat——麻烦，引出 string 类

```
char str[] = "program";   // 末尾隐含 '\0'
strcpy(a, b);  strcmp(a, b);  strcat(a, b);
```

6.6.2 string 类（重点）

- 头文件 <string>；字符串常量可隐式转换
- **操作符：**+ 连接 / = 赋值 / += 连接赋值 / 比较 / [] 取下标
- **比较规则：**逐字符比 ASCII；前缀相同则短的小
- **常用成员函数：**append 追加、assign 赋值、compare 比较、insert 插入、substr 子串、find 查找、length/size 长度、c_str() 转字符指针
- **细节：**string 是类模板 basic_string 的特殊化实例

```
string s1 = "DEF", s2;
cin >> s2;                  // 遇空格截断
s2 += s1;                   // 连接
getline(cin, s2);           // 读整行
getline(cin, s2, ',');      // 逗号作分隔符
s2.substr(0, 3);  s2.find("EF");  s2.length();
```

- **字符串常量在内存中的存放形式：**按串中字符排列次序顺序存放，每个字符占 1 字节，末尾添加 '\0' 作为结尾标记；一个字符串常量实际上表示一个隐含创建的 char 数组的首地址；应把字符串常量赋给指向常量的指针：const char *s="This is a string.";
- **字符串变量：**创建 char 数组，每个元素存放一个字符，末尾放 '\0'；存放字符串的数组元素个数应不小于字符串长度（字符个数）加 1；三种等价初始化：char str[8]={'p','r','o','g','r','a','m','\0'}; char str[8]="program"; char str[]="program";
- 字符数组的其他操作较麻烦，需借助 cstring 头文件中的字符串处理函数：复制 strcpy、按词典顺序比较 strcmp、连接 strcat；长度不确定时还需 new 动态创建、delete 释放
- string 类（需包含头文件 string）：严格说不是独立类，而是类模板 basic_string 的特殊化实例，可当作类使用；因 string 有接收 const char* 的构造函数，字符串常量和字符数组可隐式转换为 string 对象：string str="Hello world!";
- **string 类构造函数原型（6 个）：**string();（长度 0）；string(const string& rhs);（复制）；string(const char* s);（用字符串常量初始化）；string(const string& rhs, unsigned pos, unsigned n);（rhs 从 pos 起取 n 个字符）；string(const char* s, unsigned n);（s 的前 n 个字符）；string(unsigned n, char c);（字符 c 重复 n 次）
- **string 类操作符（表 6-1）：**+（连接成新串）、=（更新）、+=（等价 s=s+t）、==、!=、<、<=、>、>=（比较）、[]（访问下标 i 字符）；能这样用是因为 string 对操作符进行了重载（第 8 章）
- **两串大小比较的字典顺序规则（3 条）：**①长度相同且所有字符完全相同则相等；②不完全相同则比较第一对不相同字符的 ASCII 码，较小字符所在的串为较小串；③s1 长度 n1 小于 s2 长度 n2 且前 n1 个字符完全相同，则 s1<s2
- **string 常用成员函数：**append(const char* s) 将 s 添加到本串尾；assign(const char* s) 赋值；compare(const string&) 比较（本串<str 返回负数，>返回正数，相等返回 0）；insert(unsigned p0, const char* s) 将 s 插入本串位置 p0 之前；substr(unsigned pos, unsigned n) 取子串；find(const string&) 查找并返回第一次出现的位置；length() 返回串长；swap(string&) 交换两串
- **cin>> 输入字符串的局限：**>> 以空格作为分隔符，输入 "123 ABC" 只会读入 "123"；希望读到行末用 getline(cin, s2);（以换行符为分隔符）；getline(cin, s2, ','); 可把逗号等其他字符作为第 3 个参数指定为分隔符

```
const char *s="This is a string.";  // 字符串常量=char 数组首地址
char     str[8]="program";                  //     等     价 {'p','r','o','g','r','a','m','\0'}
char buf[20]; strcpy(buf, "abc"); strcat(buf, "def"); // cstring
string s1="DEF", s2;
s2=s1+"ABC";  s1<=s2;  s2.length();  s2.substr(1,2);
s2.find("BC");  s2.insert(1, "XX");
getline(cin, s2);            // 读整行（到换行符）
getline(cin, s2, ',');       // 以逗号为分隔符
```

第 7 章 类的继承

7.1 基类与派生类

- **继承：**特殊类（派生类）拥有其一般类（基类）的全部属性与服务，并可添加自己的新成员
- 单继承 / 多继承：一组相关类称类族
- **定义：**class 派生类名 : 继承方式 基类名 { … }; 继承方式：public / private / protected（默认 private）
- **派生类生成过程①：**吸收基类成员（全部吸收，构造/析构除外）
- **派生类生成过程②：**改造基类成员（改变访问属性、覆盖同名成员）
- **派生类生成过程③：**添加新成员

```
class Point { … };                  // 基类
class Rectangle : public Point {    // 公有继承
    double length, width;
};
```

- **继承的动机：**类的继承和派生层次结构是人们对自然界事物分类、分析、认识在程序设计中的体现——分类树最高层抽象程度最高（最普遍一般），下层具有上层特性同时加入新特征，最下层最具体；由上到下是具体化/特殊化，由下到上是抽象化
- **继承与派生的定义：**所谓继承就是从先辈处得到属性和行为特征，类的继承是新类从已有类那里得到已有特性，从已有类产生新类的过程就是类的派生；原有类称基类（父类），新类称派生类（子类）
- **派生机制的两大好处：**代码重用性（利用别人做过的已有分析和解决方案，站在巨人肩膀上）和可扩充性（软件有新发展或新认识时能高效改造扩充）
- **派生类定义语法：**class 派生类名: 继承方式 基类名 1, 继承方式 基类名 2, …, 继承方式 基类名 n { 派生类成员声明; }; 每个「继承方式」只限定紧随其后的基类
- 一个派生类可同时有多个基类（多继承），也可只有一个直接基类（单继承）；单继承可看作多继承的最简单特例，多继承可看作多个单继承的组合，学习从单继承开始
- **继承方式关键字：**public（公有继承）、protected（保护继承）、private（私有继承）；不显式给出继承方式关键字，系统默认认为是私有继承
- **类族：**派生类可继续作为基类派生新类；直接参与派生出某类的基类称直接基类，基类的基类及更高层称间接基类（「交通工具」→「汽车」→「卡车」：汽车是卡车的直接基类，交通工具是间接基类）
- **派生类生成过程三步骤：**①吸收基类成员——第一步将基类成员全盘接收，派生类包含其全部基类中除构造和

析构函数之外的所有成员，此步自动完成、程序员无法干预；②改造基类成员——包括访问控制问题（靠继承方式控制）和对基类成员的覆盖或隐藏：派生类声明一个和基类成员同名的新成员，新成员就隐藏了外层同名的基类成员（同名隐藏），直接使用成员名只能访问派生类的同名成员；③添加新成员——继承与派生机制的核心，保证派生类在功能上有所发展的关键

- 基类的构造函数和析构函数不能被继承，需要特别的初始化和扫尾清理工作时就要在派生类中加入新的构造函数和析构函数

```
class Derived: public Base1, private Base2 {  // 多继承：每个继承方式只限随后的基类
public:
Derived();  ~Derived();
};
class Rectangle: public Point { /*…*/ };    // 单继承
class Truck: public Car { /*…*/ };          // 类族：直接/间接基类
```

7.2 访问控制

- **公有继承 public：**基类的 public 仍是 public、protected 仍是 protected（接口不变，最常用）
- **私有继承 private：**基类 public/protected 都变为派生类的 private（"用一下而已"，不传给下一代）
- **保护继承 protected：**基类 public/protected 都变为 protected
- **私有继承 vs 保护继承的差别：**私有继承的基类成员不能再被派生类的子类访问，保护继承可以
- **派生类中基类成员的 4 种身份：**① 不可访问成员（基类 private，任何继承方式都到不了）② 私有成员（private 继承的 public/protected）③ 保护成员（protected 继承的、或 public 继承的 protected）④ 公有成员（public 继承的 public）
- 私有继承后基类 public 接口对外不可见 → 需在派生类中重新声明接口（using Base::f; 或写包装函数）才能对外使用

```
class B { public: int pub; protected: int prot; private: int priv; };
class D : public B {
    // pub→公有   prot→保护   priv→不可访问
};
```

- **基类成员的访问控制要解决两个问题：**①派生类中新增加成员访问从基类继承成员的问题；②派生类外部（非类族内成员）通过派生类的对象访问从基类继承的成员
- **公有继承：**基类的公有成员和保护成员的访问属性在派生类中不变（仍作为派生类的公有和保护成员，派生类其他成员可直接访问），基类的私有成员不可直接访问；类族之外只能通过派生类的对象访问从基类继承的公有成员；基类原有外部接口变成派生类外部接口的一部分
- **私有继承：**基类的公有成员和保护成员都以私有成员身份出现在派生类中，派生类其他成员可直接访问，但类族外部通过派生类对象无法直接访问；基类私有成员不可直接访问；经过私有继承后，所有基类成员都无法在以后的派生类中直接发挥作用，相当于终止了基类功能的继续派生，因此私有继承使用比较少
- **私有继承的补救：**为保证基类的一部分外部接口特征在派生类中仍存在，必须在派生类中重新声明同名成员，利用派生类对基类成员的访问能力把基类原有成员函数功能照搬过来（例 7-2 中重新声明 move/getX/getY，内部调用 Point::move 等）
- **保护继承：**基类的公有成员和保护成员都以保护成员身份出现在派生类中，基类私有成员不可直接访问；派生类其他成员可直接访问，类外通过对象无法访问
- **私有继承 vs 保护继承的关键区别：**在直接派生类中二者所有成员访问属性完全相同；区别在于继续派生时——私有继承后 Square（Rectangle 的派生类）成员和对象都不能访问间接从 Point 继承来的成员；保护继承后 Point 的公有和保护成员在 Rectangle 中都是保护成员，Square 再继承后有可能访问这些间接继承来的成员
- **保护成员的特征：**对建立该类对象的模块（普通函数、平行的其他类）而言，访问特性与私有成员一样不可访问；对该类的派生类而言，成员有可能访问从基类继承来的保护成员，但绝不能被其他外部使用者访问——在共享与隐藏之间找到平衡点
- **注意：**如果 B 是 A 的派生类，B 的成员函数只能通过 B 的对象访问 A 中定义的 protected 成员，而不能通过 A 的对象访问 A 的 protected 成员

```
class A { protected: int x; };
A a;  a.x=5;          // 错：类外访问保护成员，同私有
class B: public A {
void f() { x=5; }   // 对：派生类成员可访问继承来的保护成员
};
class Rectangle: private Point {
public:
float getX() const { return Point::getX(); } // 私有继承：需照搬接口
};
```

7.3 类型兼容规则

- **公有派生类对象可以：**赋给基类对象 / 初始化基类引用 / 赋给基类指针；反之不行
- **意义：**基类的指针/引用可统一操作派生类对象（多态的基础，第 8 章）

```
Derived d;
Base &r = d;     // 派生类对象初始化基类引用
Base *p = &d;    // 派生类对象地址赋给基类指针
```

- **类型兼容规则：**在需要基类对象的任何地方，都可以使用公有派生类的对象来替代——因为通过公有继承，派生类得到了基类中除构造、析构函数之外的所有成员，具备了基类的所有功能，凡是基类能解决的问题，公有派生类都可以解决
- **替代的三种情况：**①派生类对象可以隐含转换为基类对象（用派生类对象中从基类继承来的成员逐个赋值给基类对象的成员：b1=d1;）；②派生类的对象可以初始化基类的引用（B &rb=d1;）；③派生类的指针可以隐含转换为基类的指针（pb1=&d1;）
- 替代之后，派生类对象就可以作为基类对象使用，但只能使用从基类继承的成员——通过基类类型的指针只能访问到从基类继承来的成员，不能访问派生类自己新增的（或同名的）成员
- **类型兼容规则的意义：**对基类及其公有派生类的对象可用相同的函数统一处理（函数形参为基类对象/引用/指针时，实参可以是派生类对象），不必为每一个类设计单独的模块；类型兼容规则是多态性的重要基础之一（第 8 章）

```
class B { /*…*/ };  class D: public B { /*…*/ };
B b1, *pb1;  D d1;
b1=d1;       // ①派生类对象隐含转换为基类对象
B &rb=d1;    // ②初始化基类引用
pb1=&d1;    // ③派生类指针隐含转换为基类指针
void fun(Base1 *ptr) { ptr->display(); }  // 形参基类指针，实参可传任何公有派生类
```

7.4 派生类的构造函数和析构函数

- **派生类构造顺序：**基类构造（在初始化列表中调基类构造函数）→ 内嵌对象构造 → 自身成员初始化；与初始化列表书写顺序无关
- 若基类有默认构造函数，可省略
- 多继承时多个基类构造按声明顺序；内嵌对象按声明顺序
- using Base::Base; 可继承基类构造函数（C++11，只继承不含默认/复制/移动构造）
- **复制构造：**派生类的复制构造要先调基类复制构造
- **析构：**与构造相反（派生类 → 成员 → 基类），自动调用基类析构，不需显式写
- = delete 可删除某构造函数（如禁止某种构造方式）

```
class Rectangle : public Point {
    Line edge;   // 内嵌对象
public:
    Rectangle(int x, int y, int l)
        : Point(x, y), edge(l) {}  // 先基类，再成员
};
```

- **派生类构造函数的一般语法形式：**派生类名::派生类名(参数表):基类名 1(基类 1 初始化参数表), …, 基类名 n(基类 n 初始化参数表), 成员对象名 1(成员对象 1 初始化参数表), …, 成员对象名 m(成员对象 m 初始化参数表) { 派生类构造函数其他初始化操作; }
- **参数传递的必要性：**对基类的很多成员不能直接访问，要完成基类成员初始化必须通过调用基类构造函数；若

基类只声明了带参构造函数，派生类就必须声明构造函数以提供传参途径；如果不需要调用基类带参构造函数也不需要调用成员对象带参构造函数，派生类也可不声明构造函数（全部用默认构造函数），新增成员的初始化可用其他公有函数完成

- **派生类构造函数执行的一般顺序（3 步）：**①调用基类构造函数——调用顺序按它们被继承时声明的顺序（从左向右），不是按初始化列表书写的顺序；②对新增成员对象初始化——顺序按它们在类中声明的顺序；③执行派生类构造函数体
- **细节：**构造函数初始化列表中基类名、对象名之间的顺序无关紧要，各自出现顺序可以任意；无论顺序怎样安排，基类构造函数调用和成员对象初始化顺序都是确定的；使用默认构造函数的基类可不列出类名，使用默认构造的成员对象也不必写出
- **派生类复制构造函数：**若程序员没有编写，编译系统会生成隐含复制构造函数，自动调用基类复制构造函数，再对新增成员对象一一执行复制；若自行编写，一般需为基类传递参数：Derived::Derived(const Derived &v) : Base(v) {…}——这里用派生类对象 v 传给基类引用形参，正是类型兼容规则的应用
- C++11 using 声明继承基类构造函数：提供注明直接基类名的 using 声明语句 class Derived: public Base { public: using Base::Base; double d; }; 编译器为基类每个构造函数在派生类中生成一个形参列表完全相同的构造函数（Derived(params): Base(args){}），派生类只继承其直接基类的构造函数
- using 声明不改变构造函数访问级别（基类私有构造函数在派生类中仍是私有）；基类构造函数含默认实参时派生类将获得多个继承的构造函数（分别省略掉一个含默认实参的形参）；两个例外：①派生类自己定义的构造函数与基类某构造函数参数列表相同时，该构造函数不被继承（派生类版本替换继承版本）；②默认、复制和移动构造函数不会被继承
- **析构函数：**基类析构函数不能继承；派生类析构函数只需负责把新增非对象成员的清理工作做好，系统会自动调用基类及成员对象的析构函数；析构函数执行次序与构造函数严格相反：先执行派生类析构函数体 → 清理新增类类型成员对象 → 清理从基类继承的成员
- 自动生成的默认析构函数函数体是空的，但并非不做任何事情——它会隐含调用成员对象所在类的析构函数和基类析构函数
- **删除（delete）构造函数与继承：**基类中删除掉的构造函数在派生类中也对应删除或不可访问状态；如果基类默认构造、复制构造、移动构造函数是删除或不可访问的，则派生类中相应成员函数是被删除的（编译器无法执行派生类对象中基类部分的构造或赋值）；例：基类 Base(Base&)=delete 删除复制构造后，Derived d2(d1); 错误，d1 默认构造正确

```
class Derived: public Base2, public Base1 {
public:
Derived(int a,int b,int c,int d): Base1(a), member2(d), member1(c), Base2(b) { }
// 基类名/对象名顺序随意；实际执行：按继承声明顺序先 Base2 再 Base1，
// 再按成员声明顺序 member1、member2，最后执行函数体
private: Base1 member1; Base2 member2;
};
Derived::Derived(const Derived &v) : Base(v) { /*…*/ } // 复制构造传基类部分
class D2: public Base { public: using Base::Base; double d; }; // C++11 继承构造
```

7.5 派生类成员的标识与访问

- **同名隐藏：**派生类新增成员与基类同名时，默认访问本类成员；基类同名成员用作用域分辨符访问 Base::m
- **隐藏的全部性：**隐藏的是基类同名函数的所有重载形式（哪怕参数表不同）；想保留须 using Base::f; 或显式写 Base::f(…)
- **作用域分辨符：**类名::成员名 解决同名二义性（多继承两个基类都有同名成员时，必须写 Base1::f / Base2::f）
- **虚基类：**共同基类的成员在多继承中只保留一份——class D : virtual public B
- 虚基类由最远派生类的构造函数初始化（否则由其默认构造）

```
class D : public B1, public B2 { … };
d.f();        // 二义性：两个基类都有 f
d.B1::f();    // 用作用域分辨符指明
```

- **派生类成员按访问属性分 4 种：**①不可访问成员——从基类私有成员继承而来，派生类内外都无法访问；②私有成员——派生类内部可访问，建立对象的模块中无法访问，继续派生时变成新派生类的不可访问成员；③保护成员——派生类内部可访问，建立对象的模块无法访问，进一步派生可能成为新派生类的私有或保护成员；④公有成员——派生类内外都可访问，继续派生视继承方式可能成为新派生类的私有、保护或公有成员
- **二义性问题：**对派生类的访问要解决两个问题——唯一标识问题和可见性问题，只能访问一个能够唯一标识的可见成员；如果通过某表达式引用的成员不止一个，就是二义性问题
- **作用域分辨符「::」的一般使用形式：**类名::成员名（数据成员）；类名::成员名(参数表)（函数成员）
- **隐藏规则：**若存在两个或多个具有包含关系的作用域，外层声明标识符而内层没有再次声明同名标识符，外层标识符在内层仍然可见；内层声明同名标识符则外层标识符在内层不可见（被隐藏）；基类成员和派生类新增成员都具有类作用域，二者作用范围不同但相互包含（基类在外层、派生类在内层）
- **细节：**如果派生类声明与基类成员函数同名的新函数，即使函数参数表不同，从基类继承的同名函数所有重载形式也都会被隐藏——子类中与父类同名但参数表不同的函数不属于重载，只有相同作用域中定义的同名函数才可重载
- **多继承同名成员：**若某派生类的多个基类拥有同名成员，而派生类没有新增同名成员，「对象名.成员名」无法唯一标识，必须用作用域分辨符加基类名限定（d.Base1::var=2; p->Base2::fun();）
- **用 using 声明消除二义性：**class Derived: public Base1, public Base2 { public: using Base1::var; using Base1::fun; }; 把 Base1 中的名字引入派生类作用域，d.var 和 d.fun() 明确引用 Base1 的成员；using 的一般功能是将一个作用域中的名字引入另一个作用域
- **using 引入基类函数名的细节：**将 using 用于基类中的函数名，派生类中如果定义同名但参数不同的函数，基类的函数不会被隐藏，两个重载版本并存（class Derived2: public Base1 { public: using Base1::fun; void fun(int i){…} }; 无参和 int 参两个 fun 都可调用）
- **共同基类（共享基类）引起的二义性：**当派生类的多个直接基类本身又是从同一个共同基类派生而来时，间接基类的成员经过不同派生路径以相同名字出现于派生类中
- **例 7-7：**Base0 声明 var0 和 fun0，Base1、Base2 由 Base0 公有派生，Derived 再由 Base1、Base2 共同公有派生——Derived 类中通过 Base1、Base2 继承来的 Base0 成员 var0 和 fun0 各有两份
- **限定规则：**这时用基类名 Base0 来限定无效（Base0::var0 仍无法表明从 Base1 还是 Base2 继承来），必须使用直接基类 Base1 或 Base2 的名称来限定，才能唯一标识：d.Base1::var0=2; d.Base2::fun0();
- **存储代价：**派生类对象在内存中同时拥有成员 var0 的两份同名副本，可存放不同的数据，分别通过 Base1 和 Base2 调用 Base0 的构造函数初始化——同一成员的多份数据增加了内存开销
- **提示：**其实 Base0 类的函数成员 fun0 的代码始终只有一个，之所以调用 fun0 时仍需用基类名 Base1 或 Base2 加以限定，是因为调用非静态成员函数总是针对特定对象的，需要将指向该类的一个对象的指针作为隐含参数传递给被调函数来初始化 this 指针

```
class Base0 { public: int var0;  void fun0() { } };
class Base1: public Base0 { public: int var1; };
class Base2: public Base0 { public: int var2; };
class Derived: public Base1, public Base2 { public: int var;  void fun() { } };
Derived d;
// d.var0=1;          // 错：二义性，两份 var0
d.Base1::var0=2;      // 必须用直接基类名限定
d.Base2::fun0();
```

- **虚基类的动机：**当某类的部分或全部直接基类是从另一个共同基类派生而来时，直接基类中从上一级共同基类继承来的成员拥有相同名称，派生类对象中同名数据成员在内存中同时拥有多个副本；很多情况下只需要一个这样的数据，同一成员的多份数据增加了内存开销

- **虚基类的作用**：使用作用域分辨符可唯一标识并分别访问它们，也可以将共同基类设置为虚基类，这时从不同路径继承过来的同名数据成员在内存中就只有一份，同一个函数名也只有一个映射，从而解决同名成员的唯一标识问题
- 虚基类声明语法（在派生类的定义过程中进行）：class 派生类名: virtual 继承方式 基类名——virtual 关键字的作用范围和继承方式关键字相同，只对紧跟其后的基类起作用；声明了虚基类之后，虚基类的成员在进一步派生过程中和派生类一起维护同一个内存数据
- 使用虚基类后（例 7-8），派生类对象中只有唯一的数据成员 var0，在建立 Derived 类对象的模块中，直接使用「对象名.成员名」方式就可以唯一标识和访问这些成员，d.var0=2; d.fun0(); 不再产生二义性
- **例 7-7 与例 7-8 的比较**：前者在派生类中拥有同名成员的多个副本，分别通过直接基类名来唯一标识，可存放不同的数据、进行不同的操作，可容納更多数据；后者只维护一份成员副本，使用更为简洁、内存空间更为节省——具体程序设计中根据实际问题需要选用合适的方法

```
class Base0 { public: int var0;  void fun0() { } };
class Base1: virtual public Base0 { public: int var1; };
// Base0 为虚基类
class Base2: virtual public Base0 { public: int var2; };
class Derived: public Base1, public Base2 { public: int
var;  void fun() { } };
Derived d;
d.var0=2;  d.fun0();       // 只有一份，直接访问即可（仍可用
Base1::var0 限定）
```

- **规则**：如果虚基类声明有非默认形式的（即带形参的）构造函数，并且没有声明默认形式的构造函数，事情就比较麻烦了——这时，在整个继承关系中，直接或间接继承虚基类的所有派生类，都必须在构造函数的成员初始化列表中列出对虚基类的初始化
- 构造一个类对象的一般顺序（细节 4 步）：①若有直接或间接的虚基类，先执行虚基类构造函数；②其他非虚基类按继承声明列表中的次序执行；③类定义中按声明顺序初始化新增成员对象（成员对象在初始化列表出现则按指定参数，未出现则默认构造；基本类型成员在初始化列表则赋值，否则不做）；④执行构造函数的函数体
- **例 7-8 要点**：Derived 派生 Base1、Base2（虚继承 Base0），初始化列表同时写 Base0(var)、Base1(var)、Base2(var)；编译器只由「最远派生类」（即正在建立对象的类）的构造函数调用虚基类构造函数一次，中间基类（Base1/Base2）对虚基类构造函数的调用被自动忽略（不会初始化 3 次）
- **注意**：虚基类声明只是在类的派生过程中使用了 virtual 关键字，在程序主函数中创建了一个派生类的对象，通过成员名称就可以访问该类的成员 var0 和 fun0

```
class Base0 { public: Base0(int var) : var0(var) { }  int
var0; };
class Base1: virtual public Base0 {
public: Base1(int var) : Base0(var) { }  int var1;
};
class Base2: virtual public Base0 {
public: Base2(int var) : Base0(var) { }  int var2;
};
class Derived: public Base1, public Base2 {
public:
Derived(int var) : Base0(var), Base1(var), Base2(var) { }
// 实际只有 Derived 的 Base0(var) 生效一次，Base1/Base2 中的
被忽略
};
```

7.8 深度探索（了解即可）

- **继承 vs 组合抉择**：能组合（has-a）就不继承；只有真正的"是一种"（is-a）关系才用继承
- **内存布局**：单继承派生类对象含完整基类子对象；多继承含多个基类子对象；虚继承通过指针间接指向共享的虚基类子对象
- 基类向派生类的转换（向下转型）不保证安全，需要时配合运行时类型识别（第 8 章 dynamic_cast）

第 8 章 多态性

8.1 多态性概述

- **多态**：同一消息被不同类型的对象接收产生不同行为
- **编译时多态**：函数重载、运算符重载（静态绑定）
- **运行时多态**：虚函数（动态绑定）
- **多态的 4 种类型**：重载多态、强制多态 = 专用多态；包含多态（虚函数）、参数多态（模板）= 通用多态
- **绑定 binding**：把操作调用与代码连结的过程；静态绑定（编译时确定）/ 动态绑定（运行时确定）
- **多态的定义**：同样的消息被不同类型的对象接收时导致不同的行为；消息指对类的成员函数的调用，不同行为指不同的实现（调用不同的函数）；典型例子是运算符「+」——整型数、浮点数及相互加法被不同类型对象接收后采用不同方式相加（整型+浮点先转换再相加），就是多态现象
- **多态的 4 种类型**：重载多态（普通函数及类成员函数重载）、强制多态（将一个变元类型加以变化以符合函数或操作要求，如加法运算浮点+整型先强制转换）、包含多态（类族中定义于不同类的同名成员函数的多态行为，主要通过虚函数实现）、参数多态（与类模板相关联，使用时必须赋予实际类型才能实例化）；前两种统称专用多态，后两种统称通用多态；本章重点：重载和包含两种
- **多态的实现角度**：编译时多态（编译时确定同名操作的具体操作对象）和运行时多态（运行中才动态确定）；这种确定具体操作对象的过程就是绑定（binding，也称编联、束定）——把一条消息和一个对象的方法相结合
- 绑定分为静态绑定（编译连接阶段完成，又称早期绑定/前绑定——重载、强制、参数多态）和动态绑定（编译连接无法解决调用与执行代码的关系、要等程序运行才能确定，又称晚期绑定/后绑定——包含多态操作对象的确定通过动态绑定完成）
- **8.2 运算符重载的引入**：C++ 预定义运算符的操作对象只能是基本数据类型；用户自定义类型（类）也需要类似运算（如复数 a+b），需要编写程序说明运算符作用于类对象时实现什么功能——对已有运算符赋予多重含义，使同一运算符作用于不同类型数据时导致不同行为
- **运算符重载的实质就是函数重载**：先把指定运算表达式转化为对运算符函数的调用，运算对象转化为运算符函数的实参，再根据实参类型确定需要调用的函数，此过程在编译过程中完成

8.2 运算符重载

- **本质**：运算符重载是函数重载，重载后的运算规则由程序员定义
- **规则①**：不能改变优先级和结合性
- **规则②**：不能改变操作数个数
- **规则③**：不能创建新运算符
- **规则④**：至少一个操作数是自定义类型
- **规则⑤**：= [] () -> 只能重载为成员函数
- **重载为成员函数**：左操作数是对象本身（隐式 this），形参个数 = 操作数个数 −1
- **重载为非成员函数**：通常是友元（需要访问私有成员），二元运算符两个形参都写出
- 前置 ++ 无参、后置 ++ 带 int 占位参数区分
- 流插入 << 只能重载为非成员函数

```
class Complex {
public:
    Complex(double r=0, double i=0) : r(r), i(i) {}
    Complex operator+(const Complex &c) const {   // 成员
函数版
        return Complex(r+c.r, i+c.i);
    }
    Complex& operator++()     { r++; return *this; }   //
前置
    Complex  operator++(int) { Complex t=*this; r++; return
t; } // 后置
private: double r, i;
};
// ostream& operator<<(ostream &out, const Complex &c)
//     { return out << c.r << "+" << c.i << "i"; }   // 非
成员(友元)
```

- **重载规则（3 条）**：① C++ 运算符除了少数几个外全部可以重载，且只能重载 C++ 中已有的运算符；② 重载之后运算符的优先级和结合性不变；③ 重载是针对新类型数据的实际需要，重载功能应与原有功能相类似，

不能改变原运算符操作对象个数，同时至少有一个操作对象是自定义类型
- **不能重载的运算符：**类属关系运算符「.」、成员指针运算符「.*」、作用域分辨符「::」、三目运算符「?:」——前两个保证成员访问功能不被改变，作用域分辨符操作数是类型而非表达式
- **两种重载形式：**重载为类的非静态成员函数（语法：返回类型 类名::operator 运算符(形参表){函数体}）/ 重载为非成员函数（语法：返回类型 operator 运算符(形参表){函数体}）；operator 是定义运算符重载函数的关键字
- **参数个数规则：**重载为成员函数时参数个数比原操作数个数少一个（后置「++」「--」除外），因为第一个操作数作为函数调用的目的对象（this）；重载为非成员函数时参数个数与原操作数相同，所有操作数显式通过参数传递
- **提示：**以非成员函数形式重载时，仅在需要访问类的私有或保护成员时才声明为友元函数——不声明为友元的函数只依赖类的接口（接口不变实现就无需变化），声明为友元后函数会依赖类的实现
- **重载为成员函数：**实质是函数重载，可自由访问本类数据成员；双目运算符左操作数是对象本身数据（this 指出），右操作数通过参数表传入；例 8-1 复数加减法用临时无名对象作返回值：return Complex(real+c2.real, imag+c2.imag);
- 单目运算符++重载为成员函数（例 8-2）：前置++无形参，返回 *this 引用（先自增再返回）；后置++有一个 int 型形参（哑元，仅用于区别前置），实现 Clock old=*this; ++(*this); return old;（先保存旧值，调用前置++，返回旧对象）；细节：函数参数表中未使用的参数 C++ 允许不给参数名
- **重载为非成员函数：**操作数都通过形参表传递，形参从左到右的顺序就是运算符操作数的顺序；双目运算符 B：oprd1 B oprd2 两个操作数依次作为第一、第二个形参；前置单目 U：oprd 作为唯一形参；后置单目 U：第一个形参为 oprd，第二个形参是 int 类型形参
- **非成员函数重载的三点原因（例 8-3）：**① 要重载的运算符第一个操作数不是可更改类型（如「<<」第一个操作数是标准库 ostream 类型，无法为其添加成员函数）；② 以非成员函数形式重载支持更灵活的类型转换——Complex 构造函数使实数可隐含转换为 Complex，5.0+c1 和 c1+5.0 都可以（成员函数重载时左操作数必须是该类类型，目的对象不会被隐含转换，只有右操作数可以）；③「=」「[]」「()」「->」只能被重载为成员函数，而且派生类中的「=」运算符函数总会覆盖基类中的「=」运算符函数
- **流插入运算符重载要点（例 8-3）：**ostream& operator<<(ostream &out, const Complex &c){out<<"("<<c.real<<", "<<c.imag<<")"; return out;}——第一个参数为 ostream 引用、返回 ostream 引用，是为了支持 cout<<c1<<c2 连续输出（第二个<<的左操作数是第一个<<的返回结果）

```
class Complex {
public:
Complex(double r=0, double i=0) : real(r), imag(i) { }
Complex operator+(const Complex &c2) const {  // 成员函数重载
return Complex(real+c2.real, imag+c2.imag);
}
private: double real, imag;
};
// 非成员函数重载 + 连续输出
ostream& operator<<(ostream &out, const Complex &c) {
out<<"("<<c.real<<", "<<c.imag<<")";  return out;
}
Clock& operator++() { /*自增*/ return *this; }   // 前置++: 无形参
Clock operator++(int) { Clock old=*this; ++(*this); return old; } // 后置++: int 哑元
```

8.3 虚函数

- virtual 声明的成员函数；通过基类指针或引用调用时，按对象的实际类型动态绑定（运行时才决定调哪个）
- **覆盖规则：**派生类重写的虚函数须同名、同参数表、同返回类型（协变除外）
- **运行时多态的 3 个条件：**① 赋值兼容（基类指针/引用指向公有派生类对象）② 基类声明虚函数且派生类覆盖 ③ 经指针/引用调用；用对象名直接调用 → 静态绑定
- 虚函数一般不声明为 inline（运行时才决定调哪个，无法嵌入展开）
- **陷阱：**虚函数的缺省形参值按静态类型取（编译时绑定），与对象实际类型无关
- 虚特性沿继承链传递，派生类中可省略 virtual

```
class Shape {
public:
    virtual double area() const { return 0; }
};
class Circle : public Shape {
public:
    double area() const override { return 3.14*r*r; }  // 重写
private: double r = 1;
};
Shape *p = new Circle();
cout << p->area();   // 调 Circle::area() — 动态绑定
```

- **override（C++11）：**显式标注"这是重写"，写错签名直接编译报错
- **final：**禁止该类被继承或该函数被重写
- **虚析构：**通过基类指针 delete 派生类对象时，基类析构函数必须为虚函数，否则派生类析构不执行（资源泄露）
- **虚函数的引入：**利用一个循环结构依次处理同一类族中不同类的对象——Account *accounts[]={&sa1,&sa2,&ca}; 循环调用 accounts[i]->show()；若 show 不是虚函数，无论指针指向哪种类型实例，调用的都是 Account 类中定义的 show（静态绑定），而希望根据实例类型决定调用哪个 show
- 虚函数是动态绑定的基础；虚函数必须是非静态的成员函数；声明语法：virtual 函数类型 函数名(形参表);——virtual 关键字限定只能在类定义中的函数原型声明，不能在成员函数实现时使用
- **运行过程中多态需满足的 3 个条件：**①类之间满足赋值兼容规则；②声明虚函数；③由成员函数调用或通过指针、引用来访问虚函数；若使用对象名访问虚函数，则绑定在编译过程中进行（静态绑定）
- **习惯：**虚函数一般不声明为内联函数——虚函数调用需动态绑定，对内联函数的处理是静态的（声明为内联不会引起错误）；UML 中一般虚函数在成员函数前添加<<virtual>> 构造型
- **虚函数覆盖的 3 条判断规则：**派生类的成员函数满足下列条件的自动确定为虚函数，覆盖基类的虚函数——①与基类的虚函数有相同名称；②与基类的虚函数有相同参数个数及相同的对应参数类型；③与基类的虚函数有相同返回值或者满足赋值兼容规则的指针、引用型返回值；不仅如此，派生类中的虚函数还会隐藏基类中同名函数的所有其他重载形式
- **细节：**用指向派生类对象的基类指针仍可访问基类中被覆盖的成员函数，使用「::」限定：ptr->Base1::display() 则无论所指对象动态类型是什么，总是调用 Base1 类的 display()；在派生类函数中可用「基类名::函数名(...)」先调用基类被覆盖的函数再执行派生类特有操作
- **习惯：**派生类覆盖基类成员函数时可用也可不用 virtual 关键字，二者没有差别（很多人习惯写，以清楚提示这是虚函数）
- 基类构造函数/析构函数中调用虚函数的规则：执行派生类构造函数时需首先调用基类构造函数，若基类构造函数调用了虚函数，被调用的是基类的版本而不是派生类的——因为基类构造时对象还不是派生类对象；同样基类析构时对象已不再是派生类对象，若基类析构函数调用虚函数则调用基类的版本
- **只有虚函数是动态绑定的：**基类中声明的非虚函数通常代表不希望被派生类改编的功能，一般不要重写继承来的非虚函数（语法无强制限制）——会导致通过基类指针和派生类指针/对象调用同名函数产生不同结果，引起混乱
- **注意：**重写继承来的虚函数时，如果函数有缺省形参值，不要重新定义不同的值——虚函数是动态绑定的，但缺省形参值是静态绑定的：通过指向派生类对象的基

类指针可访问派生类虚函数，但缺省形参值只能来自基类定义

- **强调：**只有通过基类的指针或引用调用虚函数时才会发生动态绑定——若把 fun 函数参数类型设为 Base1 对象（而非 Base1*），3 次调用执行的都是 Base1::display()，因为基类对象不能表示派生类对象

```
class Base1 {
public: virtual void display() const;   // 基类声明为虚函数
};
class Base2: public Base1 {
public: void display() const;          // 自动成为虚函数（覆盖）
};
void fun(Base1 *ptr) { ptr->display(); }  // 运行时决定调哪个版本
//                           fun(&base1)→Base1::display();
fun(&base2)→Base2::display()
ptr->Base1::display();  // :: 强制调用基类版本（静态绑定）
```

- **override 说明符（C++11）：**派生类中定义与基类虚函数同名但形参列表不同的函数仍合法，但该函数与基类虚函数相互独立（没有覆盖），本希望覆盖却因弄错参数列表没达到预期；用 override 标记后编译器发现错误——标记了 override 但该函数并没有覆盖已存在的虚函数时编译报错；只有虚函数可被覆盖，非虚函数（如 f3）不能用 override 声明
- **final 说明符：**把某个函数指定为 final 意味着该函数不能被覆盖，任何试图覆盖该函数的操作都将引发错误（class Derived2{ void f1(int) const final; }; 则 Derived3 中再定义 f1(int) 报错）；final 和 override 说明符需出现在参数列表以及尾置返回类型之后

```
class Base {
public: virtual void f1(int) const;  virtual void f2();
void f3();
};
class Derived1: public Base {
public:
void f1(int) const override;  // 正确：覆盖
// void f2(int) override;     // 错：基类没有 f2(int)
// void f3() override;        // 错：f3 不是虚函数
};
class Derived2: public Base { public: void f1(int) const final; };
class Derived3: public Derived2 {
// void f1(int) const;          // 错：f1 在 Derived2 中是 final
};
```

- C++ 中不能声明虚构造函数，但可以声明虚析构函数；声明语法：virtual ~类名();
- 若一个类的析构函数是虚函数，由它派生而来的所有子类的析构函数也是虚函数；析构函数设置为虚函数后，使用指针引用时可以动态绑定，保证使用基类类型的指针就能调用适当的析构函数针对不同对象进行清理
- **判断标准：**如果有可能通过基类指针调用对象的析构函数（通过 delete），就需要让基类的析构函数成为虚函数，否则会产生不确定的后果
- **例 8-5 的教训：**Base *b=new Derived(); fun(b) 中 delete b——若 ~Base() 非虚，运行时只输出 Base destructor，派生类析构函数没有被执行，派生类中动态分配的内存（成员 p 指向的内存）没得到释放，造成内存泄漏；改为 virtual ~Base(); 后输出 Derived destructor 再 Base destructor，动态申请的内存被正确释放

```
class Base {
public: virtual ~Base() { }   // 虚析构：delete 基类指针才安全
};
class Derived: public Base {
public: ~Derived() { delete p; }
private: int *p;
};
Base *b=new Derived();
delete b;   // 虚析构→先调 ~Derived 再调 ~Base；非虚→只调 ~Base，内存泄漏
```

8.4 纯虚函数与抽象类

- **纯虚函数：**virtual 返回类型 函数名(形参) = 0; 只有声明没有实现
- **纯虚函数 vs 空虚函数：**= 0 无函数体（强制派生类实现）；virtual void f() {} 有实现、可正常调用
- **抽象类：**含纯虚函数的类；不能定义对象（可定义指针/引用）
- 派生类不重写全部纯虚函数则仍是抽象类
- **用途：**定义接口，强制派生类实现
- **典型用法：**把"被积函数"抽象成抽象类，求积分类只与接口打交道——新增函数无须改求积分类

```
class Shape {
public:
    virtual double area() const = 0;   // 纯虚函数
    virtual ~Shape() {}                // 虚析构
};
// Shape s;     // 错误：抽象类不能实例化
Shape *p;       // 指针可以
```

- **纯虚函数的引入：**基类不知道该如何处理某操作（如 Account 的 deposit/withdraw/settle），无法给出有意义的实现——能否在基类中只说明函数原型来规定整个类族的统一接口形式，而在派生类中再给出具体实现
- **纯虚函数声明格式：**virtual 函数类型 函数名(参数表)=0;——与一般虚函数成员原型书写格式不同就在后面加了「=0」；声明后基类中可以不再给出函数实现部分，函数体由派生类给出
- **细节：**①基类中仍然允许对纯虚函数给出实现，但即使给出实现也必须由派生类覆盖，否则无法实例化；②对基类中为纯虚函数定义的函数体的调用，必须通过「基类名::函数名(参数表)」的形式；③如果将析构函数声明为纯虚函数，必须给出它的实现——因为派生类析构函数执行完后需要调用基类析构函数
- **注意：**纯虚函数不同于函数体为空的虚函数——纯虚函数根本没有函数体，空虚函数函数体为空；前者所在类是抽象类不能直接实例化，后者所在类可实例化；二者都可以派生新类并在新类中给出新的实现，且新实现可具有多态特征
- **抽象类：**带有纯虚函数的类是抽象类；主要作用是通过它为一个类族建立一个公共接口，使它们更有效地发挥多态特性；抽象类处于类层次上层，自身无法实例化，只能通过继承机制生成抽象类的非抽象派生类然后再实例化
- **抽象类派生新类后的判定：**派生类给出所有纯虚函数实现，就可以定义自己的对象，不再抽象；反之如果没有给出全部纯虚函数实现，派生类仍是抽象类
- 抽象类不能实例化（不能定义抽象类对象），但可以定义抽象类的指针和引用，通过指针或引用指向并访问派生类对象；UML 中抽象类通过斜体书写类名表示，纯虚函数（抽象函数）斜体书写并添加 <<abstract>> 构造型
- **例 8-6 要点：**基类 Base1 将 display() 声明为纯虚函数成为抽象类，无法定义 Base1 对象但可定义其指针；派生类 Base2、Derived 给出实现后可定义对象，fun(Base1 *ptr){ptr->display();} 通过抽象类指针实现同一类族对象统一的多态处理

```
class Base1 {
public: virtual void display() const = 0;  // 纯虚函数：Base1 成为抽象类
};
// Base1 b1;           // 错：抽象类不能实例化
Base1 *p;             // 但可以定义抽象类的指针和引用
class Base2: public Base1 {
public: void display() const { cout<<"Base2\n"; }
};
Base2 b2;  p=&b2;  p->display();   // 输出 Base2
```

8.7 深度探索（了解即可）

- **多态类型（含虚函数）/ 非多态类型：**只有多态类型才支持运行时类型识别
- **非多态类型的利与弊：**无虚表指针、静态绑定更快；但不能用 dynamic_cast / typeid 做运行时类型识别
- **RTTI：**dynamic_cast<Derived*>(p) 安全的向下转型（失败返回 nullptr，引用失败抛异常）
- typeid(对象) 返回 type_info，可比较类型、取类型名

- **虚函数动态绑定实现：**对象中含指向虚函数表（vtable）的指针，调用时经虚表间接跳转；这就是虚函数有少量开销的原因
- **多态类型与非多态类型：**C++ 类类型分两类——多态类型（有虚函数的类）和非多态类型（所有其他类型）；基类是多态类型时，通过基类指针调用基类虚函数的实际操作由派生类决定——派生类只是继承了基类的接口，不必继承基类中虚函数的实现
- **设计多态类型的重要原则：**把多态类型的析构函数设定为虚函数（Food *f=new Icecream(); … delete f; 只有 ~Food() 是虚函数才能避免通过指针删除对象时发生不确定行为）
- **对非多态类型的公有继承应当慎重：**非多态基类的指针可指向派生类对象，但通过该指针所做的操作只能是基类本身的操作，继承作用大打折扣；凡是能通过继承非多态基类实现的派生类，一般都能用组合方式实现；且非多态基类没有虚析构函数，通过基类指针删除派生类堆对象会出现不确定性问题——一般没有太大必要
- **非多态类型的优点：**每个成员函数实现都是静态绑定，执行效率更高，不需要分配额外空间保存动态绑定所需信息；一般而言，如果一个函数执行方式十分明确、不希望派生类提供特殊实现，应声明为非虚函数
- **运行时类型识别（RTTI）的动机：**通过基类指针/引用利用多态性仅限于调用基类中声明的虚函数；若希望对派生类新函数（基类无相应虚函数）调用，需解决基类向派生类的转换——static_cast 不安全（只能在对象类型与目的类型兼容时执行，否则不确定行为），有时只有运行时才能知道指针所指对象的实际类型
- **dynamic_cast：**与 static_cast、const_cast、reinterpret_cast 并列的 4 种类型转换操作符之一，可将基类指针显式转换为派生类指针（或引用）；与 static_cast 不同：转换前检查指针所指向对象的实际类型是否与目的类型兼容，兼容才发生——指针转换不兼容得到空指针，引用转换不兼容抛出异常；另外转换前类型必须是多态类型（有虚函数）的指针/引用——C++ 只为多态类型在运行时保存类型识别信息
- **细节：**当原始类型为多态类型指针时，目的类型除派生类指针外还可是 void 指针——dynamic_cast<void*>(p) 先将 p 转为实际类型指针再转为 void 指针（得到 p 所指对象首地址；多继承情况下基类指针存储地址未必是对象首地址）
- **typeid：**C++ 关键字，两种语法形式 typeid(表达式) 或 typeid(类型说明符)；得到 type_info 类型的常引用（type_info 定义在 typeinfo 头文件），有 name() 成员获得类型名称（const char* name() const;），并重载了 == 和 != 可比较两个类型是否相同
- **typeid 的求值规则：**若 typeid 所作用表达式具有多态类型，表达式会被求值，得到描述表达式求值结果的运行时类型（动态类型）的 type_info 常引用；若表达式具有非多态类型，得到表达式静态类型的常引用，这时表达式不会被求值；只有作用于多态类型表达式时进行的才是运行时类型识别
- **typeid 的局限：**只能判断一个对象是否为某个具体类型，不会把子类型包括在内（传入 Derived 对象与 typeid(Base) 比较结果为 false）——判断对象是否为某类型或其子类型，用 dynamic_cast 更方便；C++ 标准未规定 name() 返回字符串的构造方式，各编译器实现不同；运行时类型识别需付出效率代价，不能作为常规手段，多数情况下派生类特殊性应通过在基类中定义虚函数体现

```
void fun(Base *b) {
b->fun1();
Derived1 *d=dynamic_cast<Derived1*>(b);
if(d!=0) d->fun2();        // 转换成功才调用派生类新函数
}
#include <typeinfo>
cout<<typeid(*b).name();          // 多态类型→运行时类型
if(typeid(*b)==typeid(Derived)) { } // 判断是否恰为 Derived（不含子类）
```

- **虚函数动态绑定的实现原理——直接方式：**在每个对象中除数据成员外为每个虚函数设置一个函数指针存放入口地址；派生类把新数据成员和函数指针置于从基继承的之后；各构造函数中为函数指针初始化（基类对象指向基类函数，派生类对象指向覆盖后的函数）；调用时先读取相应函数指针再通过它调用函数；致命问题：额外占用空间太大，不被广泛采用
- **虚表（Virtual Table）方式：**不同对象保存的虚函数指针都一样，可只保存一份构成虚表；每个对象只保存一个指向虚表首地址的指针——虚表指针（vptr），每个多态类型对象只需占用一个指针的额外空间；虚表本身每个多态类型只有一个，不会因新对象创建而增加
- **虚表机制：**每个类型各有一个虚表，基类声明的虚函数对应指针放在前面，派生类新增虚函数对应指针放在后面，子类型虚表和基类虚表中具有相同位置的项；通过基类指针或引用调用虚函数时，通过虚表指针找到该对象虚表，进而找到存放该虚函数指针的项，读出函数入口地址然后调用——虚函数动态绑定就是这样完成的
- **构造与虚表（提示）：**执行构造函数时首先被执行的是基类构造函数，构造派生类对象时该对象虚表指针首先指向基类虚表，只有基类构造函数调用完成后虚表指针才被指向派生类虚表——这就是基类构造函数调用虚函数时不会调用派生类虚函数的原因
- **多继承时：**每个基类各有各自虚表，继承了多个基类的派生类需要多个虚表（一个虚表分为多段，每个基类虚表指针指向其中一段的首地址）；虚表中存放的不只是虚函数指针，支持运行时类型识别的对象运行时类型信息也需通过虚表访问——只有多态类型有虚表，因此只有多态类型支持运行时类型识别

第 9 章 模板与群体数据

9.1 函数模板与类模板

- **模板：**把类型也作为参数，一段代码适用于多种类型（泛型编程的基础）
- **函数模板：**template <typename T> T max(T a, T b) { … }
- **函数模板调用：**编译器按实参类型自动实例化（也可显式 max<int>(a,b)）
- **类模板：**template <typename T> class Array { … }; 使用时给出类型参数 Array<int> a(10);
- 类模板成员函数类外定义也要写 template <typename T>
- **函数模板 ≠ 函数：**模板本身不生成目标代码，实例化后才生成函数代码
- 多个源文件引用同一模板 → 模板定义（不只是声明）应放在头文件中，否则连接出错
- 函数指针只能指向实例化后的模板函数，不能指向模板本身

```
template <typename T>
T myMax(T a, T b) { return a > b ? a : b; }
cout << myMax(3, 5);         // 实例化为 int 版
cout << myMax(2.5, 1.2);     // 实例化为 double 版
```

- **参数化程序设计：**为实现代码重用，代码必须具有通用性——现代代码需要不受数据类型影响且可自动适应数据类型变化的工具；模板是 C++ 支持参数化多态性的工具；所谓参数化多态性，就是将程序处理的对象类型参数化，使得一段程序可以处理多种不同类型对象
- **函数模板的动机：**重载函数即使同名，函数体仍要分别定义（如 int abs 和 double abs 功能完全一样只有参数类型不同）；使用函数模板，程序员只需编写一次，编译器基于调用时提供的参数类型自动产生相应函数
- **函数模板定义形式：**template <模板参数表> 类型名 函数名(参数表){函数体的定义}；模板参数表由逗号分隔的模板参数构成，可包括 3 类：①class（或 typename）标识符——指明可接受一个类型参数，代表任意类型（预定义或自定义）；②类型说明符 标识符——指明可接受一个由「类型说明符」所规定类型的常量作为参数；③template <参数表> class 标识符——指明可接收一个类模板名作为参数；类型参数可用来指定形参类型、返回值类型及声明函数中的局部变量
- **实例化：**调用时编译器从实参类型推导出类型参数（abs(n) 推出 T=int），以函数模板为样板生成一个函数，此过程称为函数模板的实例化，该函数称为函数模板的一个实例；从函数模板产生的相关函数都是同名，编译器用重载解决方法调用相应函数；函数模板本身也可用重载方式重载
- **函数模板与函数的本质区别 3 方面：**①函数模板本身在编译时不生成任何目标代码，只有由模板生成的实例会生成目标代码；②被多个源文件引用的函数模板应当连同函数一同放在头文件中，不能像普通函数那样只将声明放在头文件而把实现放在源文件；③函数指针只能指向模板的实例，不能指向模板本身

- **注意：**模板参数实参不总是可用任何类型——例 9-1 中要求实参类型应可进行插入运算（<<），STL 库中通过「概念」限制（第 10 章介绍）
- **类模板：**类是对一组对象公共性质的抽象，类模板属于更高层次的抽象——对不同类的公共性质抽象，因此类模板也常称为参数化类；使用类模板使类中某些数据成员、成员函数参数、返回值或局部变量能取不同类型
- **类模板声明语法：**template <模板参数表> class 类名{类成员声明};；类模板以外定义成员函数形式：template <模板参数表> 类型名 类名<模板参数标识符列表>::函数名(参数表)
- 类模板声明本身不是类，只有被其他代码引用时才根据引用需要生成具体类（实例化）；使用模板类建立对象：模板名<模板参数表> 对象名 1,…,对象名 n;；6.4 节 vector 就是类模板
- **模板友元（细节）：**模板参数表中类型参数也可声明为该模板类的友元类：template <typename T> class DemoClass{friend T; …};——T 被实例化后拥有对 DemoClass 成员的访问权限；特殊点：C++ 内置类型也可声明为友元，保证内置类型可用来实例化模板类（如 DemoClass<int>）
- **模板别名：**typedef DemoClass<int> IntDemoClass;（为实例化后的类定义别名）；为模板本身（而非实例化后的类）定义别名只能通过 C++11 的 using：template <typename T> using IntPair=pair<T, int>;（IntPair<string> label; 等价 pair<string,int> label;）

```
template <class T>        // 函数模板
T abs(T x) { return x<0 ? -x : x; }
// abs(n)→int abs(int); abs(d)→double abs(double)
template <class T, int SIZE=50>  // 类型参数+常量参数
class Stack { T list[SIZE]; /*…*/ };
template <typename T> using IntPair = pair<T, int>;
IntPair<string> label;   // 等价 pair<string,int>
template <typename T> class Demo { friend T; };
```

9.2 线性群体

- **数组类模板 Array<T>：**支持任意类型数组 + 越界检查
- 含指针成员的类必须自定义复制构造函数做深层复制（否则默认浅复制出错）
- **链表类：**结点类（数据 + 指向下一个结点的指针）+ 链表类（头指针、插入、删除、遍历）；动态组织、长度不限
- **栈类（后进先出 LIFO）：**push 入栈 / pop 出栈 / top 取栈顶；应用：后缀表达式整数计算器
- **栈/队列常用操作集：**push / pop / top（peek 只看不取）/ empty / size / clear
- **队列类（先进先出 FIFO）：**队尾入队、队首出队
- **线性群体分类：**直接访问群体（数组，按下标直接取）/ 顺序访问群体（链表，只能从头顺着走）
- **循环队列：**首尾相接的数组，用取模运算区分空/满

```
stack.push(x);  x = stack.pop();   // 后进先出
queue.enqueue(x); x = queue.dequeue(); // 先进先出
```

- **线性群体概念：**元素次序与其位置关系对应；按访问元素的不同方法分为直接访问、顺序访问和索引访问；栈和队列是两种特殊线性群体
- **栈：**只能从一端访问的线性群体，可访问一端称栈顶，另一端称栈底；向栈顶添加元素称「压入栈」（push），删除栈顶元素称「弹出栈」（pop）；添加删除具有「后进先出」(LIFO) 特性；应用：编译系统用栈实现函数调用参数传递和保留返回地址；表达式处理（操作数栈 ODS + 操作符栈 OPS，自左至右扫描）
- **队列：**只能向一端添加、从另一端删除的线性群体；可添加一端称队尾，可删除一端称队头；向队尾添加称「入队」，删除队头元素称「出队」；具有「先进先出」(FIFO) 特性；应用：模拟等待排队、操作系统打印作业调度
- **数组类的不足：**大小编译时已确定、运行时无法修改；不能有效避免下标越界
- **数组类模板 Array（例 9-3）：**由任意多个位置连续、类型相同的元素组成，元素个数可在程序运行时改变；私有成员 T *list（动态分配数组首地址）+ int size；主要接口：构造函数 Array(int sz=50)（assert(sz>=0); list=new T[size];）、复制构造函数、析构函数 delete[] list、重载「=」（整体赋值）、重载「[]」（越界检查 assert(n>=0 && n<size)）、指针转换运算符 operator T*()、getSize()、resize(int sz)（申请新数组、复制前 min(sz,size) 个元素、删除原数组）
- **浅层复制 vs 深层复制（分析）：**若复制构造只是 size=a.size; list=a.list;（浅层），则两对象使用同一段内存，析构时同一内存空间被两次释放，引起运行错误；正确做法是深层复制——list=new T[size]; for 循环逐个复制元素
- 与众不同的运算符——返回值为什么是引用（分析）：「[]」运算结果经常被放在赋值运算符左边作左值（a[3]=5），所以「[]」重载函数返回值必须指定为引用；const 版本返回常引用（不希望修改常对象，且 T 较复杂时避免复制构造开销）；「=」返回当前对象引用（return *this）是为了支持 a=b=c 的连续赋值并避免对象复制开销
- **类型转换函数：**operator S() 定义为 T 的成员函数，实现 T→S 的隐式/显式转换；转换运算符只能重载为成员函数；不需要指定返回类型（S 就是返回类型）；指针转换运算符让自定义类型对象可像普通指针一样使用（如把 Array 对象直接传给 read(int* p, int n)）
- **链表：**动态数据结构，结点运行时动态生成；结点=数据域+指向下一结点的指针域；单链表（只有一个后继指针）/双向链表（前趋+后继指针）；首结点称头结点，最后一个结点称尾结点，尾结点后继为空
- **结点类模板（例 9-5）成员：**数据成员 data + next；insertAfter（在结点后插入同类结点——实现顺序不可颠倒：先 q->next=p->next; 再 p->next=q;）、deleteAfter（删除后继结点并返回其地址：tempPtr=next; 无后继返回 0; next=tempPtr->next; return tempPtr）、nextNode（获取后继结点地址）
- **链表类（例 9-6）数据成员：**头指针 front、尾指针 rear、游标 prevPtr/currPtr（当前位置+前趋，配合插入删除）、size 元素个数、position 当前元素位置；成员函数：newNode/freeNode/copy（复制构造与 operator= 调用）、游标操作 reset/next/endOfList/currentPosition/data、insertFront/Rear/At/After、deleteFront/deleteCurrent、clear
- **栈类模板（例 9-8）：**template <class T, int SIZE=50>；私有成员 T list[SIZE] + int top（栈顶位置=数组下标）；构造函数 top=-1；push：assert(!isFull()); list[++top]=item；pop：assert(!isEmpty()); return list[top--]；peek：return list[top]；isEmpty：top==-1；isFull：top==SIZE-1；clear：top=-1；整型参数 SIZE 默认 50，可用常量或编译时可计算的常量表达式作实参，可当作常量使用（数组大小）
- **栈的应用（例 9-9 整数计算器）：**后缀表达式输入法（"3 5 +" 表示 3+5）；Calculator 类内嵌 Stack<double> 操作数栈；enter(num) 压栈；getTwoOperands 连续弹两个操作数（不足则报错并清栈）；compute(op) 执行运算、结果压栈并输出；除数为 0 时报错清栈；run() 读入算式，'-' 需判断是减号还是负号（str.size()>1 说明是负号），istringstream 将字符串转实数
- **队列类：**FIFO，队尾入队、队头出队；基本状态：一般、队空（count==0）、队满（count==SIZE）；数据成员 front、rear、count
- **数组实现队列的缺陷与循环队列：**普通数组出队时元素向队头移动、数据移动量大效率低；循环队列把数组弯成环形，front/rear 用取余运算 % 环形移动；判空判满靠 count：count==0 队空，count==SIZE 队满
- **队列类模板（例 9-10）：**template <class T, int SIZE=50>；构造 front(0), rear(0), count(0)；insert：assert(count!=SIZE); list[rear]=item; rear=(rear+1)%SIZE; count++；remove：assert(count!=0); temp=front; front=(front+1)%SIZE; count--; return list[temp]；getFront：return list[front]；clear：count=0; front=0; rear=0

```
template <class T> class Array {
public:
Array(int  sz=50)  {  assert(sz>=0);  size=sz;  list=new
T[size]; }
~Array() { delete[] list; }
Array(const Array<T>&a) {                  // 深层复制
size=a.size; list=new T[size];
for(int i=0; i<size; i++) list[i]=a.list[i];
}
T  &operator[](int  n)  {  assert(n>=0  &&  n<size);  return
```

```
list[n]; }
operator T*() { return list; }            // 指针转换：可传给 read(a,10)
void resize(int sz);
private: T *list; int size;
};
template <class T, int SIZE=50> class Stack {
void push(const T& item) { assert(!isFull()); list[++top]=item; }
T pop() { assert(!isEmpty()); return list[top--]; }
bool isEmpty() const { return top==-1; }
bool isFull() const { return top==SIZE-1; }
};
// 循环队列
rear=(rear+1)%SIZE;  // 入队
front=(front+1)%SIZE; // 出队
// 链表插入（顺序不可颠倒）
q->next=p->next;  p->next=q;
```

9.3 群体数据的组织（排序与查找）

C 基础 · 从简（算法思路与数据结构课相同，会写即可）

- **直接插入排序：**把元素逐个插入已排好序的子序列；n 小时效率高
- **简单选择排序：**每趟从未排序部分选最小（最大）放到已排序部分末尾
- **交换排序（起泡排序）：**相邻比较，逆序交换，一趟冒出一个最值
- **顺序查找：**逐个比较，不要求有序
- **折半查找：**要求有序，每次与中间比较缩小一半区间
- **插入排序基本思想：**每一步将一个待排序元素按其关键字值大小插入到已排序序列的适当位置，直到插完；初始 a[0] 视为已排，a[1]~a[n-1] 待排；最简单的是直接插入排序
- **直接插入排序实现：**for(int i=1; i<n; i++){ temp=a[i]; int j=i; while(j>0 && temp<a[j-1]){ a[j]=a[j-1]; j--; } a[j]=temp; }
- **选择排序基本思想：**每次从待排序序列中选关键字最小的元素，顺序排到已排序序列最后，直到排完；实现：for(int i=0; i<n-1; i++){ leastIndex=i; for(int j=i+1; j<n; j++) if(a[j]<a[leastIndex]) leastIndex=j; mySwap(a[i], a[leastIndex]); }
- **交换排序基本思想：**两两比较待排序序列元素，交换不满足顺序要求的各对元素，直到全部满足；最简单是起泡排序：第 1 个与第 2 个比较、逆序则交换，再比较第 2、3 个……直到第 n-1 与第 n 个，此为第一趟，最大元素「沉底」到第 n 个位置；共需 n-1 趟，第 i 趟对前 n-i+1 个元素进行
- **顺序查找：**最简单的查找，从数组首元素开始逐个与 key 比较，找到返回下标，否则返回 -1
- **折半查找（二分查找）：**要求元素按关键字有序；基本思想：一次比较后将序列分割成两部分，只在可能包含待查元素的一半中继续查找，逐步缩小范围；实现：low=0, high=n-1; while(low<=high){ mid=(low+high)/2; if(key==list[mid]) return mid; else if(key<list[mid]) high=mid-1; else low=mid+1; } return -1;

```
void insertionSort(T a[], int n) {
for(int i=1; i<n; i++) {
T temp=a[i]; int j=i;
while(j>0 && temp<a[j-1]) { a[j]=a[j-1]; j--; }
a[j]=temp;
}
}
int binSearch(T list[], int n, T key) {
int low=0, high=n-1;
while(low<=high) {
int mid=(low+high)/2;
if(key==list[mid]) return mid;
else if(key<list[mid]) high=mid-1; else low=mid+1;
}
return -1;
}
```

第 10 章 泛型程序设计与 C++ 标准模板库（STL）

10.1 泛型程序设计及 STL 的结构

- **泛型程序设计：**算法通过抽象（概念 Concept）表述，与具体类型无关；满足概念要求的类型就能用于该算法
- **STL 四组件：**容器（container）、迭代器（iterator）、函数对象（function object）、算法（algorithms）——算法与容器通过迭代器解耦
- 容器存放数据；迭代器遍历容器；算法操作迭代器指定的序列；函数对象作为算法的自定义策略
- 泛型程序设计=编写不依赖于具体数据类型的程序；C++ 中模板是泛型程序设计的主要工具；主要思想：将算法从特定数据结构中抽象出来，使算法通用、可作用于各种不同数据结构，不必为每种容器编写一套同样的算法
- **协议：**软件复用中复用双方需遵守一定协议——过程化复用的协议是函数原型（参数数量/类型要求）；面向对象复用的协议是派生类覆盖基类成员函数须维持参数表、保证返回类型兼容；泛型程序设计的协议由函数模板声明体现
- **概念（concept）：**描述泛型程序设计中作为参数的数据类型所需具备的功能——内涵=这些功能，外延=具备这些功能的所有数据类型；如「可以比大小、具有公有复制构造函数并可用 = 赋值的所有数据类型」=概念 Sortable
- **模型（model）：**具备一个概念所需功能的数据类型（如 int 是 Sortable 的模型）；概念之间有包含关系（子概念/精炼）：Sortable 是 Comparable 和 Assignable 的子概念
- **STL 历史：**最初由 HP 公司 Alexander Stepanov 和 Meng Lee 开发，1994 年纳入 C++ 标准；STL 定义了一套概念体系，类模板/函数模板参数都用概念规定；STL 是开放的，类型参数可以是标准库类型或自定义类型（只要是所要求概念的模型）
- **STL 4 种基本组件：**容器（container，容纳一组元素的对象）、迭代器（提供顺序访问容器中每个元素的方法，是泛化的指针）、函数对象（行为类似函数的对象，泛化的函数）、算法（70 多个，覆盖查找、排序、消除、计数、比较、变换、置换、容器管理等，最重要特性是统一性）
- **13 种基本容器：**vector、deque、list、forward_list、array、set、multiset、map、multimap 及 4 种无序容器（unordered_set/map/multiset/multimap）；使用不同容器需包含不同头文件
- **容器分两类：**顺序容器（一组同类型元素以严格线性形式组织：vector、deque、list、forward_list、array）和关联容器（按索引/键快速提取元素：set 和 map 属之；按组织方式又分有序[键按顺序存储]和无序[哈希函数]）

10.2 迭代器

- **迭代器：**抽象的"通用指针"，是算法与容器的桥梁（++ 前进、* 解引用）
- **输入/输出流迭代器：**istream_iterator / ostream_iterator 对接流
- **五种分类（能力递增）：**输入迭代器 → 输出迭代器 → 前向迭代器 → 双向迭代器（list、set）→ 随机访问迭代器（vector、deque，支持 +n、-n、[]、比较）
- **区间：**算法普遍用 [first, last) 前闭后开区间
- **辅助函数：**advance(it,n) 前进 n 步、distance(f1,f2) 距离
- **逆向迭代器：**rbegin()/rend() 反向遍历

```
vector<int> v = {3, 1, 2};
sort(v.begin(), v.end());   // 区间 [begin, end)
for (auto it = v.begin(); it != v.end(); ++it)
    cout << *it << " ";
```

- **迭代器：**提供顺序访问容器中每个元素的方法；可用 * 访问所指元素，++ 获得指向下一个元素的迭代器，-> 直接访问成员；s.begin() 指向第一个元素，s.end() 指向最后一个元素的下一位置；迭代器是算法和数据结构的「中间人」——比指针更抽象，可以指向容器中的一个位置，不必关心真正物理地址
- 输入流迭代器 istream_iterator<T>：类模板，从输入流连续输入某类型数据；T 需满足：有默认构造函数、可用 >> 输入；支持 *、->、++（* 访问刚读取的元素，++ 读下一个元素）；默认构造的迭代器指向输入流结束位置，与流比较可判结束
- 输出流迭代器 ostream_iterator<T>：向输出流连续输出数据；T 需可用 << 输出；构造：ostream_iterator(ostream& out) 或

ostream_iterator(ostream& out, const char* delimiter)（分隔符）

- **适配器（adapter）：**为已有对象提供新接口的对象，本身一般不提供新功能，只为改变对象接口；输入/输出流迭代器把流接口变为迭代器接口，属于适配器
- **5 种迭代器概念（子概念关系）：**输入迭代器、输出迭代器、前向迭代器、双向迭代器、随机访问迭代器；前向是输入/输出的子概念，双向是前向的子概念，随机访问是双向的子概念——能力弱的概念一定是能力强的概念的子集
- **输入迭代器：**只读、单向、不可重复遍历（istream_iterator 是典型）；若 p1==p2 则 ++p1==++p2；输出迭代器：只写、单向，*p1=t 写入；连续两次写或连续两次 ++ 间未写则行为不确定
- **前向迭代器：**可读可写、可重复的单向遍历，*p1 结果具 T& 类型，不再有输出迭代器「++ 与写入必须交替」的限制；双向迭代器：支持反向移动 --p1；随机访问迭代器：能力几乎和指针一样——p1+=n、p1-=n、p1+n、p1-n、p1-p2（返回整数）、p1 op p2（< <= > >= 比较）、p1[n]（等价 *(p1+n)）；vector 的迭代器是随机访问迭代器
- **迭代器区间 [p1, p2)：**含 p1 所指、不含 p2 所指；p1==p2 时空区间；同一容器且对 p1 执行 n 次 ++ 后 p1==p2 才是合法区间
- 设计算法用迭代器作参数时，应尽量用内涵尽可能小（外延尽可能大）的迭代器概念，适用范围最广
- **迭代器辅助函数：**advance(iter, n) 使 iter 前进 n 个元素（双向/随机访问 n 可取负值）；distance(first, last) 计算 first 经过多少次 ++ 到达 last，皆为随机访问迭代器时值等于 last-first（调用前必须 last>=first）
- **容器相关迭代器类型：**S::iterator（普通迭代器）、S::const_iterator（常迭代器，只读）；C++11 起 s1.cbegin()/cend() 明确返回常迭代器；可逆容器提供 rbegin()/rend() 逆向迭代器（类型 S::reverse_iterator）——逆向的 ++ 映射为普通的 --
- 逆向迭代器是普通迭代器的适配器；S::reverse_iterator(p1) 得对应逆向迭代器，逆向迭代器成员 base() 得原迭代器；r1.base()==p1 但指向元素相差一个（r1 指向 p1-1 所指元素）；s1.rbegin()==S::reverse_iterator(s1.end())，s1.rend()==S::reverse_iterator(s1.begin())
- **随机访问容器：**迭代器是随机访问迭代器，可用整数直接访问元素 s1[n]

```
vector<int> s;
vector<int>::iterator it=s.begin();   // 指向首元素
vector<int>::const_iterator cit=s.cbegin();
cout<<*it;  it++;  it->member;
copy(s.rbegin(), s.rend(), ostream_iterator<int>(cout, "
"));
istream_iterator<double> eos;          // 默认构造=流结束
transform(istream_iterator<double>(cin), eos,
ostream_iterator<double>(cout, "\t"), square);
advance(it, 3);  int n=distance(s.begin(), it);
```

10.3 容器的基本功能与分类

- **共同功能：**迭代器、size/empty、max_size、swap、== != 比较；不同容器头文件不同（<vector> <list> <map> 等）
- **分类：**顺序容器 / 关联容器（有序 + 无序）/ 容器适配器（stack、queue、priority_queue）

10.4 顺序容器

- **公共操作：**insert、erase、resize、clear、front/back、push_back/pop_back、push_front/pop_front（list/deque）
- **vector：**动态数组，随机访问快；尾部插入删除 O(1)，中间插入删除慢；迭代器在插入/删除后可能失效（旧式 for 下标遍历删除用 v.erase(it--)）
- **迭代器失效细则：**erase 使被删元素及其后的迭代器失效（erase 返回下一个有效迭代器）；insert 使插入点之后的迭代器失效；push_back 触发扩容（及 reserve）使全部迭代器失效
- **vector 容量管理：**capacity()/reserve()/shrink_to_fit()
- **deque：**分段数组，首尾插入删除都快
- **list：**双向链表，任意位置插入删除快、不支持随机访问，splice 拼接转移元素
- **forward_list：**单向链表；array：定长数组（栈上）
- **插入迭代器：**back_inserter(v)（尾插）、front_inserter（头插）、inserter(v, pos)——算法结果写入容器

```
vector<int> v;
v.reserve(100);      // 预留容量，减少重分配
v.push_back(1); v.push_back(2);
v.insert(v.begin(), 0);   // 任意位置插入
v.erase(v.begin()+1);      // 删除
```

- **适配器：**stack（后进先出）、queue（先进先出）、priority_queue（优先队列，最大元素先出）
- 适配器不支持迭代器 → 不能用 STL 算法直接遍历适配器

```
stack<int> s;  s.push(1); s.top(); s.pop();
priority_queue<int> q;  q.push(3); q.push(5);
// q.top() = 5 — 最大的先出
```

- **顺序容器基本功能：**默认构造空容器；== != < <= > >= 按字典序比较元素；begin()/end()；clear()；empty()；构造函数 S s(n, t)（n 个 t）、S s(n)（n 个元素均为 T()）、S s(q1, q2)（[q1,q2) 区间构造）；赋值函数 assign 与构造函数一一对应（先清除原元素）；C++11 列表初始化：list<int> numberseq={1,4,5,7};
- **插入：**s.insert(p1, t)（返回指向新元素的迭代器）、s.insert(p1, n, t)、s.insert(p1, q1, q2)（区间插入无返回值）、s.emplace(p1, args)（args 传 T 的构造函数构造新元素）；删除：s.erase(p1)（返回被删的下一个元素的迭代器）、s.erase(p1, p2)；resize(n)（多则末尾删除，少则末尾用 T() 填充）
- **首尾直接访问：**s.front()、s.back()（forward_list 无 back）；尾部插删：push_back(t)、emplace_back(args)、pop_back()；头部插删（list 和 deque 支持、vector 不支持）：push_front(t)、pop_front()
- **向量 vector：**元素连续存储在数组中；支持高效随机访问和高效向尾部加入/删除元素；插入位置之后元素顺序后移，效率不高、越靠前越慢
- **向量动态扩展机制：**空间不够时自动分配更大空间，原数据复制到新空间并释放原空间；每次扩展分配空间一般大于所需；已分配空间不会被释放（删元素后多出的空间留待再用），故容量常大于实际元素个数；size() 元素个数、capacity() 容量、reserve(n)（容量≥n 则不动，否则扩至不小于 n）——大量插入前用 reserve 预留空间避免多次重新分配
- **迭代器失效规则（vector）：**push_back 引起容量扩展会使此前获得的迭代器/指针/引用失效（内存地址改变）；未扩展时插入位置之后的迭代器/指针/引用失效，之前不受影响；删除元素后被删元素之前的继续有效、之后的失效；shrink_to_fit() 回收未使用空间使 capacity() 与 size() 相等（同样适用于 string 和 deque）；释放多余空间技巧：vector<T>(s.begin(), s.end()).swap(s);
- **双端队列 deque：**支持向两端高效插入数据、支持随机访问；实现为分段数组——元素分段存放在大小固定的数组中，各段首地址保存在索引数组；两端插入不会移动已有元素，但会使所有迭代器失效（不使指向已有元素的指针/引用失效）；两端删除效率高，只使被删元素的迭代器/指针/引用失效；中间插入/删除效率低（越靠近中间越低），使所有迭代器、指针、引用失效
- **列表 list：**不能随机访问，但可在任何位置高效插入删除（实现为双向链表）；特殊操作——接合 splice（将一个列表中删除的元素插入到另一个列表，不复制元素只改指针）：splice(p, s2)（s2 全部插入 p 之前，s2 清空）、splice(p, s2, q1)、splice(p, s2, q1, q2)；原先指向被接入元素的迭代器/指针/引用不受影响
- **列表成员函数版算法：**remove、remove_if、sort、unique、merge、reverse 等（针对 list 提供，效率更高；直接用通用 STL 算法不能达到最高效率）
- 单向链表 forward_list 和数组 array（C++11 新增）：forward_list 目标达到与最好手写单向链表相当性能，不支持 size 操作；未定义 insert/emplace/erase，定义 insert_after/emplace_after/erase_after（对 p1 所指元素之后的结点操作）；array 是对内置数组的封装，对象大小固定，定义时需显式指明容器大小
- **顺序容器选择建议：**只需向尾部加元素→vector；两端插入删除但需随机访问→deque；中间插入删除、不需随机访问→list 或 forward_list；array 相对内置数组更安全易用但大小固定不支持增删

- **插入迭代器（适配器）**：通过输出迭代器接口向指定位置插入元素——front_insert_iterator（只适用于 deque 和 list）、back_insert_iterator、insert_iterator（适用于所有顺序容器）；辅助函数：front_inserter(s)、back_inserter(s)、inserter(s, i)；例：copy(istream_iterator<int>(cin), istream_iterator<int>(), back_inserter(s));
- **容器适配器**：template <class T, class Container=deque<T>> class stack; template <class T, class Container=deque<T>> class queue;——栈可用任何顺序容器作基础容器，队列只允许前插顺序容器（deque 或 list）；共同操作：== != < <= > >=、size()、empty()、push(t)、pop()；不支持迭代器；s.top() 返回栈顶引用；s.front() 队头、s.back() 队尾
- 优先级队列 priority_queue：template <class T, class Container=vector<T>>；基础容器必须是支持随机访问的顺序容器，默认 vector；弹出顺序与压入顺序无关，与元素大小有关（每次弹出最「大」元素）；无比较操作；top() 返回即将被弹出的最「大」元素引用；元素类型为类/结构体时 < 运算符必须有定义，默认按 < 决定元素大小

```
vector<int> v;  v.reserve(1000);       // 预留避免多次重新分配
v.push_back(5);  v.insert(v.begin()+1, 9);
v.erase(v.begin());  v.resize(20);
cout<<v.front()<<" "<<v.back()<<" "<<v.capacity();
list<int> a={1,2}, b={3,4};
b.splice(b.end(), a);                  // a 清空，元素接合到 b，不改指针只改指针
deque<int> d;  d.push_front(1);  d.push_back(2);
stack<int> st;  st.push(1);  st.top();  st.pop();
priority_queue<int> pq;  pq.push(3);  pq.push(9);  pq.top();
// 9
```

10.5 关联容器

- 按键组织、自动有序（默认升序，按 < 比较，可自定义比较函数）；二分查找实现，查找快
- **分类**：单重（不允许重复键）/ 多重（允许重复）、简单（set）/ 二元（map）
- **set/multiset**：只有键；map/multimap：键 + 值（pair）；make_pair(k,v) 构造键值对
- **map 下标运算符**：m[key] 访问/插入元素（key 不存在时插入默认值）；multimap 无下标运算符
- **查找计数**：find(k)（返回迭代器）、count(k)、lower_bound/upper_bound/equal_range
- 关联容器插入/删除不使其他迭代器失效（结点式结构）→ 可在遍历中安全删除
- **比较函数须满足严格弱序**：a<a 必为假；a<b 与 b<a 不同时成立；满足传递性——否则运行期出错
- **map[] 陷阱**：m[key] 在 key 不存在时会插入默认值 → 判断键是否存在用 find/count，不要用 []
- **无序容器（C++11）**：unordered_set / unordered_map 等，哈希表实现，O(1) 查找，不保证有序

```
map<string, int> score;
score["Tom"] = 90;             // 插入/赋值
score.insert(make_pair("Li", 85));
cout << score["Tom"];          // 90
set<int> s = {3, 1, 2};        // 自动有序去重
```

- 关联容器每个元素都有一个键（key），元素位置不能由程序员随意决定，而是按键的取值升序排列；迭代器遍历 [begin(), end()) 总是升序：*iter <= *(iter+1) 总成立
- **查找优势**：顺序容器查找需逐个访问，最坏 n 次比较；关联容器将元素按键组织在一棵「平衡二叉树」中，最坏约 $\log_2 n$ 次比较
- **两种概念划分**：按是否允许重复键值分单重关联容器（键唯一：set 和 map）和多重关联容器（相同键允许重复：multiset 和 multimap）；按键和元素的关系分简单关联容器（以元素本身作为键：set/multiset，只有一个类型参数）和二元关联容器（元素由键和附加数据共同构成：map/multimap，两个类型参数——前键后附加数据）
- 二元关联容器元素用二元组 pair 表示（<utility> 头文件）：template <class T1, class T2> struct pair { T1 first; T2 second; … };（成员均 public）；支持列表初始化：pair<string,int> p={"first", 2};
- 关联容器的键之间必须能用 < 比大小；类类型需重载 <；「<」重载必须构成「严格弱序关系」：①非自反性：对任何 x，x<x 必须返回 false；②传递性：若 x<y 和 y<z 皆 true，则 x<z 必须 true
- **构造**：S s(q1, q2)（区间构造；单重关联容器中相同键只有第一个元素加入，多重关联容器全部无条件加入）
- **单重关联容器 s.insert(t)**：只有不存在相同键元素时才能成功，返回 pair<S::iterator, bool>（成功返回新元素迭代器和 true，否则返回同键元素迭代器和 false）；多重关联容器插入总会成功，返回已插入元素的迭代器；s.insert(p1, t) 的 p1 是提示插入位置（提示准确可提高插入效率，不准确也能正确插入）
- **基于键的查找和计数**：s.find(k)（无则返回 s.end()）；s.lower_bound(k)（第一个键值不小于 k 的元素的迭代器）；s.upper_bound(k)（第一个键值大于 k）；s.equal_range(k)（pair 区间 [p1,p2) 刚好包含所有键为 k 的元素）；s.count(k)（键为 k 的元素个数）
- 关联容器插入和删除不会使任何已有迭代器、指针或引用失效；迭代器遍历容器过程中避免使用插入删除操作
- **集合 set**：由不重复元素构成，元素本身有序，可高效查找指定元素、方便得到指定大小范围元素所处区间
- **映射 map**：像一个「字典」——按键查找附加数据；例 10-10 要点：courses.insert(make_pair("CSAPP", 3)); 查找 auto iter=courses.find(name); 累加 iter->second; courses.erase(iter)；通过键删除 courses.erase(name) 比通过迭代器删除效率低（需先按键找元素）
- **make_pair 函数**：template <class T1, class T2> make_pair(T1 v1, T2 v2){ return pair<T1,T2>(v1, v2); }——省去显式给出 pair 类型的烦琐
- **map 重载 [] 运算符**：s[k] 查找键 k 元素，存在则返回附加数据引用，不存在则插入新元素并返回其附加数据引用（初值为 V()，数值/指针类型为 0）；可直接 s[k]++；但 [] 不能替代 insert/find——它会为新键自动创建新元素，无法判断容器中是否有指定键
- **多重集合与多重映射**：用法与 set/map 差不多，差异主要在去除了键必须唯一的限制；较少用 find（一个键对应多个元素，find 所得迭代器位置不确定），访问一个键对应的每一个元素用 equal_range，得到元素个数用 count；映射的 [] 不被多重映射支持（一个键不能对应唯一元素）
- **无序容器（C++11）**：unordered_set、unordered_map、unordered_multiset、unordered_multimap；不用比较运算符组织元素，而是通过哈希函数和键类型的 == 运算符；哈希函数=将给定类型的值映射到整数值的函数，相等值必须映射到相同整数，不相等值尽量映射到不同整数；提供与有序容器相同操作，可互相替换，输出顺序与有序容器版本不同；默认用 hash<key_type> 生成哈希值（标准库为内置类型和 string 等提供 hash），自定义关键字类型需提供自己的 hash 模板版本

```
map<string, int> courses;
courses.insert(make_pair("CSAPP", 3));
auto r=courses.insert(make_pair("CSAPP", 4)); // 键已存在：r.second=false
cout<<courses["CSAPP"];    // 3；键不存在则插入新元素（值 0）
courses["CSAPP"]=5;  courses["CSAPP"]++;
map<char,int> cnt;
if(isalpha(c)) cnt[tolower(c)]++;        // 例 10-11 字母计数
multimap<string,string> mm;
auto range=mm.equal_range(name);          // 遍历一个键的所有值
for(auto it=range.first; it!=range.second; ++it) cout<<it->second;
set<int> s={3,5,7};  s.insert(4);  *s.begin();  // 最小元素
```

10.6 函数对象

- **函数对象**：重载了 operator() 的类对象，可像函数一样调用；可携带状态，是算法的策略参数
- **标准函数对象**：plus<T>、minus<T>、multiplies<T>、less<T>、greater<T>、equal_to<T> 等，配合算法改变行为
- **函数对象按参数/返回值分 5 类**：产生器（无参）、一元函数、二元函数、一元谓词、二元谓词（返回 bool 的称谓词）

- **lambda 限制**：形参不能有默认参数；默认不能修改值捕获的副本，须加 mutable
- **lambda 表达式（C++11）**：[捕获](形参) -> 返回类型 { 体 }；捕获 & 引用、= 值、空捕获不写 []；常用于自定义排序/查找规则
- **bind（C++11）**：bind(&函数/函数对象, 占位符_1, 定值…) 绑定部分参数生成新函数对象；bind2nd 为旧式（已废弃）

```
sort(v.begin(), v.end(), greater<int>());  // 降序
auto f = [](int x) { return x * x; };
cout << f(5);                 // 25
auto g = bind(plus<int>(), _1, 10);
cout << g(5);                 // 15
```

- 函数对象（function object / functor）：行为类似函数的对象，可不需参数也可带若干参数；任何普通函数、函数指针、lambda 表达式和任何重载了调用运算符 operator() 的类的对象都可作为函数对象传给算法作参数；大多数 STL 算法可用一个函数对象作为参数
- **常用函数对象分 5 类**：产生器（Generator）、一元函数（Unary Function）、二元函数（Binary Function）、一元谓词（Unary Predicate）、二元谓词（Binary Predicate）；一般按参数个数分：0 个=产生器、1 个=一元函数、2 个=二元函数
- accumulate 算法（<numeric>）：template<class InputIt, class T, class BinaryFunction> T accumulate(InputIt first, InputIt last, T val, BinaryFunction op); 对数组元素累积运算；val 为累积初值，op 为累积规则（缺省为 +）
- **类形式函数对象相比普通函数的优势**：可携带更多额外状态信息（例 10-14 类 MultClass 重载 operator()）
- 标准函数对象（<functional>，均为内联）：算术类 plus<T>（x+y）、minus<T>、multiplies<T>（x*y）、divides<T>、modulus<T>（x%y）、negate<T>（一元，-x）；谓词类 equal_to<T>、not_equal_to<T>、greater<T>、less<T>、greater_equal<T>、less_equal<T>、logical_and<T>、logical_or<T>、logical_not<T>
- **提示**：6 个比较大小关系函数对象依赖 T 的关系运算符；T 自定义类型时只需重载 <（如 less_equal(x,y) 转化为 !(y<x)）；2 种判等函数对象只依赖 ==
- lambda 表达式（C++11，未命名的内联函数）：形式 [捕获列表](参数列表) -> 返回类型 { 函数体 }；捕获列表=函数中使用的局部变量的列表；可忽略返回类型和参数列表（函数体一条 return 语句时可自动推断返回类型）；lambda 不能有默认参数
- **捕获方式**：值捕获 [size]、引用捕获 [&size]、隐式值捕获 [=]、隐式引用捕获 [&]；混用：[=, &x]（除 x 外均值捕获）、[&, x]（除 x 外均引用捕获）；在 lambda 中改变被捕获的值需 mutable 关键字（值捕获不改变原变量）；引用捕获可改变原值；C++14 支持 auto 类型参数
- **函数对象参数绑定 bind**：template<class F, class… Args> bind(F&& f, Args&&… args); auto newFunc=bind(Func, arg_list); _n 占位符映射 newFunc 调用时的参数与 arg_list 中参数的对应关系（_1 第一个参数……）；using namespace std::placeholders;
- **bind 示例（例 10-17）**：auto p=find_if(a.begin(), a.end(), bind(greater<>(), _1, 40));（40 绑定到 greater 的第 2 个参数，实现「大于 40」的一元谓词）；auto myLess=bind(greater<>(), _2, _1);（更换占位符位置实现 less）

```
int mult(int x, int y) { return x*y; }
accumulate(a, a+N, 1, mult);          // 连乘（普通函数作函数对象）
accumulate(a, a+N, 1, multiplies<int>());
sort(a.begin(), a.end(), greater<int>());   // 从大到小
auto lmd=[](int a, int b) { return a<b; };
auto inc=[count1]() mutable { return ++count1; };
auto p=find_if(a.begin(), a.end(), bind(greater<>(), _1, 40));
```

10.7 算法

- **四大类**：不可变序列算法（不修改元素）、可变序列算法（修改元素）、排序和搜索算法、数值算法；都在 <algorithm>（数值在 <numeric>）
- **不可变序列**：for_each、find、find_if、count、count_if、equal、mismatch、search、search_n、adjacent_find、max/min
- **可变序列**：copy/copy_n/copy_backward、fill/fill_n、generate、remove/remove_if（不真正删除，返回新末尾，配合 erase）、replace、reverse、rotate、swap、transform、unique（去相邻重复）、partition
- **排序搜索**：sort（随机迭代器、快速排序）、stable_sort（稳定）、partial_sort（局部排序）、binary_search/lower_bound/upper_bound/equal_range（要求有序）、merge/inplace_merge、min/max_element、next_permutation、集合操作 set_union/intersection/difference/includes、堆操作 make_heap/push_heap/pop_heap/sort_heap、nth_element
- **数值算法（<numeric>）**：accumulate 累加、partial_sum 部分和、adjacent_difference 相邻差、inner_product 内积

```
vector<int> v = {3, 1, 4, 1, 5};
sort(v.begin(), v.end());
int n = count(v.begin(), v.end(), 1);   // 1 的个数：2
v.erase(remove_if(v.begin(), v.end(),
          [](int x){ return x < 2; }), v.end());
```

- **STL 算法是通用的**：使用迭代器类型而非仅能用于一种数据结构；算法从迭代器获得元素，迭代器知道元素在容器中的位置（外科医生与助手比喻）；用户可在自定义数据结构应用这些算法（只需迭代器类型满足算法要求）；几乎所有算法头文件都是 <algorithm>
- **STL 算法分 4 大类**：不可变序列算法、可变序列算法、排序和搜索算法、数值算法
- **不可变序列算法（表 10-6）**：不修改容器内容——for_each（对每个元素进行某操作）、find、find_if（找首个 pred(x) 为真的元素）、adjacent_find（相邻重复）、find_first_of、find_end、count、count_if、mismatch（找两个序列第一个不匹配点）、equal（判断两区间相等）、search（找子序列）、search_n（连续 n 次）
- **可变序列算法（表 10-7）**：copy（三种：copy、copy_n、copy_backward）、fill/fill_n（填相同值）、generate/generate_n（调用函数对象的返回值填充）、partition/stable_partition（满足关系的元素放到不满足的之前，stable 保持相对次序）、rotate、unique/unique_copy（删除连续相等元素）、random_shuffle、remove/remove_if（注意不真正删除容器元素，只是后置）、replace/replace_if、reverse/reverse_copy、swap/iter_swap/swap_ranges、transform
- **注意**：可变序列算法可能改变序列元素的值，而迭代器所指向的位置可能并不可用，对不可用位置修改元素会引起访问出错，更可能导致程序崩溃
- **copy 细节**：当两个区间属于同一容器且目标头位于源区间 [first1,last1) 中时，必须使用 copy_backward——copy 从头开始复制会覆盖掉原先的值；fill 和 generate 的区别：fill 填相同数值，generate 调用指定的产生函数逐一填充
- **排序和搜索算法分类**：排序（sort、partial_sort、partial_sort_copy、stable_sort）；4 个二分搜索（binary_search、lower_bound、upper_bound、equal_range）；2 个合并（merge、inplace_merge）；4 个最值（min、max、min_element、max_element）；3 个排列（lexicographical_compare、next_permutation、prev_permutation）；5 个有序序列集合操作（includes、set_union、set_intersection、set_difference、set_symmetric_difference）；4 个堆操作（make_heap、pop_heap、push_heap、sort_heap）；另有 nth_element
- **sort 两种重载**：void sort(RandomIt first, RandomIt last)；void sort(RandomIt first, RandomIt last, UnaryPredicate comp)；第一种从小到大，第二种可指定比较函数（元素有多个字段时可按不同字段排序）；sort 要求随机访问迭代器（实现使用快速排序）
- **stable_sort**：原型与 sort 几乎相同，不同在于相等数值的元素排序前后相对位置保持不变；可用于多关键字同时排序的场合（字段 1 相等时按字段 2 排）；只要求双向迭代器（内部结构复杂的容器随机迭代器效率较低时是很好的选择）；一般情况下 stable_sort 的效率比 sort 要低

- **数值算法（<numeric>，4 个）：**accumulate（所有元素的和）、partial_sum（部分和 b_i=Σa_j，则区间 [j,k) 的和为 b_{k-1}-b_{j-1}，一次减法完成）、adjacent_difference（相邻元素的差，相当于 partial_sum 的反运算）、inner_product（两个序列对应元素乘积之和，内积）；partial_sum 第二种形式可指定运算函数（如乘法代替加法）

第 11 章 流类库与输入输出

11.1 I/O 流的概念

- C++ 无输入输出语句，I/O 流类库是 C 语言 I/O 函数在面向对象方法中的替换产品
- 流是 I/O 流类的中心概念；从流中获取数据 = 提取操作（读），向流中添加数据 = 插入操作（写）
- 流是一种抽象，负责在数据生产者和消费者之间建立联系并管理数据的流动；程序建立流对象并指定它与文件对象连接
- 操作系统将键盘、屏幕、打印机、通信端口作为扩充文件处理
- I/O 流类库基础是一组类模板；流的基本单位除 char 外还可以是 wchar_t；标准头文件已用 typedef 为 char 实例定义别名，可直接把这些别名看作流类的类名
- C++ 语言中没有输入输出语句，但标准库中有面向对象的输入输出软件包——I/O 流类库；流是 I/O 流类的中心概念
- **两个对象与流的抽象：**程序与外界环境进行信息交换时存在两个对象——一个是程序中的对象，另一个是文件对象；流是一种抽象，它负责在数据的生产者和数据的消费者之间建立联系，并管理数据的流动；程序建立一个流对象并指定它与某个文件对象建立连接，程序操作流对象，流对象通过文件系统对所连接的文件对象产生作用；流对象是程序中的对象与文件对象进行交换的界面，程序将流对象看作是文件对象的化身
- 操作系统将键盘、屏幕、打印机和通信端口作为扩充文件来处理，这种处理通过操作系统的设备驱动程序实现；从 C++ 程序员角度看，这些设备与磁盘文件等同，与设备的交互也是通过 I/O 流类实现的
- **流涉及的范围不止于此：**凡是数据从一个地方传输到另一个地方的操作都是流操作，如网络数据交换、进程数据交换；一般意义下的读操作在流数据抽象中被称为（从流中）提取，写操作被称为（向流中）插入
- **I/O 流类库的基础是一组类模板：**类模板提供了库中的大多数功能，可作用于不同类型的元素；流的基本单位除普通字符（char）外还可以是其他类型（如 wchar_t）；流的基本单位的数据类型就是模板的参数；C++ 标准头文件中已用 typedef 为这些模板面向 char 类型的实例定义了别名，可直接把这些别名看作流类的类名
- **I/O 流类层次：**ios（流基类）、istream（输入流基类）、ostream（输出流基类）、iostream（输入输出流基类）、ifstream/ofstream/fstream（文件流）、istringstream/ostringstream/stringstream（字符串流）、streambuf（流缓冲区基类）、filebuf/stringbuf
- **预定义输出流对象：**cout 标准输出流；cerr 标准错误输出流（无缓冲、立即输出）；clog（有缓冲、缓冲区满时输出）；重定向：cout<< 可把标准输出重定向到文件，2> 重定向标准错误

```
// 提取与插入
int x;  cin>>x;          // 提取（读）
cout<<x;               // 插入（写）
cerr<<"error";      // 无缓冲立即输出
clog<<"log";        // 有缓冲
```

11.2 输出流

- **流类层次：**ios（流基类）/ istream / ostream / iostream / ifstream / ofstream / fstream（文件流）/ istringstream / ostringstream / stringstream（字符串流）/ streambuf（流缓冲区基类）/ filebuf / stringbuf
- **预定义输出流对象：**cout 标准输出流；cerr 标准错误输出流（无缓冲、立即输出）；clog（有缓冲、缓冲区满时输出）
- **重定向：**">" 把标准输出重定向到文件；"2>" 重定向标准错误

11.2.1 构造输出流对象

- **方法①：**默认构造 + open 成员函数 ofstream file; file.open("filename");
- **方法②：**构造函数指定文件名 ofstream file("filename");
- **方法③：**同一流对象先后打开不同文件（须先 close）

11.2.2 插入运算符与操纵符

- 插入 << 运算符为所有标准数据类型预先设计
- **操纵符 manipulator：**定义在 ios_base 类（如 hex()）和 iomanip 头文件（如 setprecision）
- width 成员函数 / setw 操纵符：为每个输出项指定宽度；仅影响紧随其后的域，域输出完恢复默认；不截断数值（超出宽度显示全部值）
- **fill 成员函数：**设置填充字符，默认空格
- 输出流默认右对齐
- setiosflags(ios_base::left) 设置左对齐，影响持久（不同于 setw 只对一次有效），须 resetiosflags 恢复
- **格式标志位：**skipws / left / right / internal / dec / oct / hex / showbase / showpoint / uppercase / showpos / scientific / fixed / unitbuf
- 浮点数输出精度默认 6 位有效数字，setprecision 改变；fixed 定点、scientific 科学格式

11.2.3 文件输出流成员函数

- **文件输出流成员函数 3 种类型：**①与操纵符等价的成员函数 ②执行非格式化写操作的成员函数 ③其他修改流状态的成员函数
- 顺序格式化输出可仅用插入运算符和操纵符；随机访问二进制磁盘输出使用其他成员函数
- **open 函数：**ofstream file("filename", ios_base::out | ios_base::binary); 第二个参数默认 ios_base::out 可省略
- **打开模式（表 11-2）：**app（文件尾添加）、ate（打开现存文件并查找到结尾）、in（输入；对 ofstream 用 in 可避免删除现存内容）、out（输出，隐含指定）、trunc（已存在则删除原有内容）、binary（二进制模式，默认文本模式）
- **close 函数：**关闭文件输出流关联的磁盘文件；文件使用完毕后必须关闭（ofstream 析构函数虽自动完成）
- **put 函数：**cout.put('A'); 精确输出一个字符（不受宽度和填充方式影响）
- **write 函数：**把内存一块内容写入文件，长度参数指出写字节数；遇空字符不停止，可写完整类结构；需 reinterpret_cast 把对象地址显式转 char*：file.write(reinterpret_cast<char *>(&dt), sizeof(dt));
- **seekp 和 tellp：**文件输出流内部指针指出下一次写数据位置；seekp 设置该指针（随机方式向磁盘输出），tellp 返回文件位置指针值
- **错误处理成员函数（表 11-3）：**bad()（不可恢复错误非 0）、fail()（不可恢复错误或预期条件如转换错误/文件未找到）、good()（所有错误标记和文件结束标记都清除则非 0）、eof()（遇到文件结尾条件非 0）、clear()（默认参数调用清除所有错误位）、rdstate()（返回当前错误状态）
- **"!" 运算符重载：**if(!cout) 等价于 if(cout.fail())
- **void*() 重载：**if(cout) 等价于 if(!cout.fail())，但不检测文件结尾

11.2.4 二进制输出文件

- 默认文本方式输出；Linux 行分隔符 \\n，Windows \\n+\\r；文本模式每输出 \\n 会写入操作系统行分隔符（Windows 下自动扩充回车符）
- **例：**IA-32 下 int 10 的 4 字节内容为 10、0、0、0，文本模式写 \\n 会变成 10、13、0、0、0 → 需二进制模式：打开模式设 ios_base::binary，所写字符不转换

11.2.5 字符串输出流 ostringstream

- 用于生成字符串；两个构造函数——第一个形参是流的打开模式（默认 ios_base::out），第二个接收 string 型常对象为内容初值+打开模式
- 与 ofstream 同为 ostream 派生类，有插入运算符、操纵符、write 函数、各种控制格式成员函数，但没有 open/close
- **特有函数 str()：**返回 string 对象（用该输出流生成字符串的内容）
- 典型用法 = 数值转字符串（toString 模板函数）
- **构造输出流对象 3 法：**①默认构造+open 成员函数；②构造函数指定文件名；③同一流对象先后打开不同文件（须先 close）

- **操纵符 manipulator**：定义在 ios_base 类（如 hex()）和 iomanip 头文件（如 setprecision）；插入 << 运算符为所有标准数据类型预先设计
- width 成员函数 / setw 操纵符：为每个输出项指定宽度；仅影响紧随其后的域，域输出完恢复默认宽度；不截断数值（超出宽度显示全部值）；fill 成员函数设置填充字符（默认空格）；输出流默认右对齐
- setiosflags(ios_base::left) 设置左对齐，影响持久（不同于 setw 只对一次有效），须 resetiosflags 恢复；格式标志位：skipws/left/right/internal/dec/oct/hex/showbase/showpoint/uppercase/showpos/scientific/fixed/unitbuf
- 浮点数输出精度默认 6 位有效数字，setprecision 改变；fixed 定点、scientific 科学格式
- **文件输出流成员函数 3 种类型**：①与操纵符等价的成员函数；②执行非格式化写操作的成员函数；③其他修改流状态且不同于操纵符或插入运算符的成员函数；顺序格式化输出可仅用插入运算符和操纵符；随机访问二进制磁盘输出使用其他成员函数
- **open 函数**：ofstream file("filename", ios_base::out | ios_base::binary); 第二个参数默认 ios_base::out 可省略；打开模式（表 11-2）：app（文件尾添加）、ate（打开现存文件并查找到结尾）、in（输入；对 ofstream 用 in 可避免删除现存内容）、out（输出，隐含指定）、trunc（已存在则删除原有内容）、binary（二进制模式，默认文本模式）
- **close 函数**：关闭文件输出流关联的磁盘文件；文件使用完毕后必须关闭（ofstream 析构函数虽自动完成）
- **put 函数**：cout.put('A'); 精确输出一个字符（与 cout<<'A' 默认相同，但后者受宽度和填充方式影响）
- **write 函数**：把内存一块内容写入文件输出流，长度参数指出写字节数；遇空字符不停止，可写完整类结构；file.write(reinterpret_cast<char*>(&dt), sizeof(dt));——需用 reinterpret_cast 把对象地址显式转 char*
- **seekp 和 tellp**：文件输出流内部指针指出下一次写数据位置；seekp 设置该指针（可随机方式向磁盘输出），tellp 返回文件位置指针值
- **错误处理成员函数（表 11-3）**：bad()（不可恢复错误非 0）、fail()（不可恢复错误或预期条件如转换错误/文件未找到；clear 后若错误标记清除则非 0）、good()（所有错误标记和文件结束标记都清除则非 0）、eof()（遇到文件结尾条件非 0）、clear()（设置内部错误状态，默认参数调用清除所有错误位）、rdstate()（返回当前错误状态）；！运算符重载：if(!cout) 等价于 if(cout.fail())；void*() 重载：if(cout) 等价于 if(!cout.fail())（但不检测文件结尾）
- **二进制输出文件**：默认文本方式输出；Linux 行分隔符 \n，Windows \n+\r；文本模式每输出 \n 会写入操作系统行分隔符（Windows 下自动扩充回车符）；IA-32 下 int 10 的 4 字节内容为 10、0、0、0，文本模式写 \n 会变成 10、13、0、0、0 → 需二进制模式（打开模式设 ios_base::binary，所写字符不转换）
- 字符串输出流 ostringstream：用于生成字符串；与 ofstream 同为 ostream 派生类，有插入运算符、操纵符、write 函数、各种控制格式成员函数，但没有 open/close；特有函数 str()：返回 string 对象（用该输出流生成字符串的内容）；典型用法=数值转字符串

```
ofstream file("data.bin", ios_base::out | ios_base::binary);
file.write(reinterpret_cast<char*>(&dt), sizeof(dt));
file.close();
cout<<setw(10)<<setfill('*')<<left<<3.14;  // 宽度 10 左对齐 填充*
cout<<setprecision(2)<<fixed<<3.14159;    // 3.14
ostringstream oss;  oss<<42;  string s=oss.str();  // "42"
file.seekp(3*sizeof(int));  // 定位写指针
```

11.3 输入流

- **3 个最重要输入流类**：istream、ifstream、istringstream；istream 类最适合顺序文本模式输入
- cin 完成标准输入设备（键盘）输入；标准输入可在命令行重定向 "<"，数据从文件读入
- ifstream 类支持磁盘文件输入，可指定二进制或文本模式；构造函数指定文件名则构造时自动打开，否则默认构造后用 open 打开；基类 ios 和 istream 所有功能都包含在 ifstream 中

11.3.1 构造输入流对象

- 仅使用 cin 则不需构造；读文件需构造
- **方法①**：默认构造 + open：ifstream myFile; myFile.open("filename");
- **方法②**：构造时指定文件名：ifstream myFile("filename");（C 风格字符串数组文件名）
- C++11 可用标准 string 文件名：string filename="file.txt"; ifstream myFile2(filename);

11.3.2 使用提取运算符

- 提取运算符 >> 对所有标准 C++ 数据类型预先设计好
- 格式化文本输入，以空白符为分隔
- 输入含空格文本不便 → 非格式化输入或 getline 读包含空格的文本块再分析

11.3.3 输入流操纵符

- 只有少数操纵符对输入有实际影响（最重要的是进制操纵符 dec、oct、hex）
- hex 接收处理各种输入流格式；序列 124n5 变换成数值 124（除 0~9、A~F、a~f、X 外字符引起变换终止，使 fail 返回 true）

11.3.4 输入流相关函数

- **open 函数**：ifstream 打开文件模式默认 ios_base::in；binary 以二进制模式打开（默认文本模式）
- **close 函数**：析构自动关闭，但同一流对象打开另一文件需先用 close 关闭当前文件
- **get 函数**：非格式化，与 >> 相似但读入数据时包括空白字符（>> 缺省拒绝空白字符）；while((ch=cin.get())!=EOF) cout.put(ch);——Ctrl+Z+Enter 读入 EOF 程序结束
- **getline 函数**：istream 成员函数 getline 允许从读取内容中删除终止字符（默认换行），但只能存入字符数组（大小不能自动扩展）
- 非成员函数 getline 可存到 string 对象更方便：getline(cin, line, 't')（第三个可选参数 = 终止字符），声明在 string 头文件
- **read 函数**：从文件读字节到指定存储器区域，长度参数确定读字节数；遇到文件结束或文本模式遇文件结束标记字符时读结束
- **二进制记录读写（例 11-9）**：ofstream os("payroll", ios_base::out | ios_base::binary); os.write(reinterpret_cast<char*>(&e1), sizeof(e1)); ifstream is("payroll", ios_base::in | ios_base::binary); if(is) is.read(reinterpret_cast<char*>(&e2), sizeof(e2));——数据记录严格格式化、无终止回车换行
- **seekg 和 tellg**：文件输入流内部指针指向下一个将读数据位置；seekg 设置该指针（例：is.seekg(3*sizeof(int)) 定位后 read 读第 4 个整数），tellg 返回当前读指针位置（streampos 类型）
- **面向记录的数据管理**：固定长度记录尺寸 × 记录号得相对文件末尾字节位置，再 get 读记录

11.3.6 字符串输入流 istringstream

- 从字符串中读取数据；两个构造函数（一个 string 对象 + 打开模式，默认 ios_base::in）；常用创建：string str=...; istringstream is(str);
- 有提取运算符、操纵符、read、getline 等；无 open/close
- 典型用法 = 字符串转数值（fromString 模板：istringstream is(str); T v; is>>v; return v;）——把支持 ">>" 提取符的类型的字符串表示转为该类型数据
- 输入流对象是数据流出的源头；3 个最重要输入流类 istream、ifstream、istringstream；istream 类最适合顺序文本模式输入；cin 完成标准输入设备输入；标准输入可在命令行重定向 <，数据从文件读入
- ifstream 类支持磁盘文件输入，可指定二进制或文本模式；构造函数指定文件名则构造时自动打开，否则默认构造后用 open 打开；基类 ios 和 istream 的所有功能都包含在 ifstream 中
- **构造输入流对象**：①默认构造+open：ifstream myFile; myFile.open("filename"); ②构造时指定文件名和模式（C++11 起可直接用 string 作文件名）：ifstream myFile2(filename);
- **提取运算符 >>**：对所有标准 C++ 数据类型预先设计好；格式化文本输入，以空白符为分隔；输入含空格文

本不便 → 用非格式化输入或 getline 读包含空格的文本块再分析

- **输入流操纵符**：定义在 ios_base 类和 iomanip 头文件中的操纵符可应用于输入流，但只有少数有实际影响（最重要的是进制操纵符 dec、oct、hex）；hex 接收处理各种输入流格式；序列 124n5 变换成数值 124（除 0~9、A~F、a~f、X 外字符引起变换终止，使 fail 返回 true）
- **get 函数**：非格式化，与 >> 相似但读入数据时包括空白字符（>> 缺省拒绝空白字符）；while((ch=cin.get())!=EOF) cout.put(ch);——Ctrl+Z+Enter 读入 EOF 程序结束
- **getline 函数**：istream 成员函数 getline 允许从读取内容中删除终止字符（默认换行），但只能存入字符数组（大小不能自动扩展）；非成员函数 getline 可存到 string 对象更方便：getline(cin, line, 't')（前两个参数=输入流+string 对象，第三个可选=终止字符），声明在 string 头文件
- **read 函数**：从文件读字节到指定存储器区域，长度参数确定读字节数；遇到文件结束或文本模式遇文件结束标记字符时读结束；二进制记录读写：os.write(reinterpret_cast<char*>(&e), sizeof(e)); ifstream is("payroll", ios_base::in | ios_base::binary); is.read(reinterpret_cast<char*>(&e2), sizeof(e2));
- **seekg 和 tellg**：文件输入流保留指向文件中下一个将读数据位置的内部指针；seekg 设置该指针（is.seekg(3*sizeof(int)) 定位后 read 读第 4 个整数），实现面向记录的数据管理：固定长度记录尺寸×记录号得相对文件开头字节位置，再 get 读记录；tellg 返回当前文件读指针位置（streampos 类型）
- 字符串输入流 istringstream：从字符串中读取数据；常用创建：string str=…; istringstream is(str); 有提取运算符、操纵符、read、getline 等；无 open/close；典型用法=字符串转数值：istringstream is(str); T v; is>>v; return v;
- fstream 类支持磁盘文件输入和输出：可读可写，对象含两个逻辑子流（一个输入一个输出）；stringstream 类支持面向字符串的输入输出：对同一字符串内容交替读写，由两个逻辑子流构成

```
ifstream in("data.txt");          // 构造时打开
if(!in) { cerr<<"open failed"; }
string line;
while(getline(in, line)) { /* 读整行 */ }
char ch;  while((ch=cin.get())!=EOF) cout.put(ch);
istringstream is("42 3.5");
int n; double d;  is>>n>>d;           // 字符串转数值
is.seekg(3*sizeof(int));
is.read(reinterpret_cast<char*>(&x), sizeof(x));
Date dt;  cin>>dt;                    // 重载 >> 后各种输入流通用
```

11.4 输入输出流

- iostream 对象可是数据的源或目的
- fstream 类支持磁盘文件输入和输出（可读可写），对象含两个逻辑子流（一个输入一个输出）
- stringstream 类支持面向字符串的输入输出：对同一字符串内容交替读写；两个逻辑子流构成
- fstream 和 stringstream 从 iostream 派生